

UNIVERSITÉ PARIS 1
PANTHÉON SORBONNE

(Software) Product Line Engineering

Camille Salinesi

Camilo Correa Restrepo



1. Analysis

A….

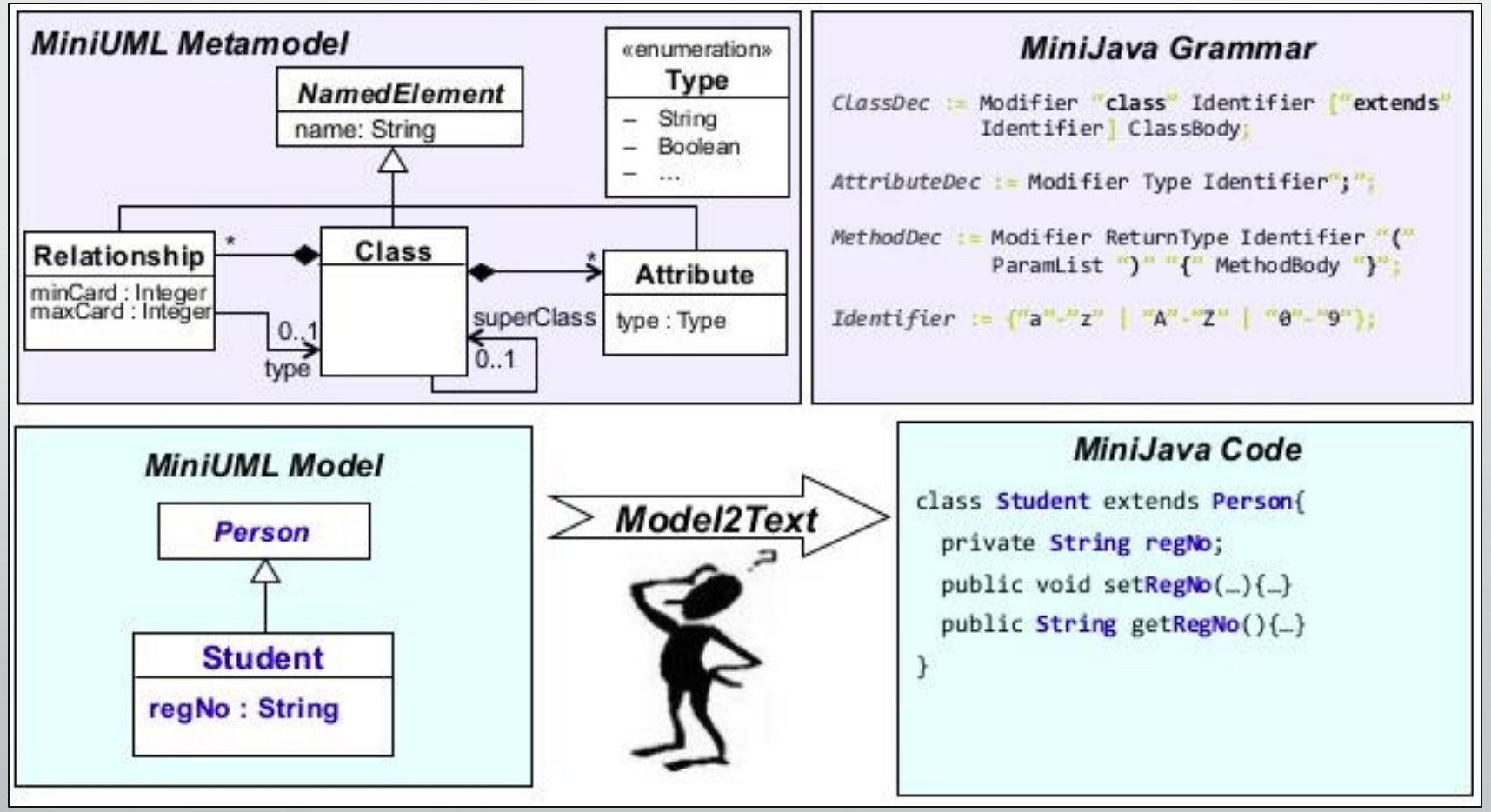
B….

C. Transformation, verification, diagnosis and fixing processes

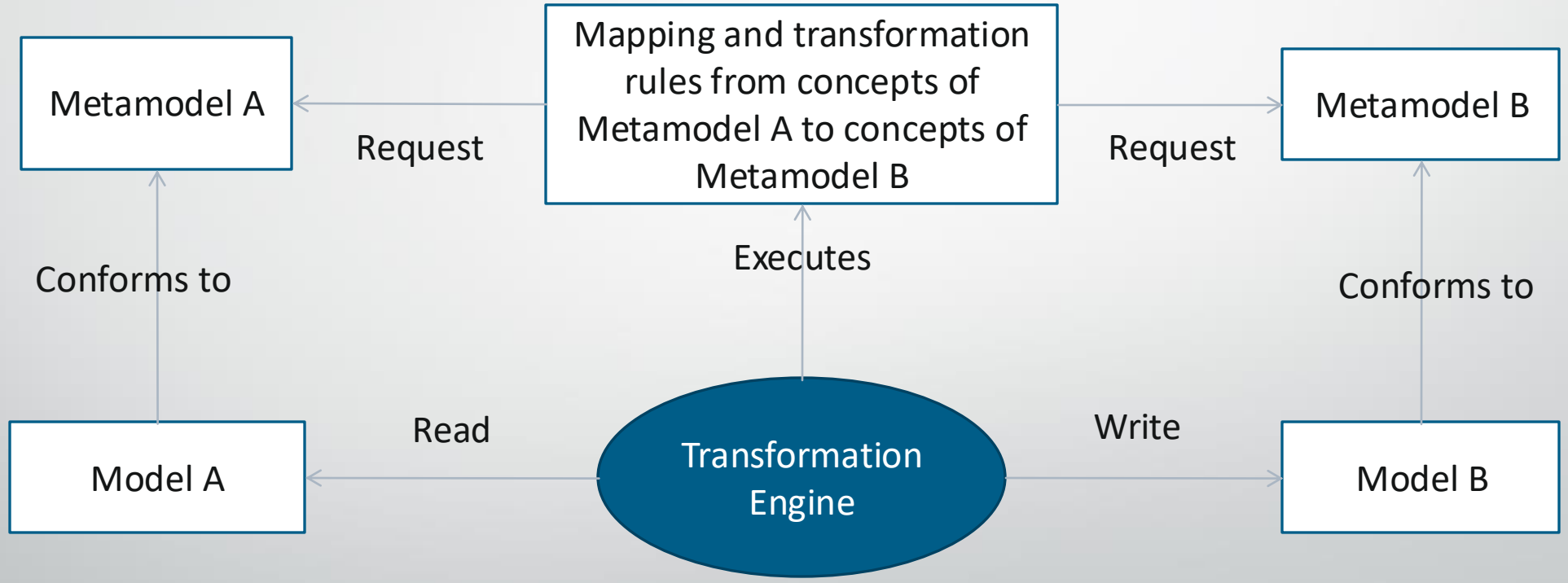
What is model-to-text transformation?

Model transformation aims to provide a mean to specify the way to produce target models from a number of source models.

Formally, a simple model transformation has to define the way for generating a model **Mb**, conforming to a metamodel **MMb**, from a model **Ma** conforming to a metamodel **MMa**.



Tranformation framework



Why to transform a PLM?

Graphical representation of models makes it impossible to directly reason on them. Because of this, models are usually transformed to executable code in order **to reason on them.**

In the case of PLMs:

Batory, D.S., 2005.

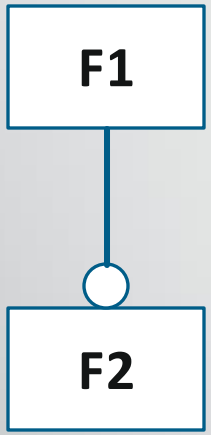
Benavides et al. 2005, 2007, 2010

Van Deursen, A., Klint, P., 2002

Karataş et al. 2010

Mazo et al. 2011

Transforming FMs to constraint programming: the **optional** concept



If {F1, F2} ∈ {true, false}: then

- $F2 \Rightarrow F1$
- $\neg F2 \vee F1$

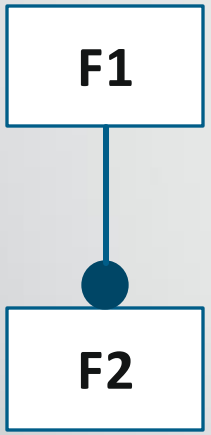
If {F1, F2} ∈ {0, 1}: then

$F1 \geq F2$

If {F1, F2} ∈ N^+ U {0}: then

$F2 \geq 1 \Rightarrow F1 \geq 1$

Transforming FMs to constraint programming: the **mandatory** concept



If {F1, F2} ∈ {true, false}: then

- F1 ⇔ F2
- (¬F1 ∨ F2)^(F1 ∨ ¬F2)

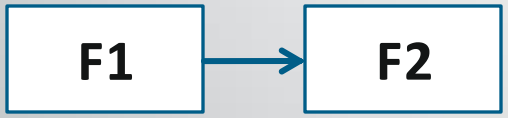
If {F1, F2} ∈ {0, 1}: then

F1 = F2

If {F1, F2} ∈ N^+ U {0}: then

F2 ≥ 1 ⇔ F1 ≥ 1

Transforming FMs to constraint programming: the **requires** concept



If $\{F1, F2\} \in \{true, false\}$: then

- $F1 \Rightarrow F2$
- $\neg F1 \vee F2$

If $\{F1, F2\} \in \{0, 1\}$: then

- $(1-F1 + F2) > 0$
- $F2 \geq F1$

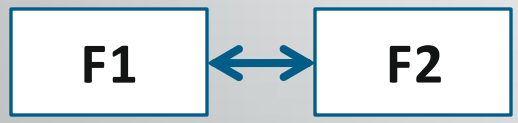
If $\{F1, F2\} \in N^{+} \cup \{0\}$: then

$F1 \geq 1 \Rightarrow F2 \geq 1$

Transforming FMs to constraint programming: the **excludes** concept

If $\{F1, F2\} \in \{true, false\}$ then

- $(F1 \wedge \neg F2) \vee (\neg F1 \wedge F2) \vee (\neg F1 \wedge \neg F2)$
- $\neg(F1 \wedge F2)$
- $\neg F1 \vee \neg F2$



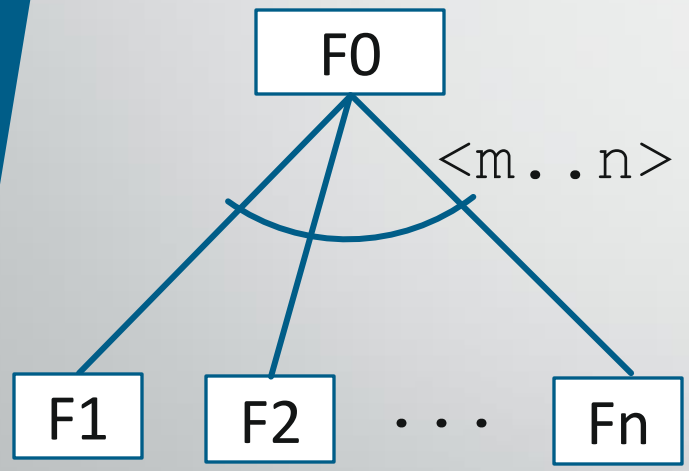
If $\{F1, F2\} \in \{0, 1\}$ then

$F1 + F2 \leq 1$

If $\{F1, F2\} \in N^{+} \cup \{0\}$: then

$F1 * F2 = 0$

Transforming FMs to constraint programming: **Group cardinality with <m..n>boundaries**



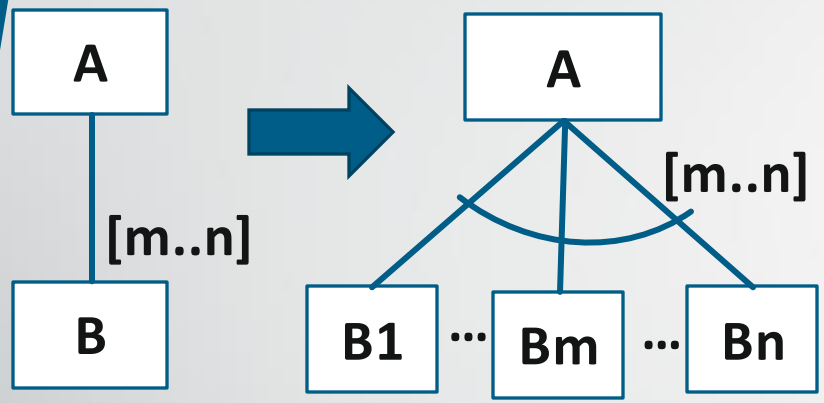
If {F0, F1, F2, …, Fn} ∈ {0, 1} then
F0 ≥ F1 ∧
F0 ≥ F2 ∧ …
F0 ≥ Fn ∧
F0 ==> (m ≤ F1+ F2 + … + Fn ≤ n)

Or simply:

m*F0 ≤ F1+ F2 + … + Fn ≤ n*F0

If {F0, F1, F2, …, Fn} ∈ N^+ U {0} then
(F0 > 0) ⇔ BoolF0 ∧
(F1 > 0) ⇔ BoolF1 ∧ … ∧ (Fn > 0) ⇔ BoolFn ∧
m*BoolF0 ≤ BoolF1 + … + BoolFn ≤ n*BoolF0

Transforming FMs to constraint programming: **individual cardinalities**



If {A, B1, …, Bn} ∈ {0, 1} then:

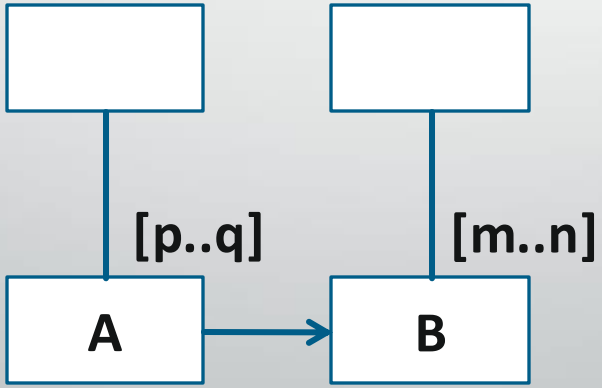
A ≥ B1 ∧ ...∧ A ≥ Bn ∧

A ==> (m ≤ B1 + … + Bn ≤ n)

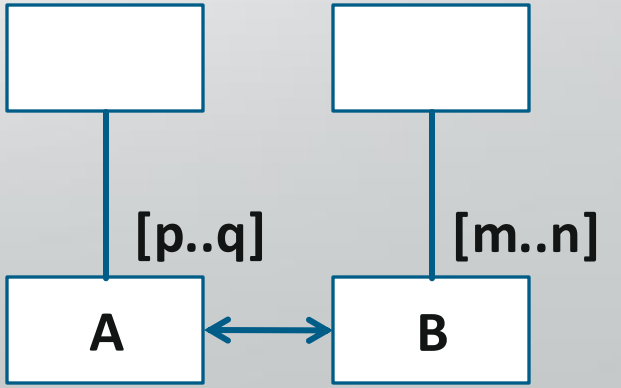
Or simply:

A <==> (m ≤ B1 + … + Bn ≤ n)

Exercise 1:



Exercise 2:



Transforming FMs to constraint programming: **feature attributes**

F1
Att1: {value1} Att2: {value2, value3}

$$F1 \in \{true, false\} \wedge$$
$$Att1 \in \{value1, \ldots\} \wedge$$
$$F1 \Leftrightarrow (Att1 > 0)$$

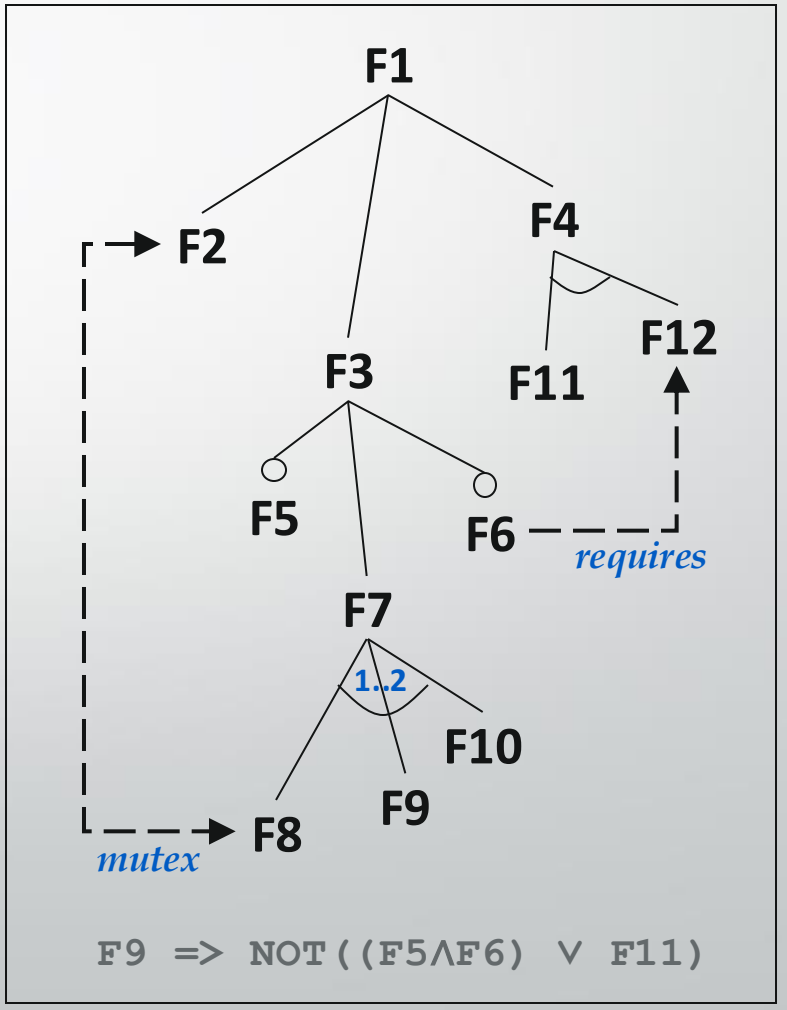
Example:

$$Att2 \in \{0, 2, 4, 16, 32\} \wedge$$
$$F1 \Leftrightarrow (Att2 > 0)$$

Exercise : with GNU Prolog

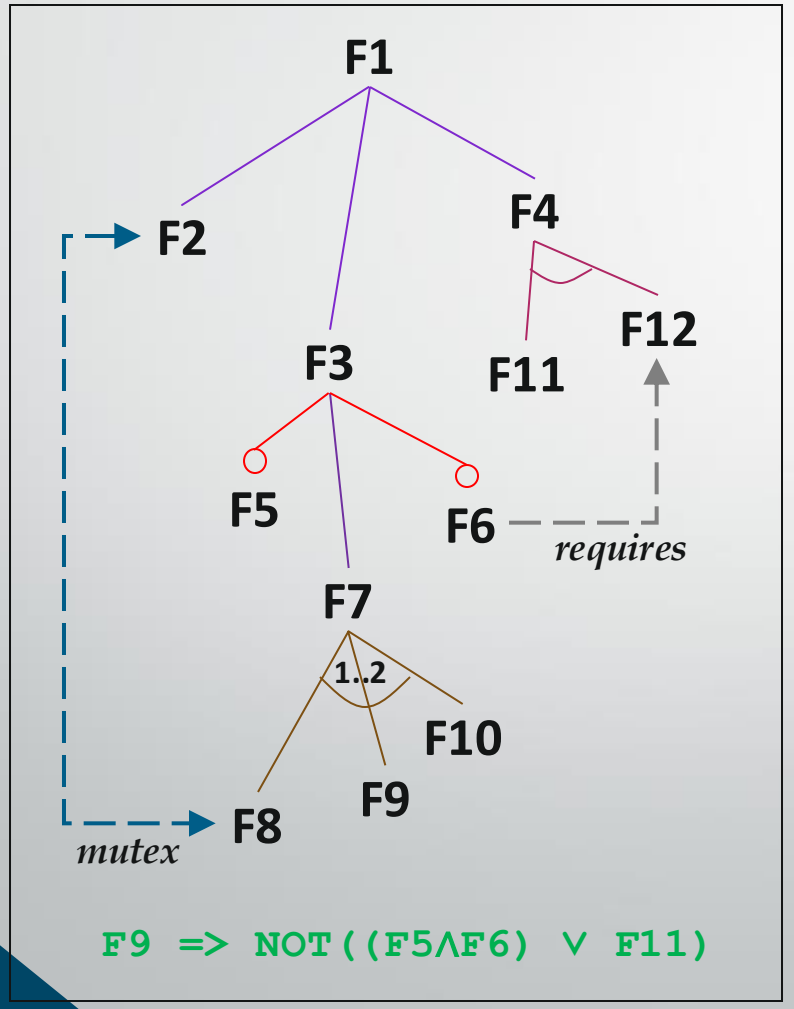
Let's transform the following model into a constraint program in GNU Prolog, and:

1. What is the resulting code?
2. How to generate some configurations automatically?
3. Which is the semantics of the model?
4. How many products in the model?
5. Are there some problems in the model? Which ones?



1. What is the resulting code?

Match the colors



productLine.pl

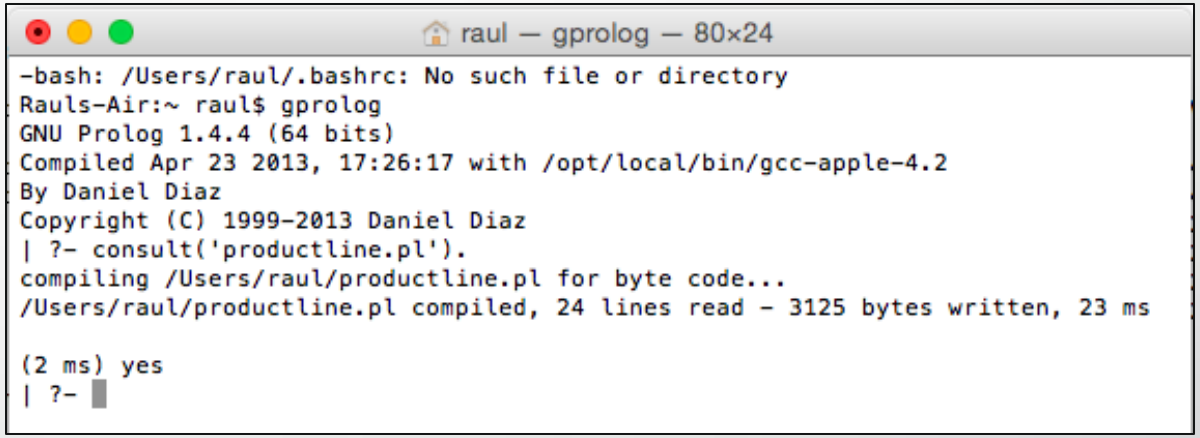
```
productline(L):-
% L is the list of variables of the model
L = [F1, F2, F3, F4, F5, F6, F7, F8, F9, F10, F11, F12],
% this function defines the domain L; all variables can be 0 or 1
fd_domain(L, 0, 1),
F1 #= 1,
F1 #= F2,
F1 #= F3,
F1 #= F4,
F3 #= F7,
F3 #>= F5,
F3 #>= F6,
F7 #=< F8 + F9 + F10,
F8 + F9 + F10 #=< F7 * 2,
F4 #>= F11,
F4 #>= F12,
F4 #= F11 + F12,
F2 + F8 #=< 1,
F6 #==> F12,
F9 #==> #\((F5 #/\ F6) #\/ F11),
%to finish the constraint program and give a value to each variable
fd_labeling(L).
```

2. How to generate some configurations automatically?

```
productline(L):-
% L is the list of variables of the model
L = [F1, F2, F3, F4, F5, F6, F7, F8, F9, F10, F11, F12],
fd_domain(L, 0, 1),
F1 #= 1,
F1 #= F2,
F1 #= F3,
F1 #= F4,
F3 #>= F5,
F3 #= F7,
F3 #>= F6,
F7 #=< F8 + F9 + F10,
F8 + F9 + F10 #=< F7 * 2,
F4 #>= F11,
F4 #>= F12,
F4 #= F11 + F12,
F2 + F8 #=< 1,
F6 #==> F12,
F9 #==> #\((F5 #/\ F6) #\/ F11),
fd_labeling(L).
```

Step 1: create a file and call it productLine.pl in a plain text editor (the extension should be .pl)

Step 2: start GNU Prolog (gprolog) with double click on the executable file (Win) or executing the **`gprolog`** command in your terminal. Then, compile the productLine.pl file with **`consult`**:



Step 3: call your program with **`productline(L).`** as follows:

```
(2 ms) yes
| ?- productline(L).

L = [1,1,1,1,0,0,1,0,0,1,0,1] ?
```

Step 4: with `;` you can generate more and more solutions:

```
L = [1,1,1,1,0,0,1,0,0,1,0,1] ? ;

L = [1,1,1,1,0,0,1,0,0,1,1,0] ? ;
```

3. Which is the semantics of the model?

There are two ways to generate all the solutions (products) or the semantics of the model:

First way to do it: generating one by one with ; as presented here:

```
| ?- productline(L).

L = [1,1,1,1,0,0,1,0,0,1,0,1] ? ;

L = [1,1,1,1,0,0,1,0,0,1,1,0] ? ;

L = [1,1,1,1,0,0,1,0,1,0,0,1] ? ;

L = [1,1,1,1,0,0,1,0,1,1,0,1] ? ;

L = [1,1,1,1,0,1,1,0,0,1,0,1] ? ;

L = [1,1,1,1,0,1,1,0,1,0,0,1] ? ;

L = [1,1,1,1,0,1,1,0,1,1,0,1] ? ;

L = [1,1,1,1,1,0,1,0,0,1,0,1] ? ;

L = [1,1,1,1,1,0,1,0,0,1,1,0] ? ;

L = [1,1,1,1,1,0,1,0,1,0,0,1] ? ;

L = [1,1,1,1,1,0,1,0,1,1,0,1] ? ;

L = [1,1,1,1,1,1,1,0,0,1,0,1]

(3 ms) yes
| ?-
```

Second way to do it: with the findall function:

```
| ?- findall(L, productline(L), Semantics).

Semantics = [[1,1,1,1,0,0,1,0,0,1,0,1],[1,1,1,1,0,0,1,0,0,1,1,0],[1,1,1,1,0,0,1,0,
1,0,0,1],[1,1,1,1,0,0,1,0,1,1,0,1],[1,1,1,1,0,1,1,0,0,1,0,1],[1,1,1,1,0,1,1,0,1,0,
0,1],[1,1,1,1,0,1,1,0,1,1,0,1],[1,1,1,1,1,0,1,0,0,1,0,1],[1,1,1,1,1,0,1,0,0,1,1,0]
,[1,1,1,1,1,0,1,0,1,0,0,1],[1,1,1,1,1,0,1,0,1,1,0,1],[1,1,1,1,1,1,1,0,0,1,0,1]]

(1 ms) yes
| ?-
```

4. How many products in the model?

There are two ways to know the number of products in the product line:

First way to do it: counting the generated product. Thus, if you count the products just in front you get 12 products.

```
| ?- productline(L).

L = [1,1,1,1,0,0,1,0,0,1,0,1] ? ;

L = [1,1,1,1,0,0,1,0,0,1,1,0] ? ;

L = [1,1,1,1,0,0,1,0,1,0,0,1] ? ;

L = [1,1,1,1,0,0,1,0,1,1,0,1] ? ;

L = [1,1,1,1,0,1,1,0,0,1,0,1] ? ;

L = [1,1,1,1,0,1,1,0,1,0,0,1] ? ;

L = [1,1,1,1,0,1,1,0,1,1,0,1] ? ;

L = [1,1,1,1,1,0,1,0,0,1,0,1] ? ;

L = [1,1,1,1,1,0,1,0,0,1,1,0] ? ;

L = [1,1,1,1,1,0,1,0,1,0,0,1] ? ;

L = [1,1,1,1,1,0,1,0,1,1,0,1] ? ;

L = [1,1,1,1,1,1,1,0,0,1,0,1]

(3 ms) yes
| ?-
```

Second way to do it: you generate all the solutions with findall and you put the answer into a list variable (for instance, called Semanctis), and then you count the number of elements in the resulting list (i.e., Semantics) with the length function:

```
| ?- findall(L, productline(L), Semantics), length(Semantics, NumberOfProducts).

NumberOfProducts = 12
Semantics = [[1,1,1,1,0,0,1,0,0,1,0,1],[1,1,1,1,0,0,1,0,0,1,1,0],[1,1,1,1,0,0,1,0,
1,0,0,1],[1,1,1,1,0,0,1,0,1,1,0,1],[1,1,1,1,0,1,1,0,0,1,0,1],[1,1,1,1,0,1,1,0,1,0,
0,1],[1,1,1,1,0,1,1,0,1,1,0,1],[1,1,1,1,1,0,1,0,0,1,0,1],[1,1,1,1,1,0,1,0,0,1,1,0]
,[1,1,1,1,1,0,1,0,1,0,0,1],[1,1,1,1,1,0,1,0,1,1,0,1],[1,1,1,1,1,1,1,0,0,1,0,1]]

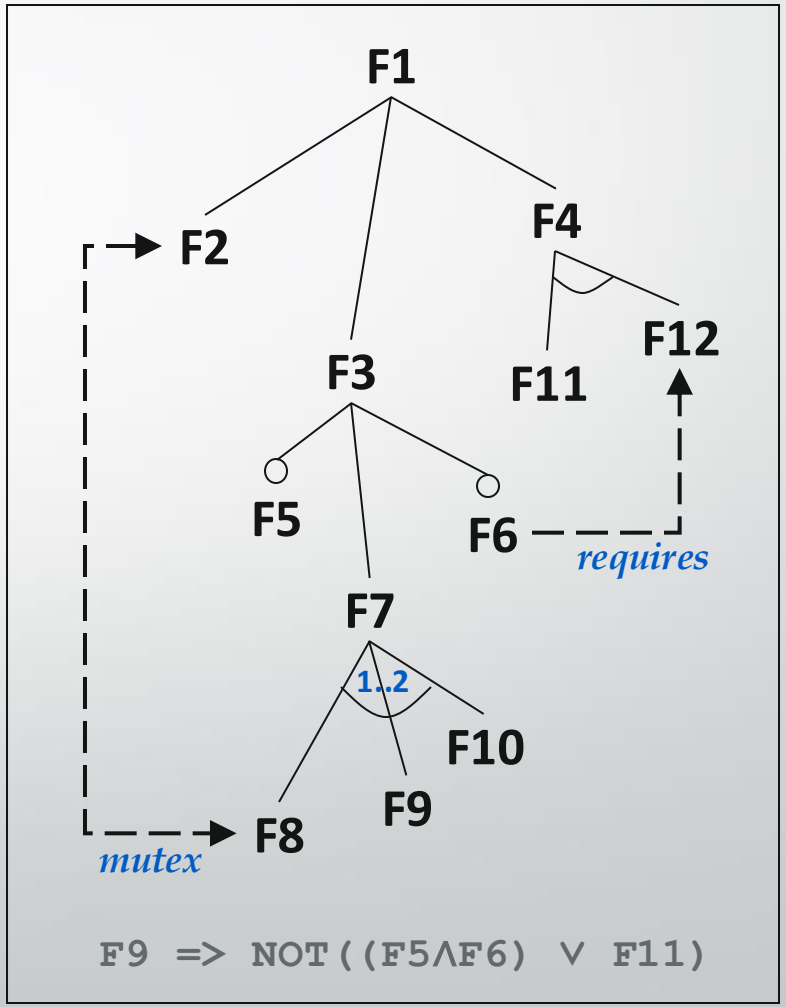
yes
| ?-
```

5. There are some problems in the model? Which ones?

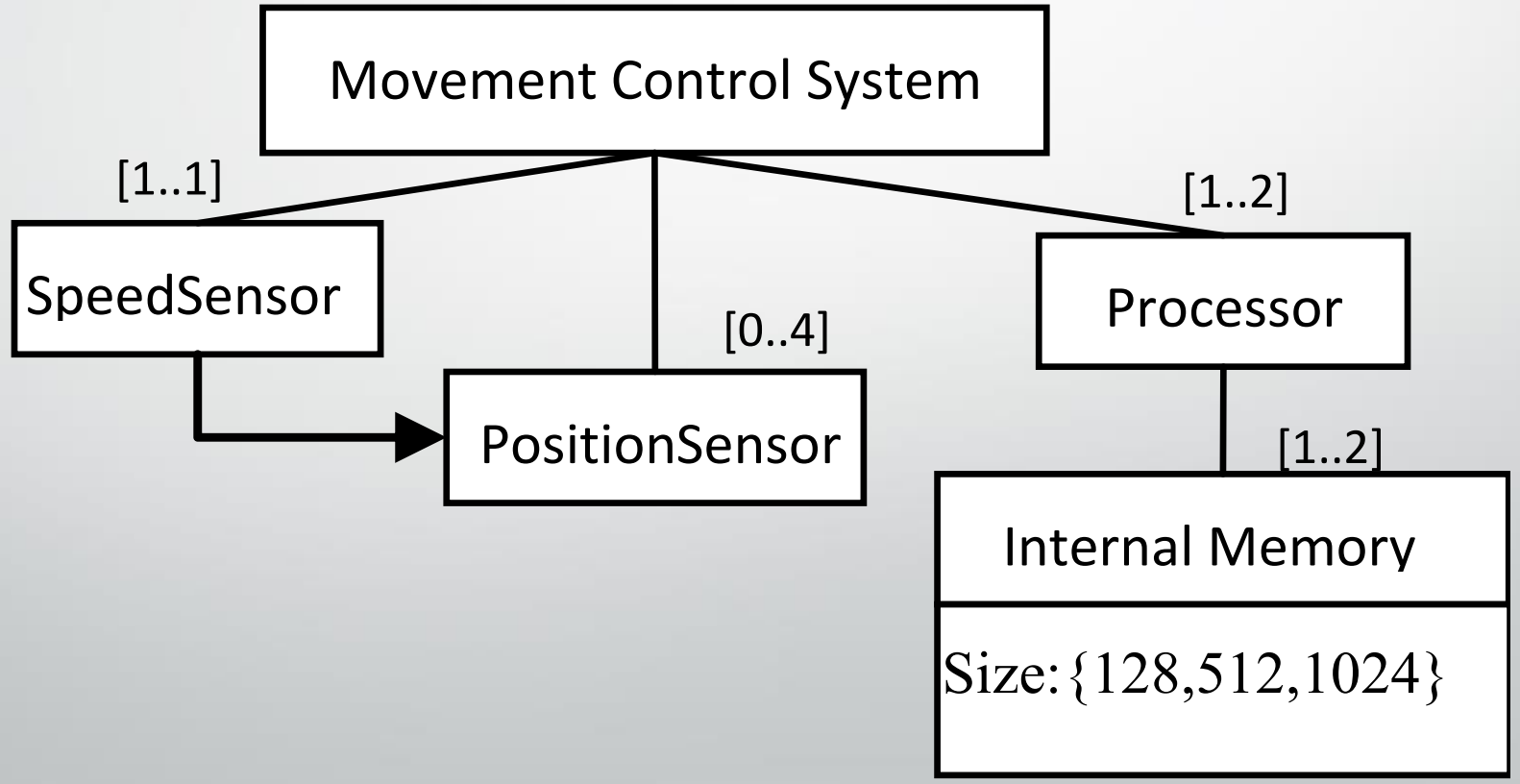
Yes, there are!

For instance, you have maybe noticed that the feature F8 is never selected, in any configuration, right?
Why, because F2 is full mandatory and the F8 is optional, so, F8 has no chances to be selected, never!
So, F8 is called a dead feature because is represented in the product line model but in reality, it is a dead variable because it cannot be used in any product!

This explanation is enough for now, it was just good to introduce the next subject of the course: Verification of product line models!

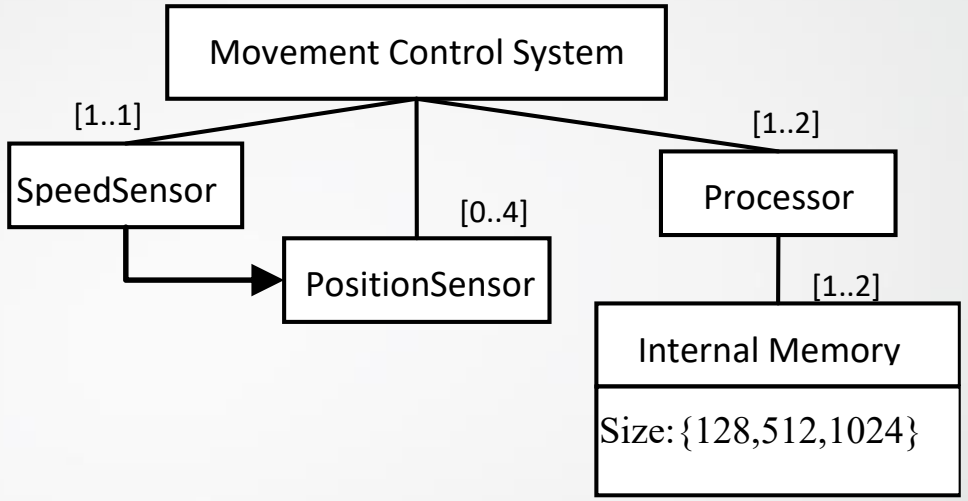


Exercise: represent the following PLM as a high-level (solver-independent) constraint program



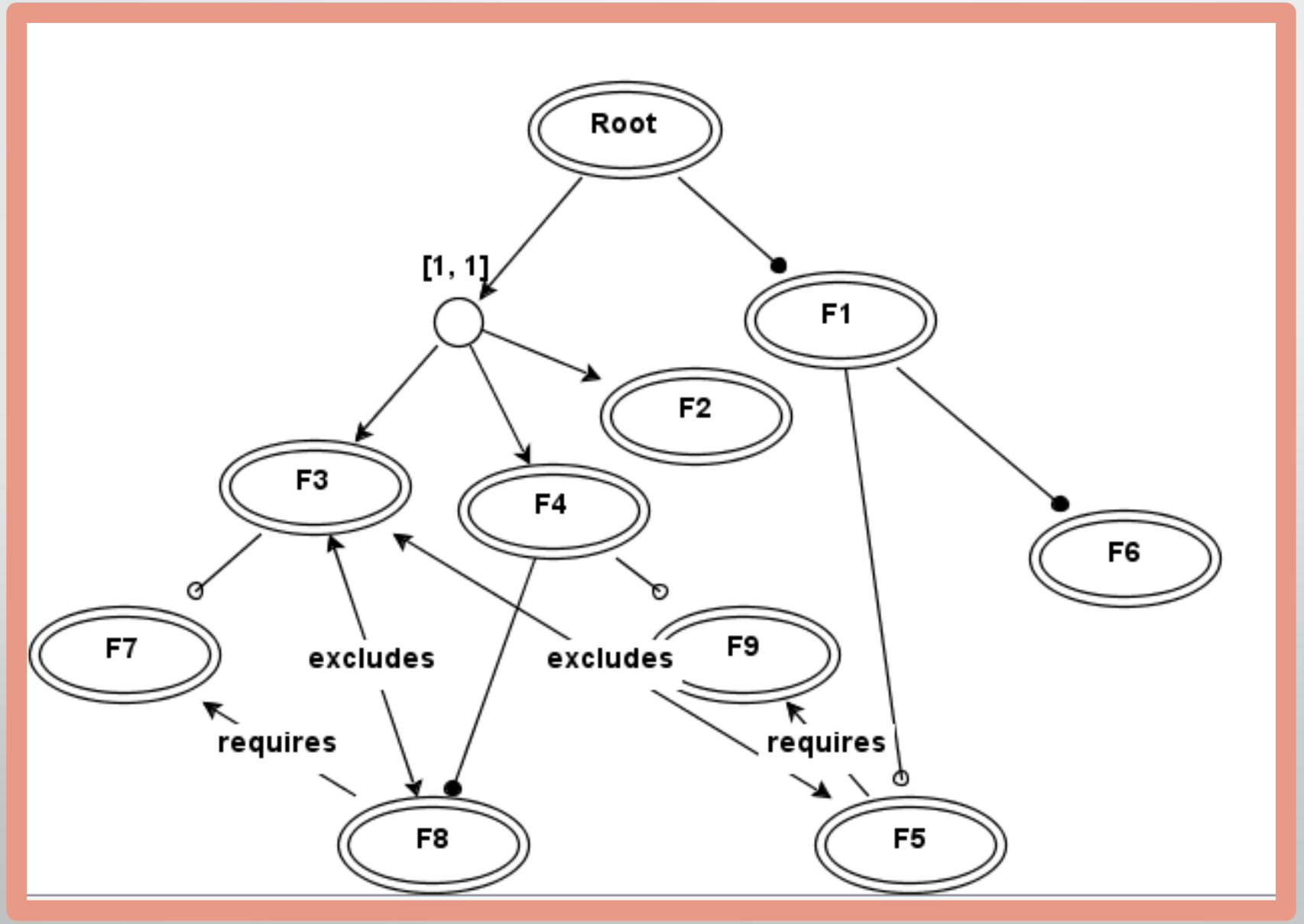
Solution: PLM represented with a high level constraint language

[MovementControlSystem, SpeedSensor, PositionSensor, PositionSensor1, PositionSensor2, PositionSensor3, PositionSensor4, Processor, Processor1, Processor2, InternalMemory1, InternalMemory2, Size] ^
[MovementControlSystem, SpeedSensor, PositionSensor, PositionSensor1, PositionSensor2, PositionSensor3, PositionSensor4, Processor, Processor1, Processor2, InternalMemory1, InternalMemory2] ∈ {0,1} ^
Size ∈ {0, 128,512,1024}
PositionSensor1 ⇒ PositionSensor ^
PositionSensor2 ⇒ PositionSensor ^
PositionSensor3 ⇒ PositionSensor ^
PositionSensor4 ⇒ PositionSensor ^
Processor1 ⇒ Processor ^
Processor2 ⇒ Processor ^
InternalMemory1 ⇒ InternalMemory ^
InternalMemory2 ⇒ InternalMemory ^

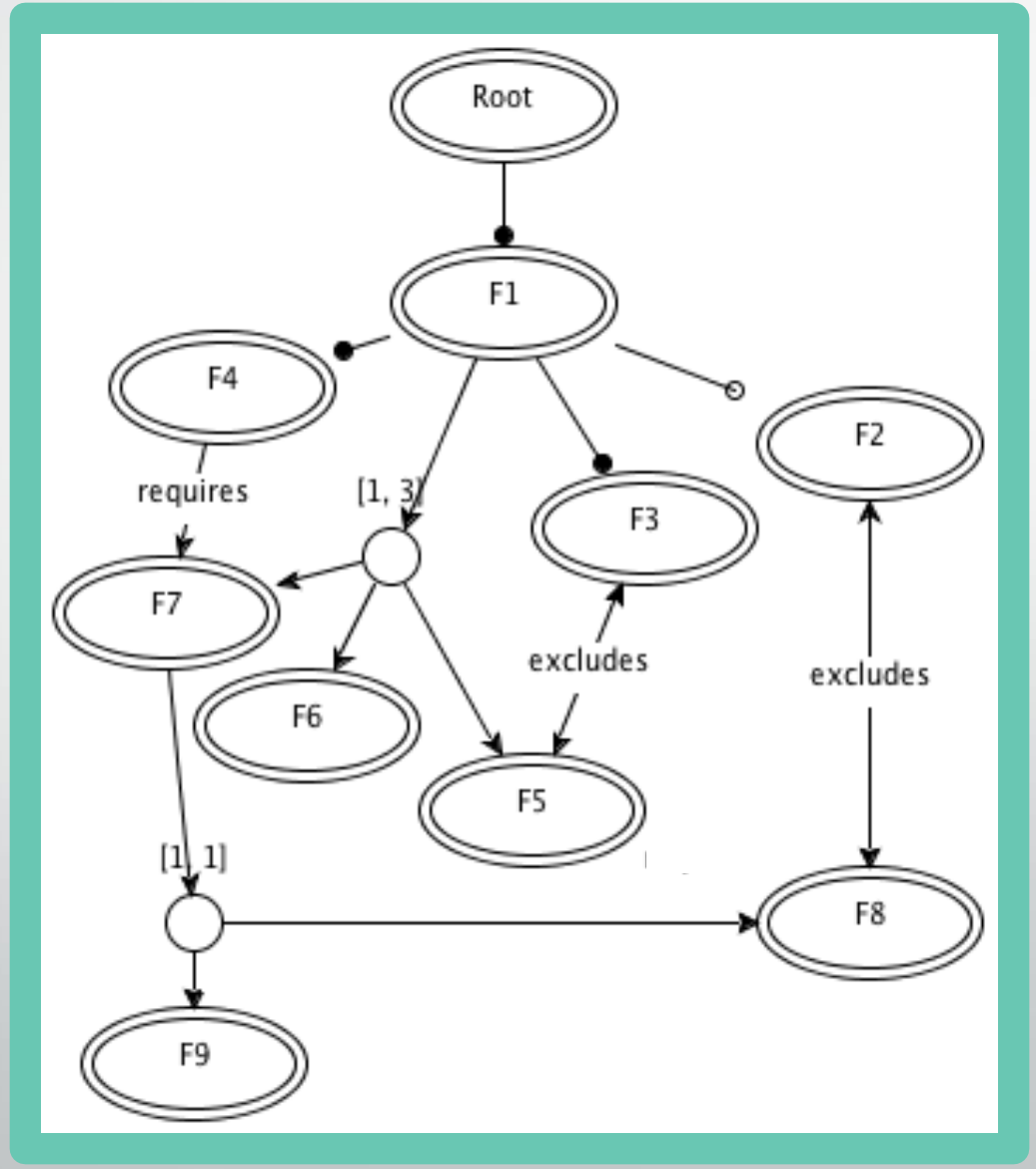


PositionSensor ⇒ (0 ≤ PositionSensor1 + PositionSensor2 + PositionSensor3+PositionSensor4 ≤ 4)^
Processor ⇒ (1 ≤ Processor1 + Processor2 ≤ 2) ^
InternalMemory ⇒ (1 ≤ InternalMemory1 + InternalMemory2 ≤2)
MovementControlSystem ⇔ SpeedSensor ^
MovementControlSystem ⇔ Processor ^
Processor ⇔ InternalMemory ^
(MovementControlSystem ⇒ PositionSensor ≥ 0) ^
(PositionSensor ⇒ MovementControlSystem) ^
InternalMemory ⇔ Size > 0 ^
SpeedSensor ⇒ PositionSensor

Exercise: Represent and execute some configurations of this model in GNU Prolog…How many products in the model?...



Exercise: Represent and execute some configurations of this model in GNU Prolog…How many products in the model?...



Verification,

at the domain engineering level, consists of finding defects in the product line model

Why verification of PLMs is important?

- The success of this development paradigm highly depends on the quality of the PLMs.
- Defects that are not identified or not corrected will
  - inevitably spread to the products created from the product line or
  - affect the evolution of the product line

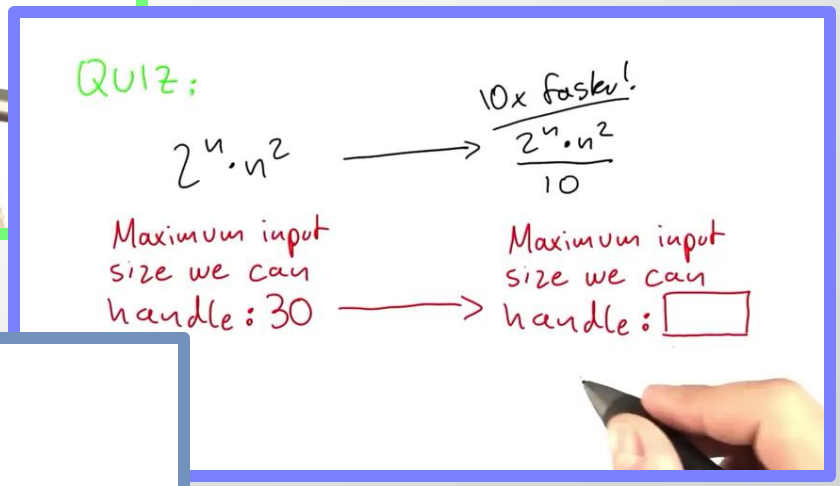
  which can drastically diminish the benefits of the product line strategy (Von der Maßen & Lichter 2004, Benavides 2007)

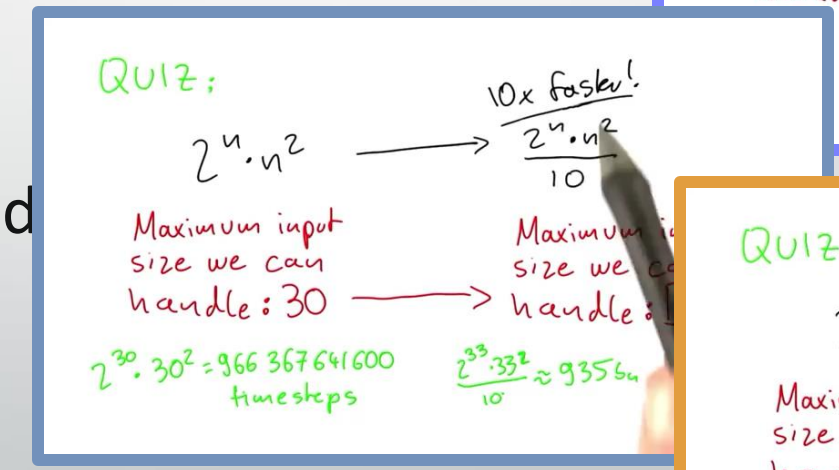
Why verification is difficult?

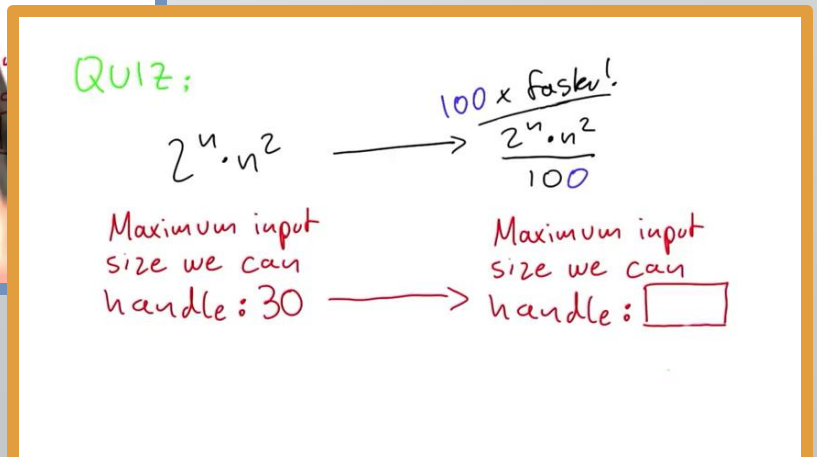
- Product line modeling is an error-prone activity
  (Von der Maßen *et al.* 2004, Batory *et al.* 2005)
- Product line verification is a NP-complete problem
  (Yan et al. 2009).
- A product line specification represents not one, but a collection of products that is defined implicitly and that may include contradictory requirements
  (Lauenroth *et al.* 2010).











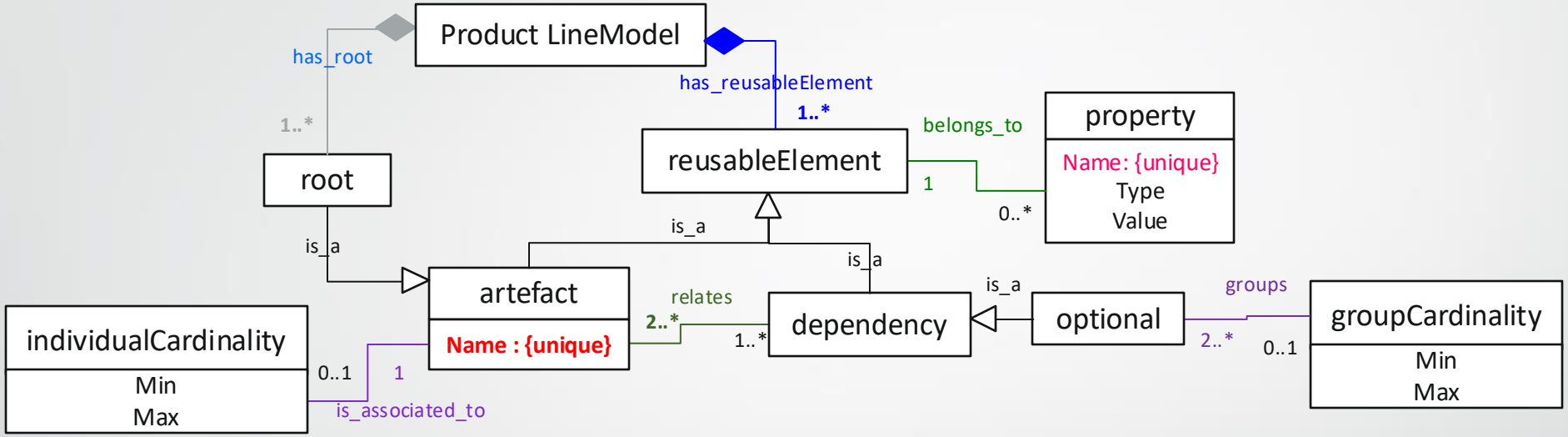
Generic verification of product line models



Generic and adaptable Conformance checking

Generic semantics-related verification

Dependencies among common product line model elements



Root uniqueness/multiplicity

Reusable elements uniqueness/multiplicity

Dependency completeness

Non-overlap of the artefact in a same model

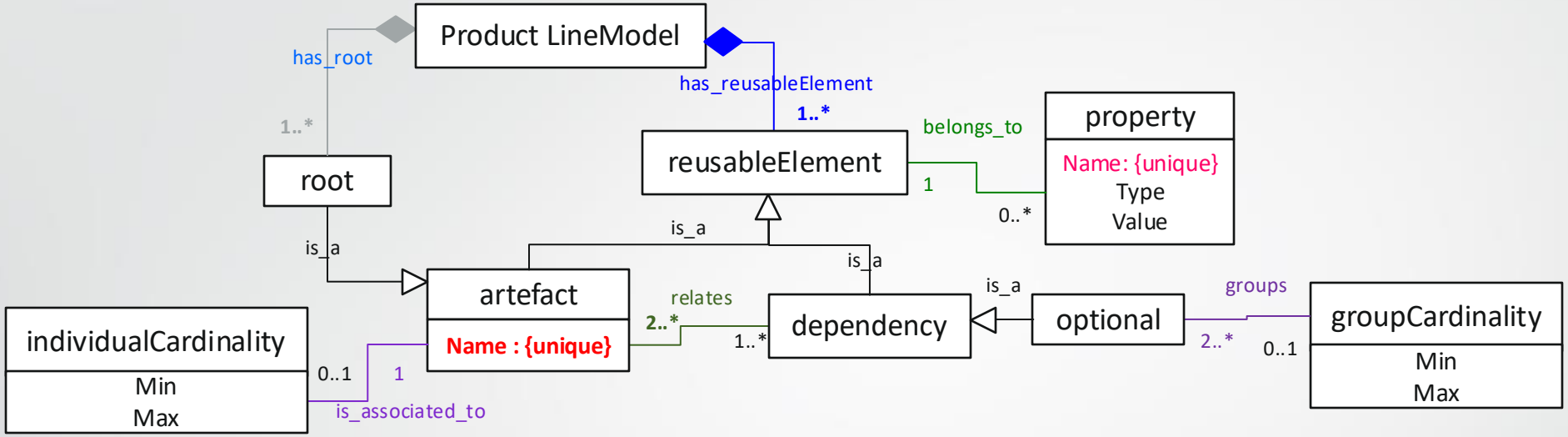
Properties belonging to a single reusable element

Association of individual cardinalities to artefacts

Non-overlap of the properties that belong to the same reusable element

Optional dependencies in group cardinality

Dependencies among common PLM elements



Predicates:

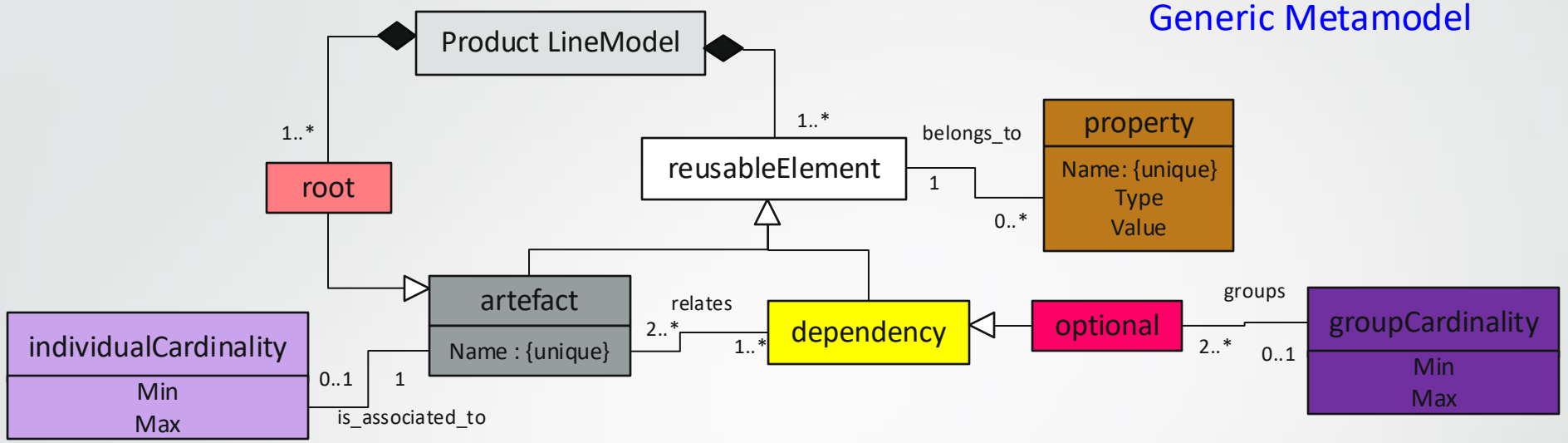
- *has_root*(M, R) returns *true* if the artefact R is a root of the product line model M , and *false* otherwise.
- *has_reusableElement*(M, R) returns *true* if there is a reusable element R in M, and *false* otherwise.
- *belongs_to*(R, P) reuturns *true* if P is a property of R, and *false* otherwise.
- *is_a*(A, B) returns *true* if B is a subclass of A, and *false* otherwise.
- *relates*(D, [A1, A2]) : evaluates if there a list of artefacts [A1, A2, …, Ai] that are related to a dependency D.
- *is_associated_to*(A, IC): evaluates if there is an individual cardinality IC associated to an artefact A.
- *groups*(GC, [O1, O2]) returs *true* if the optional dependencies O1 and O2 belong to the group cardinality GC.
- *isOptional*(M, A): returs true if the artefact A is modeled as optional in the model M, and false otherwise.
- *oneProduct*(M): returns true if a given PLM M allows the configuration of at least one product, and false otherwise.
- *oneProductWithConstraint*(M,C): returns true if M allows the configuration of at least one product that respects a constraint C.
- *find*(M, n): returns true if n different products can be derived from a PLM M, and false otherwise.

Functions:

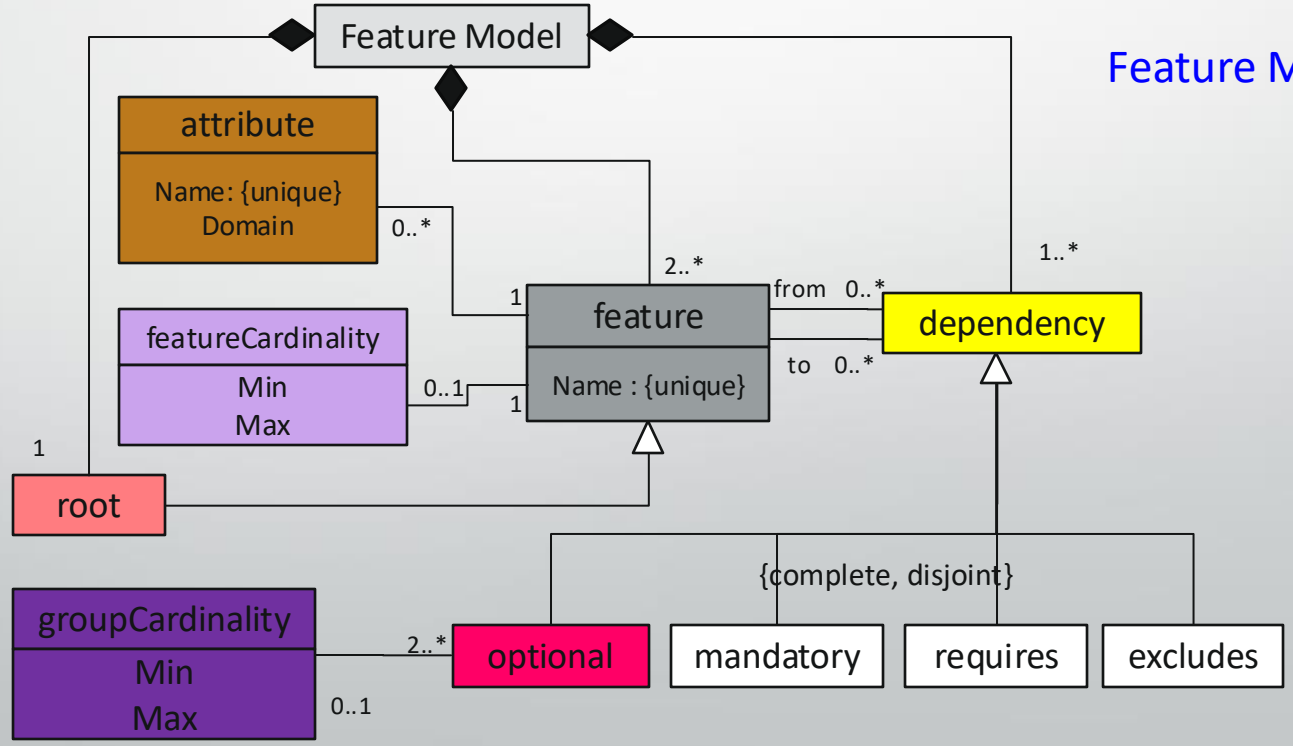
isPLMofOptional(O), semanticOf(M), eraseOneDependency(M, D),isPLMofDependency(D)

The case of Feature Models

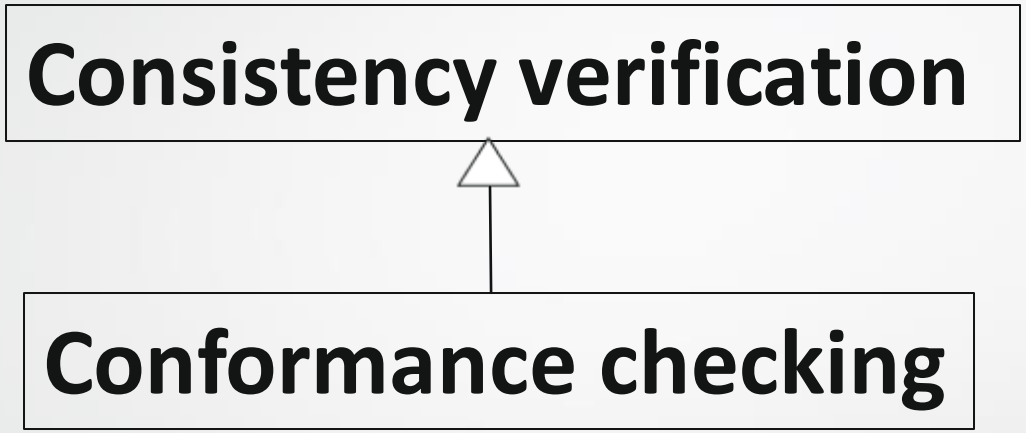
Generic Metamodel



Feature Model Metamodel

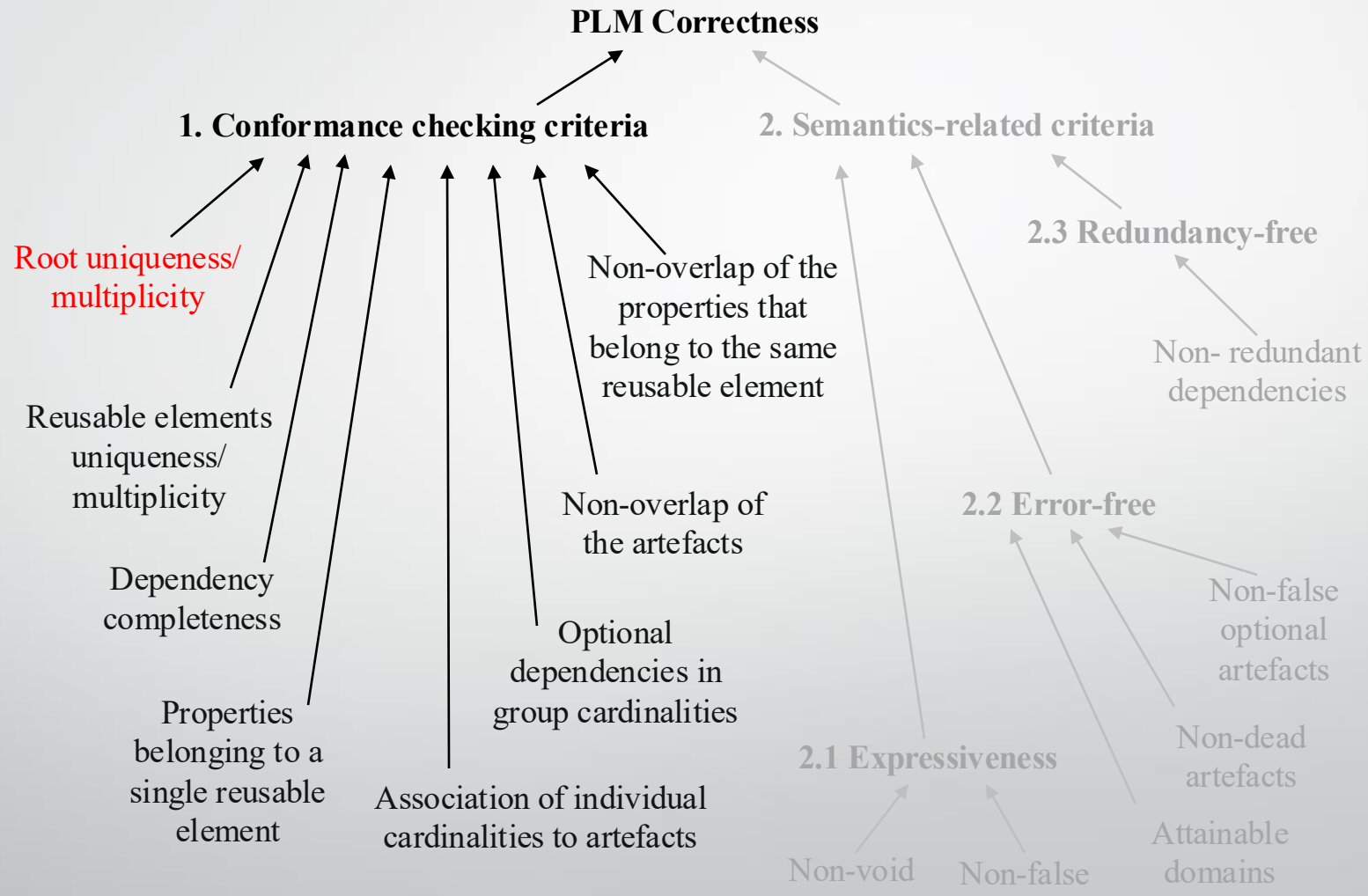


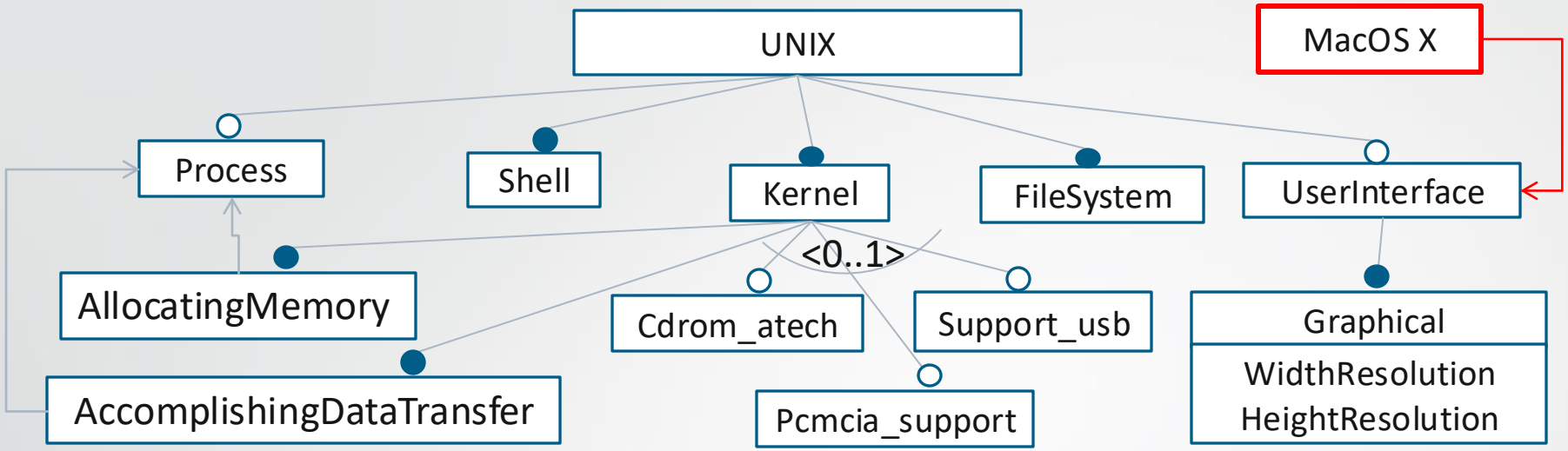
Conformance checking



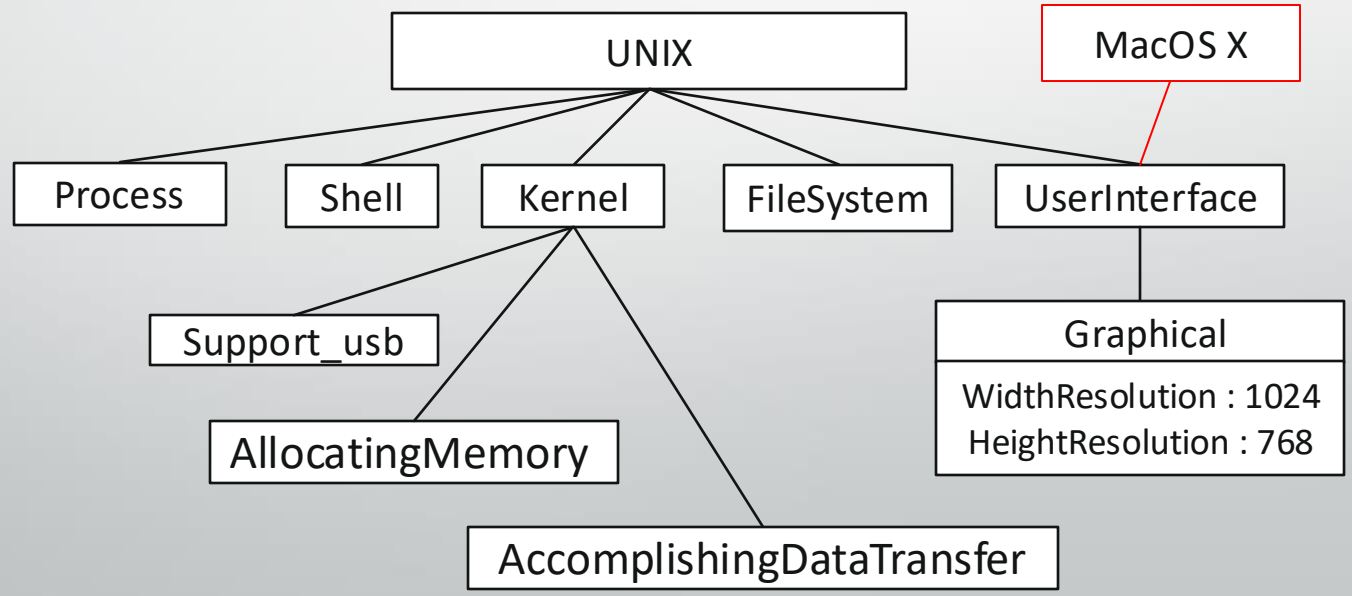
In conformance checking “it is checked that a model satisfies the constraints captured in the meta-model, i.e., that the model is indeed a valid instance of the meta-model” (Paige et al. 2007).

Typology of verification criteria

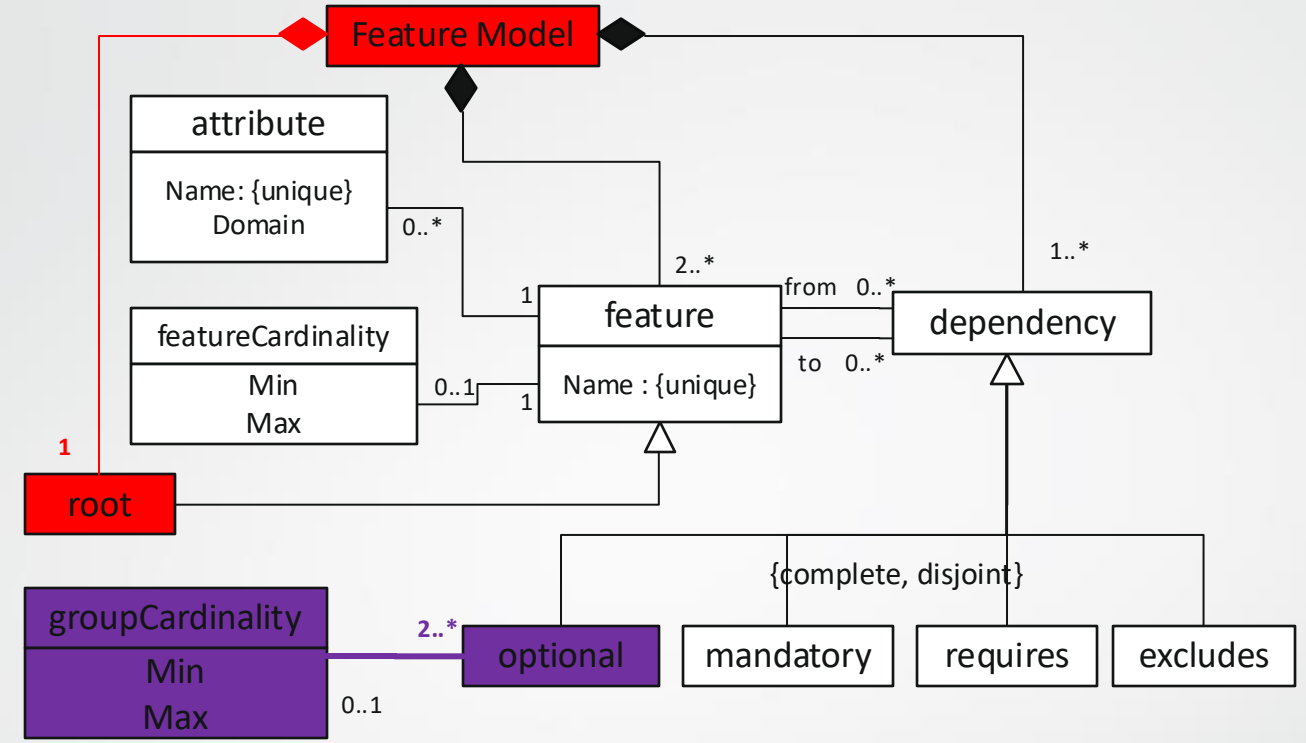




Graphical→ relation([WidthResolution, HeightResolution], {[800, 600], [1024,768], [1366,768]})

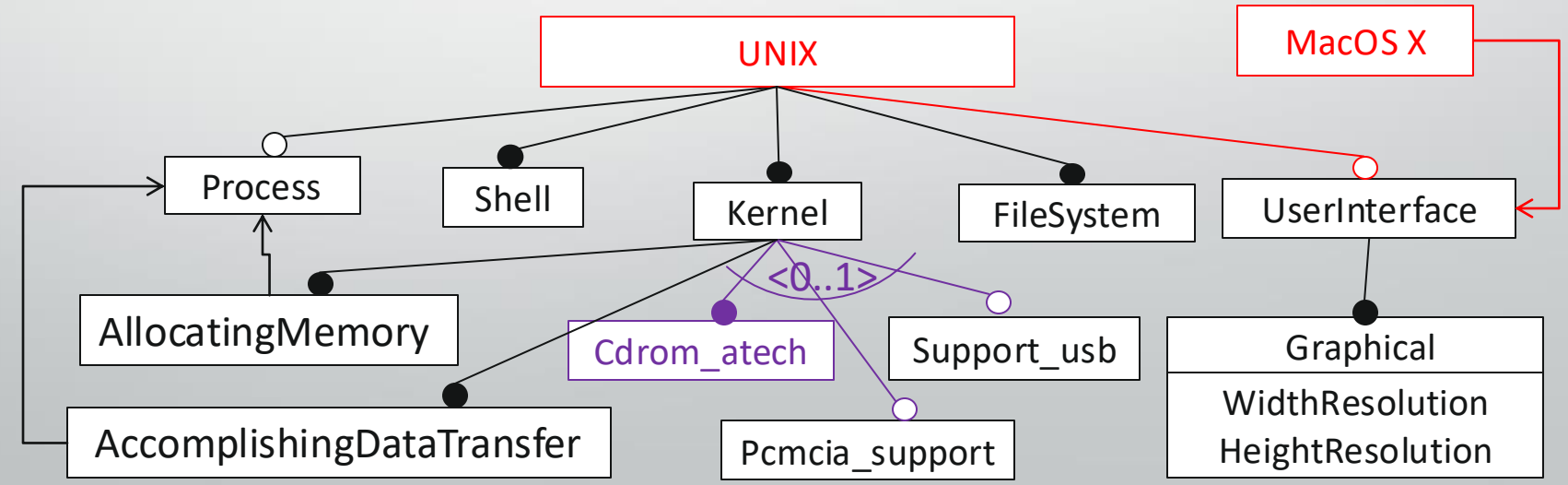


FM meta-model

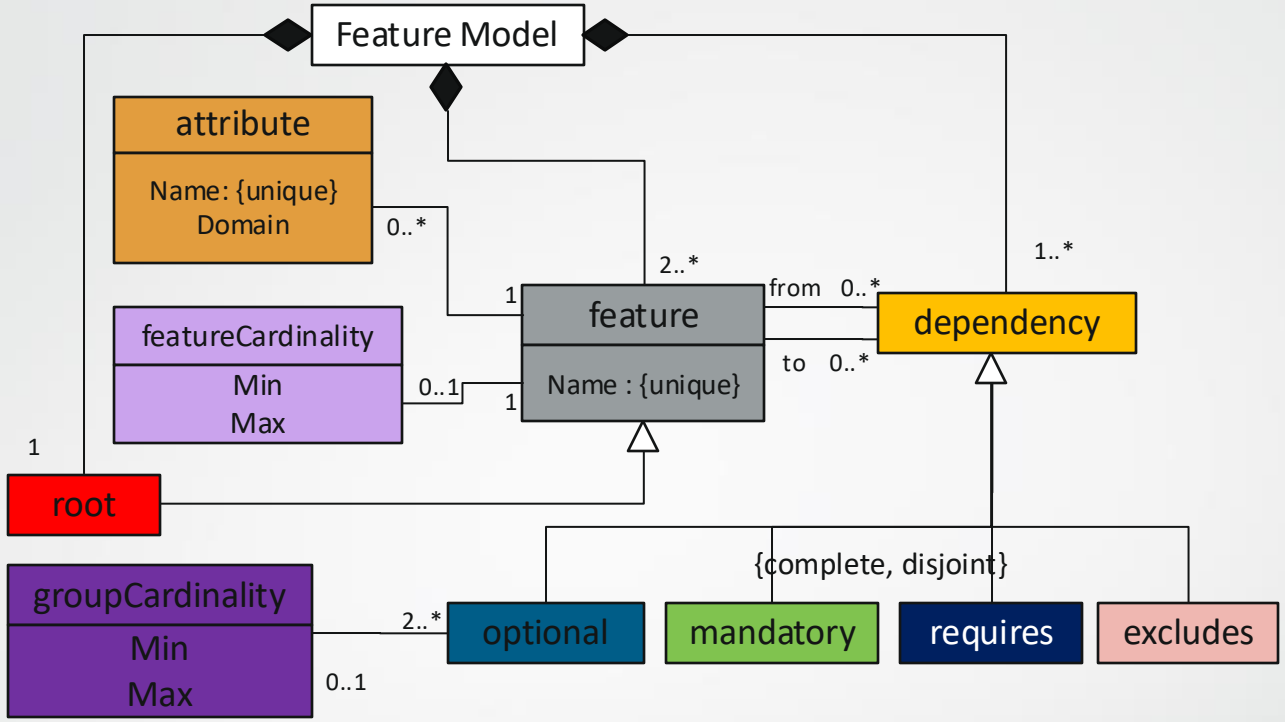


Conformance checking

FM



FM Meta-model



Represent it

CLP representation of the meta-model

feature(IdFeature, Name, LIdAttributes).

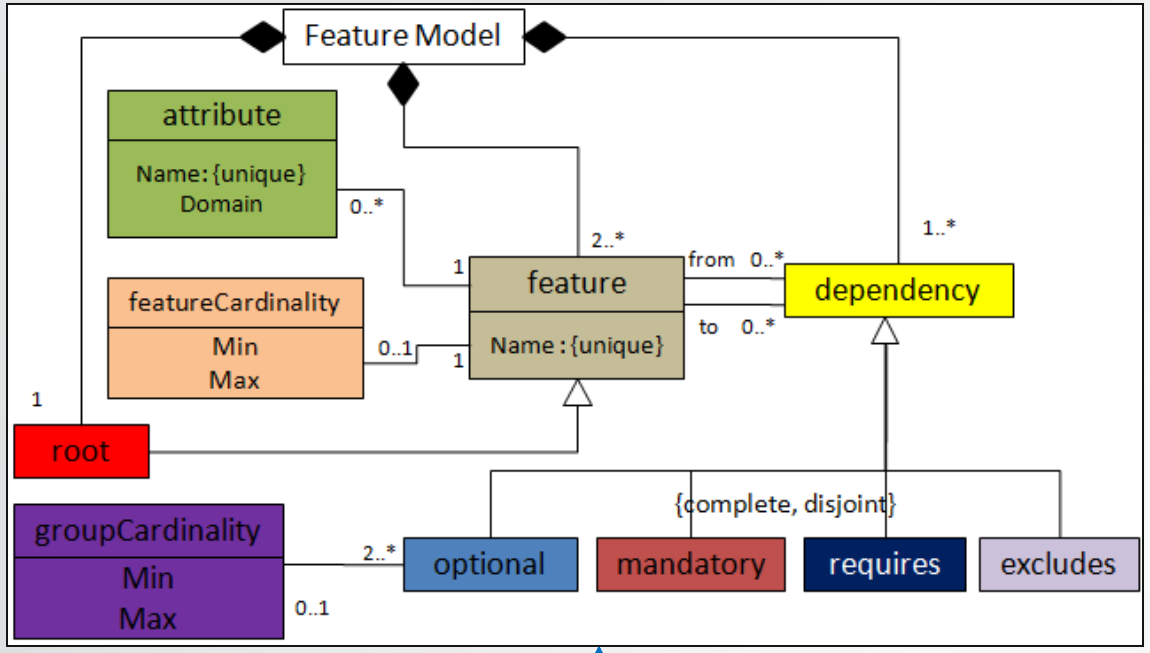
attribute(IdAttribute, Name, Domain).

root(IdFeature).

featureCardinality(IdFeature, Min, Max).

dependency(IdDependency, IdFeature1, IdFeature2, DependencyType).

groupCardinality(LIdDependencies, Min, Max).



Conforms to

feature(IdFeature, Name, LIdAttributes).

root(IdFeature).

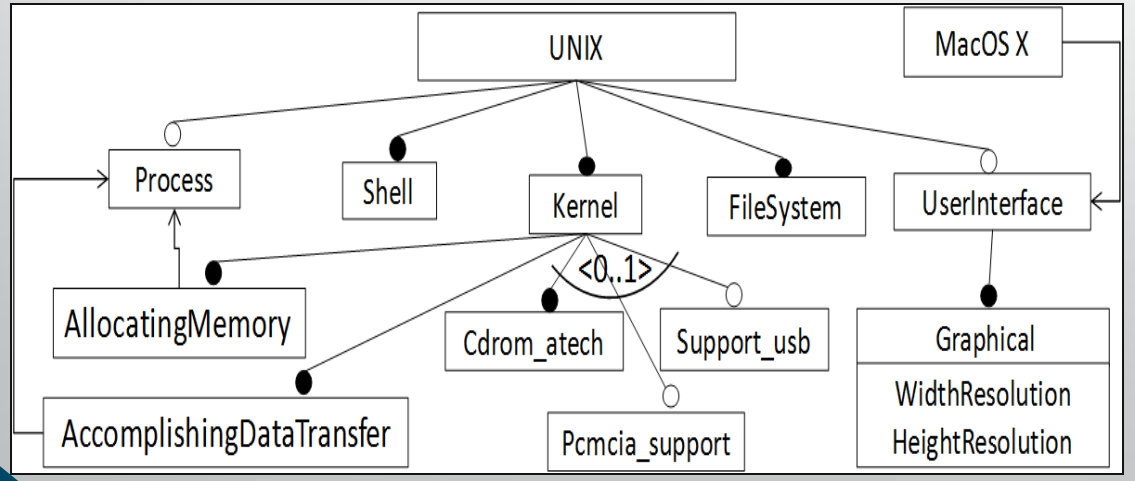
attribute(IdAttribute, Name, Domain).

dependency(IdDependency, IdFeature1, IdFeature2, DependencyType).

groupCardinality(LIdDependencies, Min, Max).

featureCardinality(IdFeature, Min, Max).

Conforms to



(1) root(fea1).
(2) feature(fea1, 'UNIX',[]).
(3) root(fea2).
(4) feature(fea2, 'MacOSX',[]).
(5) feature(fea3, 'UserInterface', []).
(6) feature(fea4, 'Graphical', [att1,att2]).
(7) attribute(att2, 'HeightResolution ', [600,768]).
(8) dependency(dep1, fea1, fea3, optional).
(9) dependency(dep2, fea2, fea3, requires).
(10) dependency…

Conformance rules – Example 1

Root uniqueness/multiplicity

Formalization

$$\forall M, \exists R.\ \text{producLineModel}(M) \wedge \text{root}(R) \wedge \text{has_root}(M,R)$$

adapted from

In our feature-based formalism, product line models and product models should have one and only one root.

Implementation

```
(1) conformance_1(LRootId) :-
(2)    findall(FeatureId, root(FeatureId), LRootId),
(3)    length(LRootId, N),
(4)    N \== 1.
```

Conformance rules – Example 2

Optional dependencies in group cardinalities

Formalization

$$\forall G, \exists X1, \exists X2.\ groupCardinality(G) \wedge optional(X1) \wedge optional(X2) \wedge X1 \neq X2 \wedge isPLMofOptional(X1) = isPLMofOptional(X2) \wedge groups(G, [X1, X2])$$

adapted from

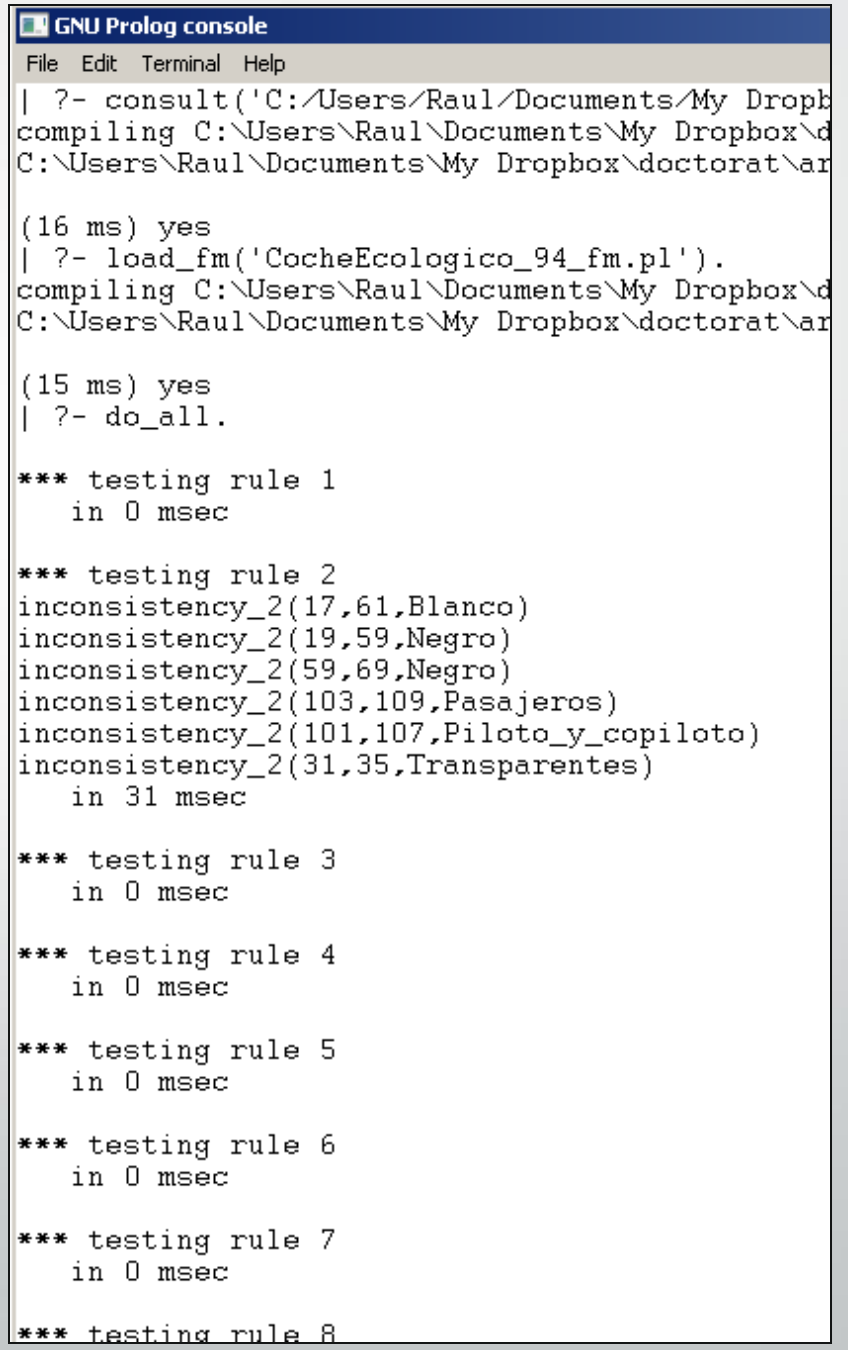
Features intervening in a group cardinality dependency should not be mandatory features

Implementation

```
(1) conformance_2(DepId, FeatureId) :-
(2) groupCardinality(LDepId, _, _),
(3) member(DepId, LDepId),
(4) dependency(DepId, _, FeatureId, mandatory).
```

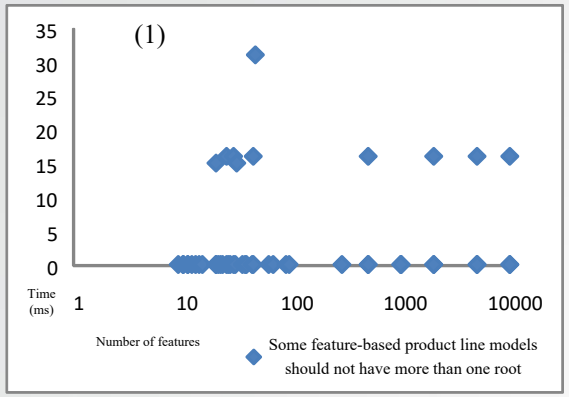
Conformance Checker of Feature Models

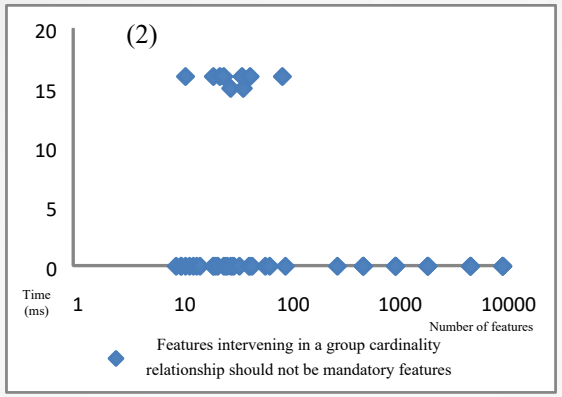
Built in GNU Prolog 1.3

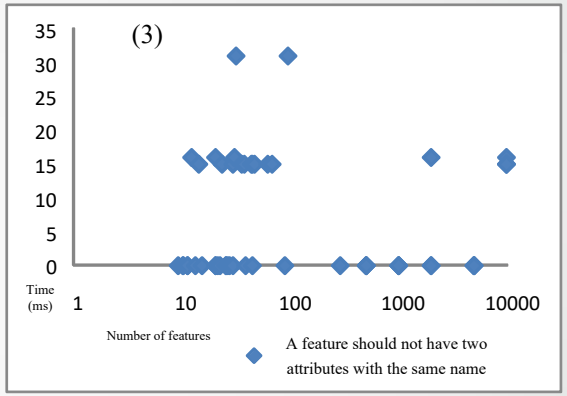


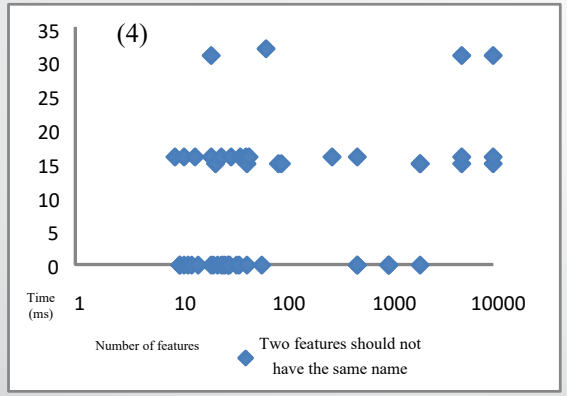
Conformance checking.

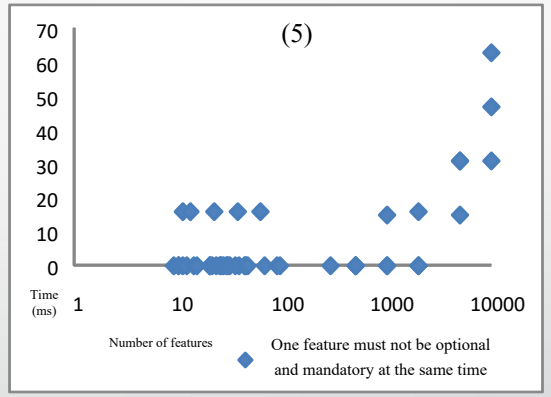
Validation of our approach with 49 FMs up to 10000 features

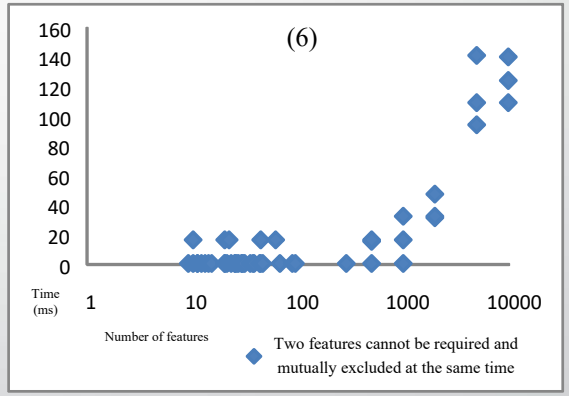


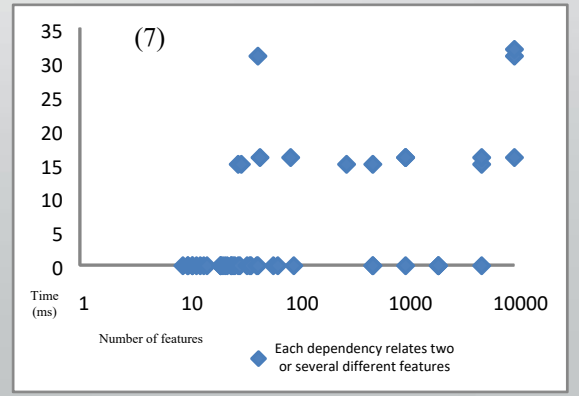












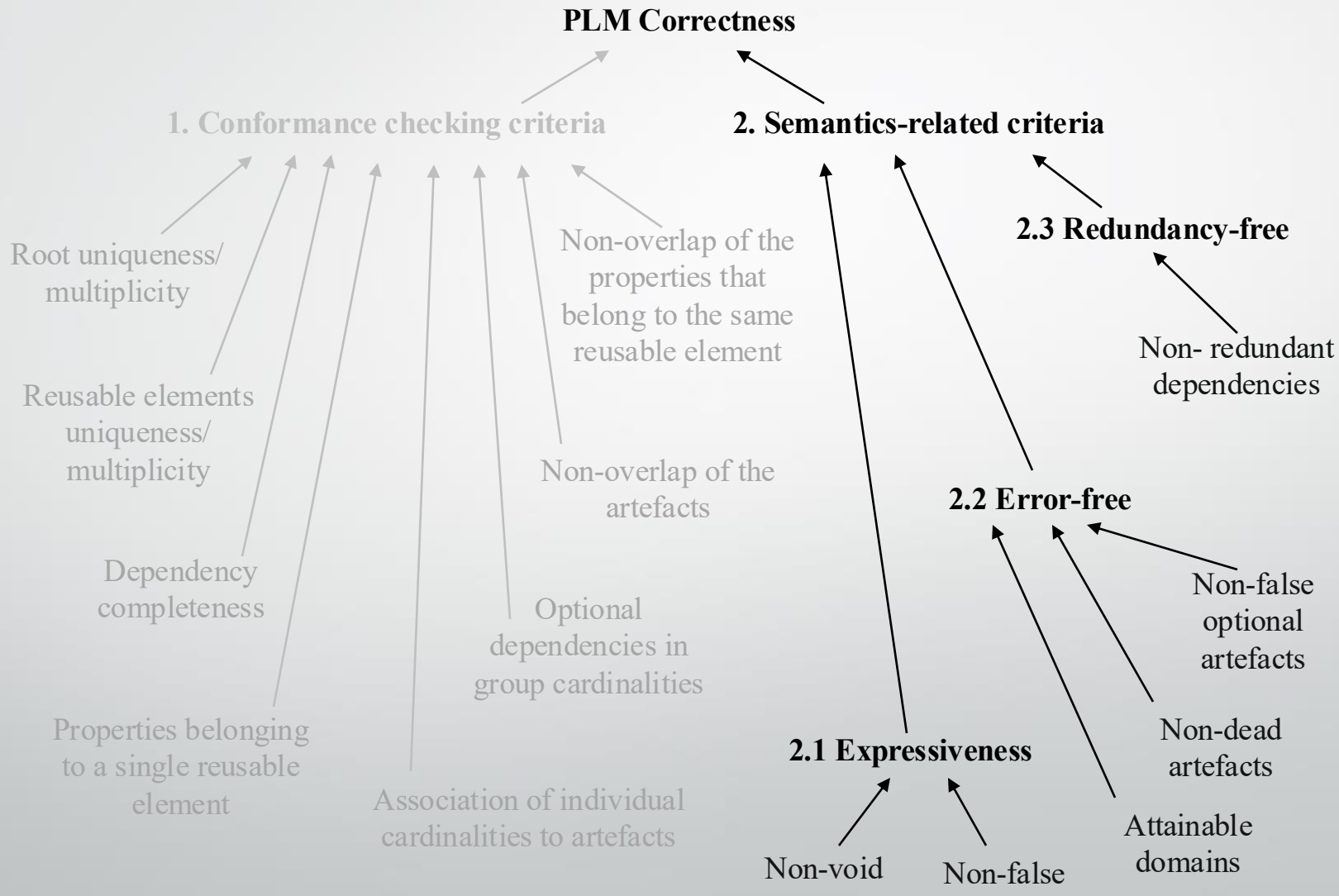
Execution time (ms)

Log 10 (Number of features)

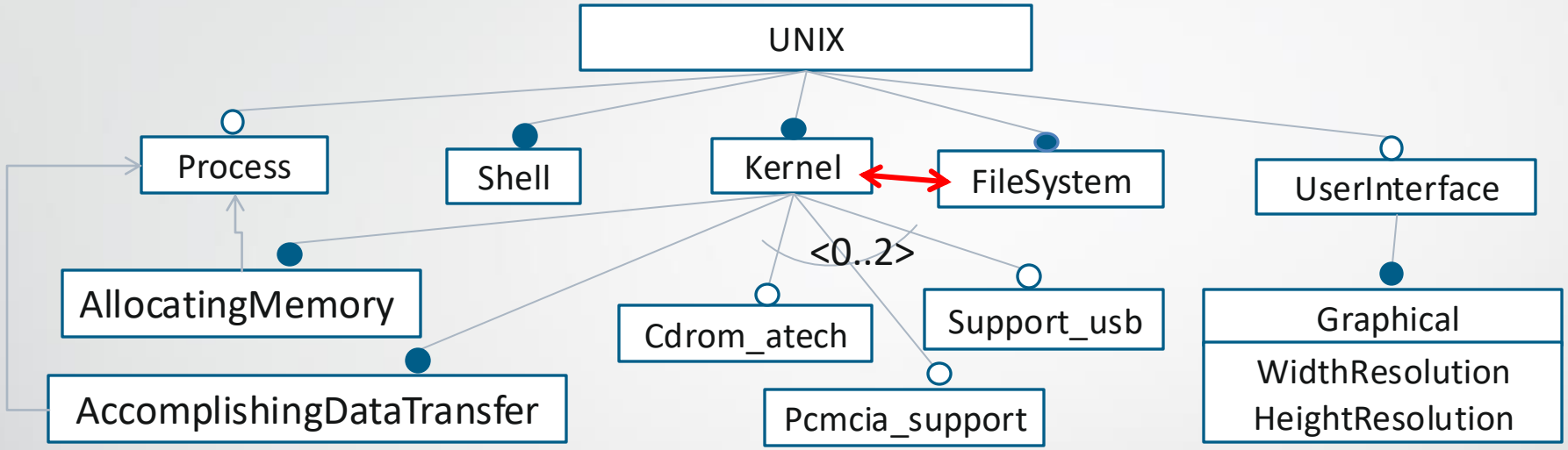
Semantics-related verification

Semantics-related verification is about the identification of non-structural defects on product line models.

Semantics-related verification



Semantics-related verification
Example: Non-void product line models



Graphical→ relation([WidthResolution, HeightResolution], {[800, 600], [1024,768], [1366,768]})

getOneSolution(M) → "false" → Void model

Semantics-related verification

Example: Non-void product line models

A void PLM is defined as a model that does not permit the configuration of any product.

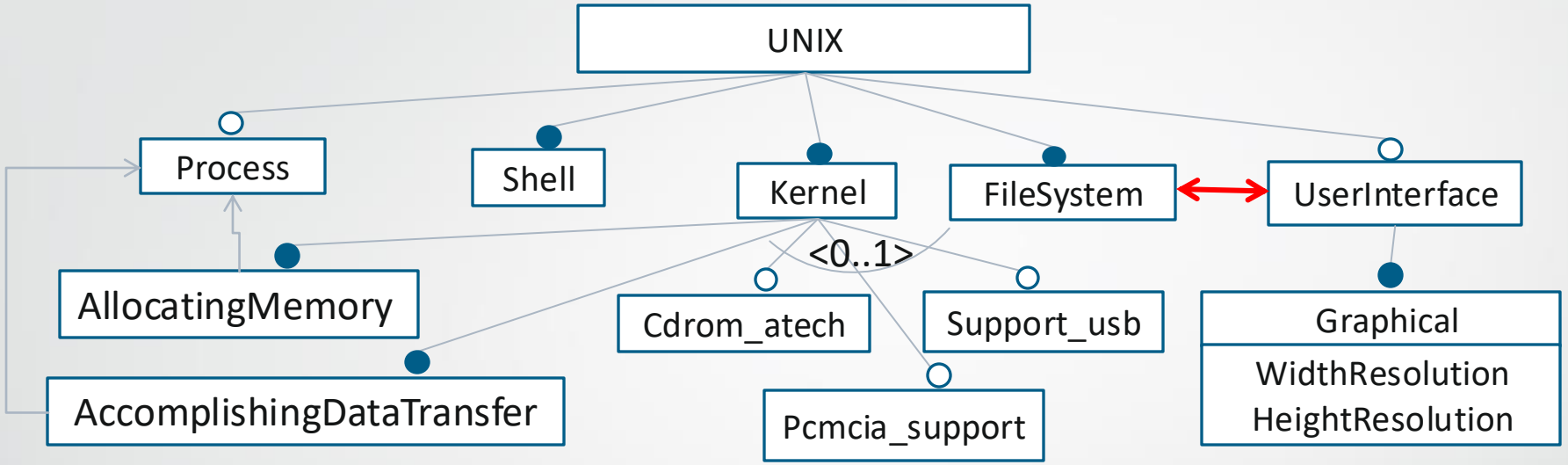
Formalization by the following Boolean function:

$$nonVoidPLM(M) \triangleq producLineModel(M) \wedge find(M,1)$$

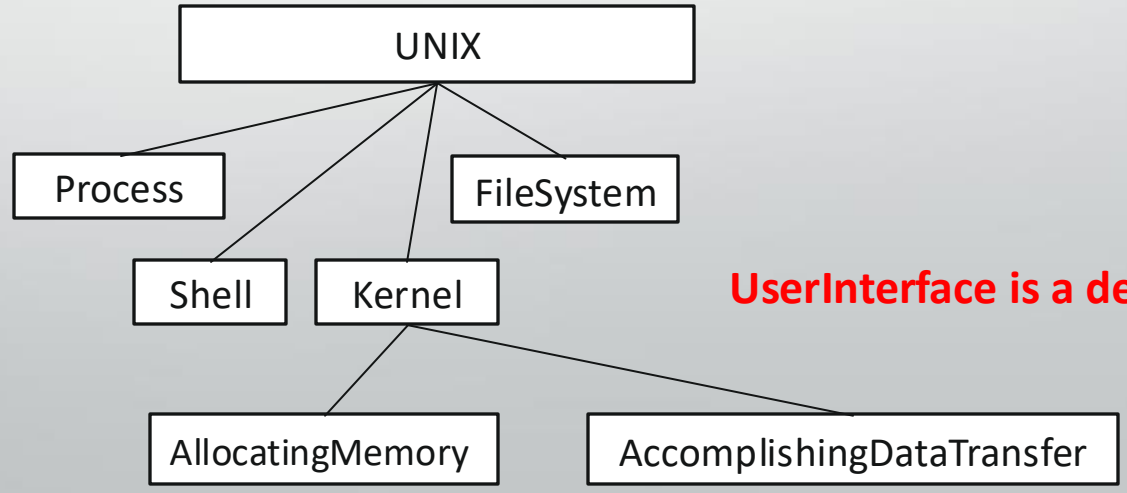
Algorithm:

```
nonVoidPLM(PLM M) {
        Answer = getOneSolution(M);
        If (Answer ≠ “false”){
                Write (Answer);
        }
        Else{
                Write (“Void PLM”);
        }
}
```

Semantics-related verification
Example: Non-dead features



Graphical→ relation([WidthResolution, HeightResolution], {[800, 600], [1024,768], [1366,768]})



UserInterface is a dead feature

Semantics-related verification
Example: Non-dead features

An artefact is dead if it cannot appear in any product of the product line

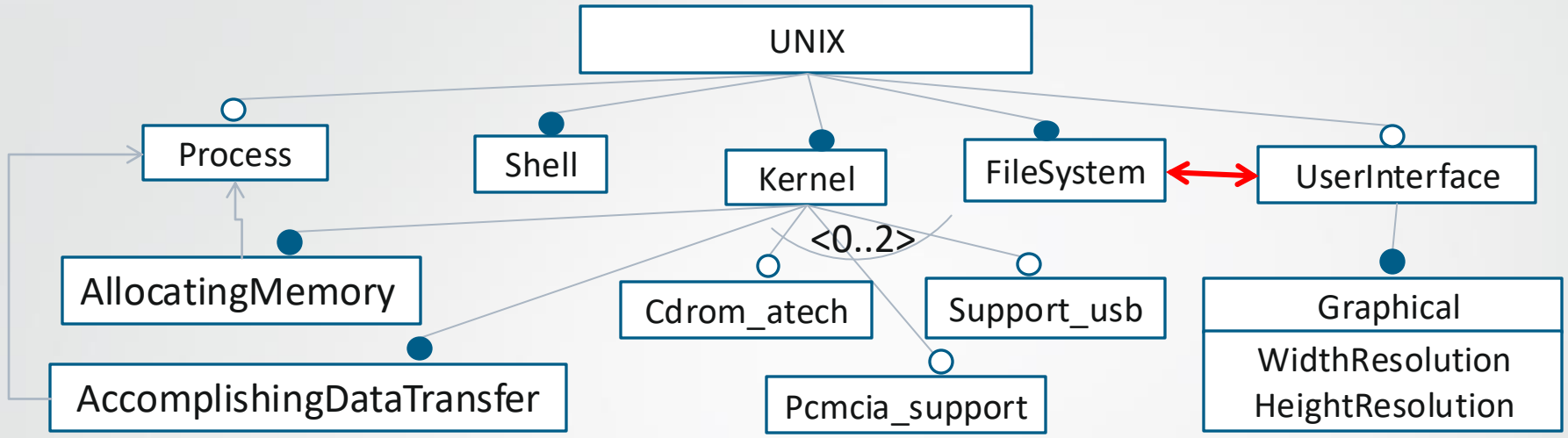
Formalization:

$$\forall M, \forall R, \forall A \;.\; \text{producLineModel}(M) \wedge \text{reusableElement}(R) \wedge \text{artefact}(A) \wedge \text{has_reusableElement}(M, R) \wedge \text{is_a}(R, A) \wedge \text{oneProductWithConstraint}(M, \text{'A>0'})$$

Algorithm:

```
DeadArtefacts(PLM M) {
        DeadArtefactsList = all artefacts of M;
        For (each variable V de DeadArtefactsList) {
        Product = oneProductWithConstraintn(M, "V > 0");
                If (Product = "false") {
                        Write ("The artefact " + V + " is  dead");
                }
                Else {
        Erase V and all the other non-zero variables in Product, from DeadArtefactsList;
                }
        }
}
```

Semantics-related verification. Example: Non-dead features



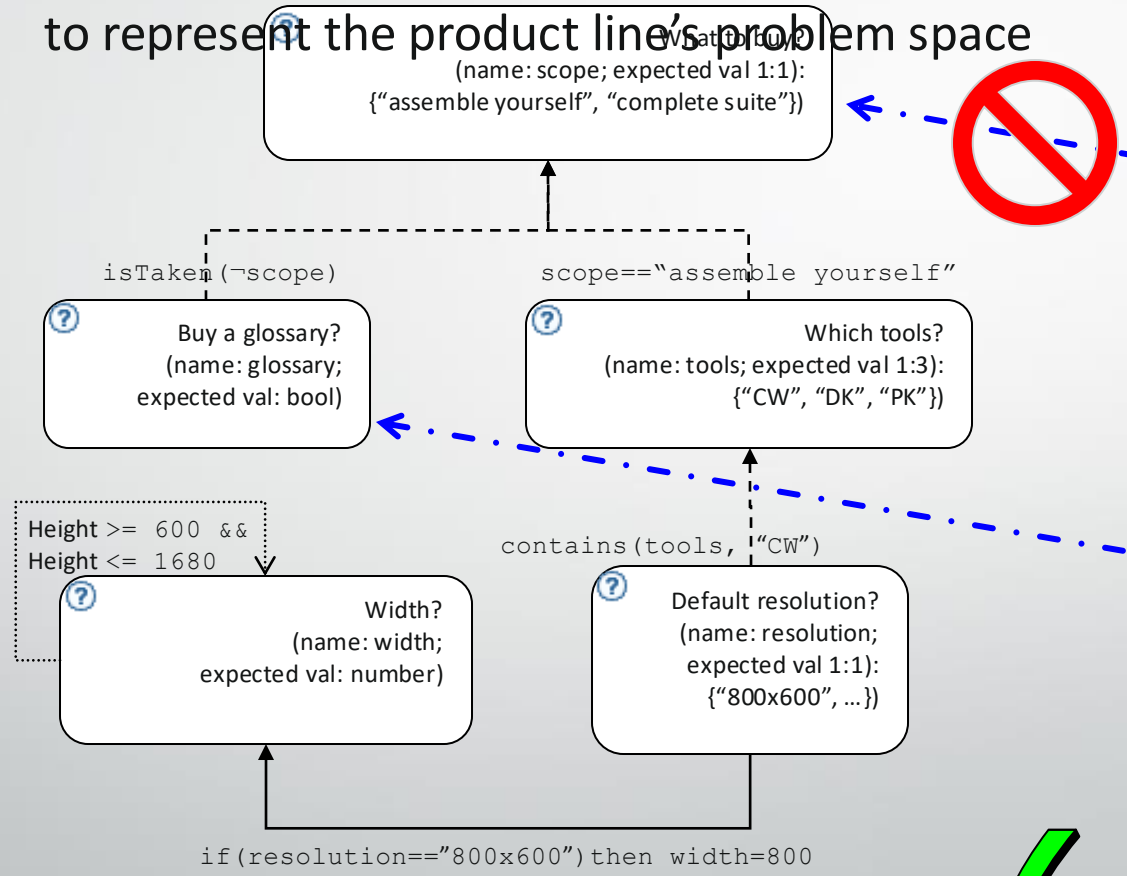
Graphical→ relation([WidthResolution, HeightResolution], {[800, 600], [1024,768], [1366,768]})

0. If the model is not void:
1. **deadArtefactsList** = [UNIX, Process, Shell, Kernel, AllocatingMemory, AccomplishingDataTransfer, FileSystem, UserInterface, Graphical, Cdrom_atech, Pcmacia_support, Support_usb].
2. oneProductWithConstraint(M, 'Kernel > 0')
3. P1 = [UNIX=1, Process=1, Shell=1, Kernel=1, AllocatingMemory=1, AccomplishingDataTransfer=1, FileSystem=1, UserInterface=0, Graphical=0, Cdrom_atech=1, Pcmacia_support=1, Support_usb=0]
4. **deadArtefactsList** = [UserInterface, Graphical, Support_usb].
5. oneProductWithConstraint(M, 'UserInterface > 0') → "false" → UserInterface is a dead feature

Multi-model verification
The case of Dopler variability models

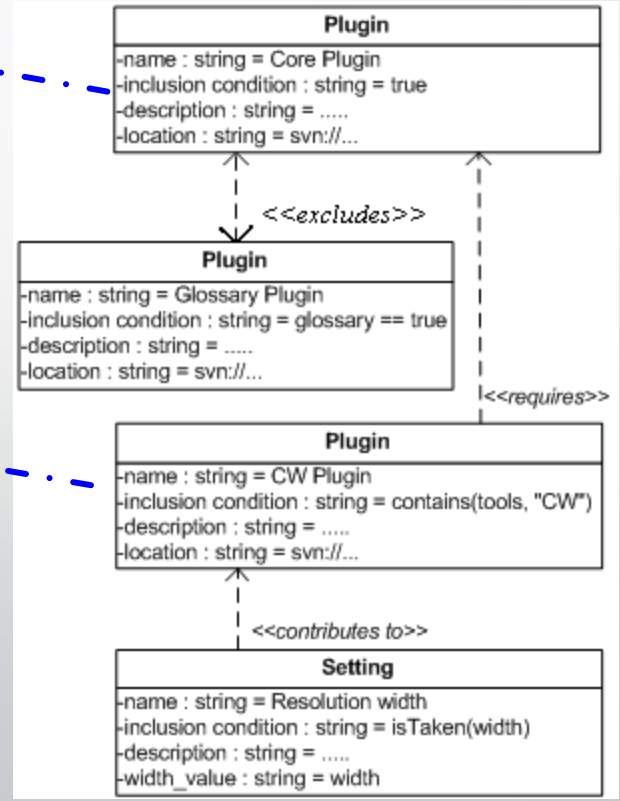
Decision model

to represent the product line's problem space



Asset model

to represent the solution space



Domain-specific verification
Example: Non-void (**DOPLER**) product line models

A void PLM is defined as a model that does not permit the configuration of any product.

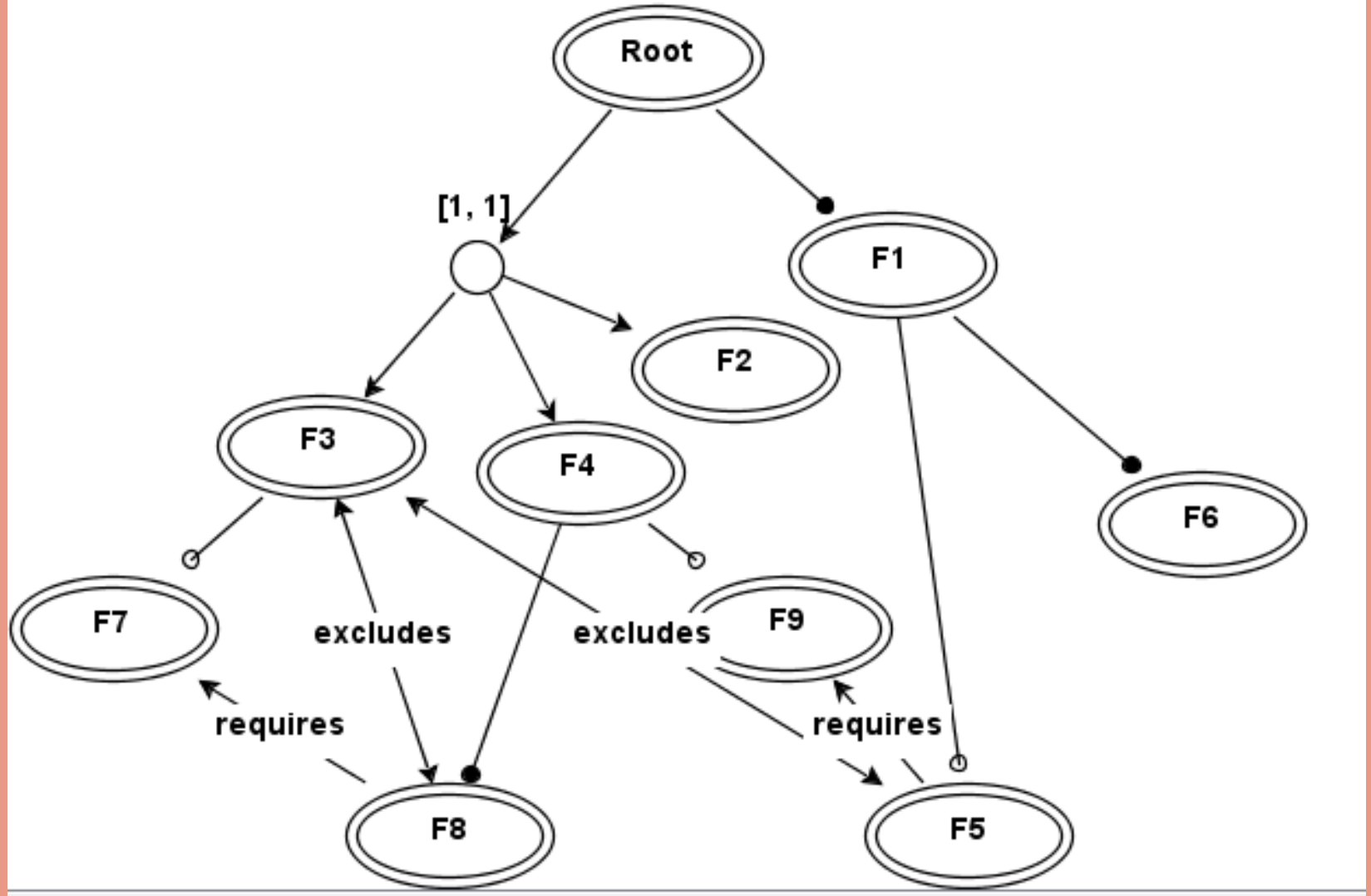
Formalization by the following Boolean function:

$$nonVoidPLM(M) \triangleq producLineModel(M) \wedge find(M,1)$$

Algorithm:

```
nonVoidPLM(PLM M) {
        Answer = getOneSolution(M);
        If (Answer ≠ “false”){
                Write (Answer);
        }
        Else{
                Write (“Void PLM”);
        }
}
```

Exercise: How to proceed to know if the models is void or not, and how to find the dead feature(s) if any?

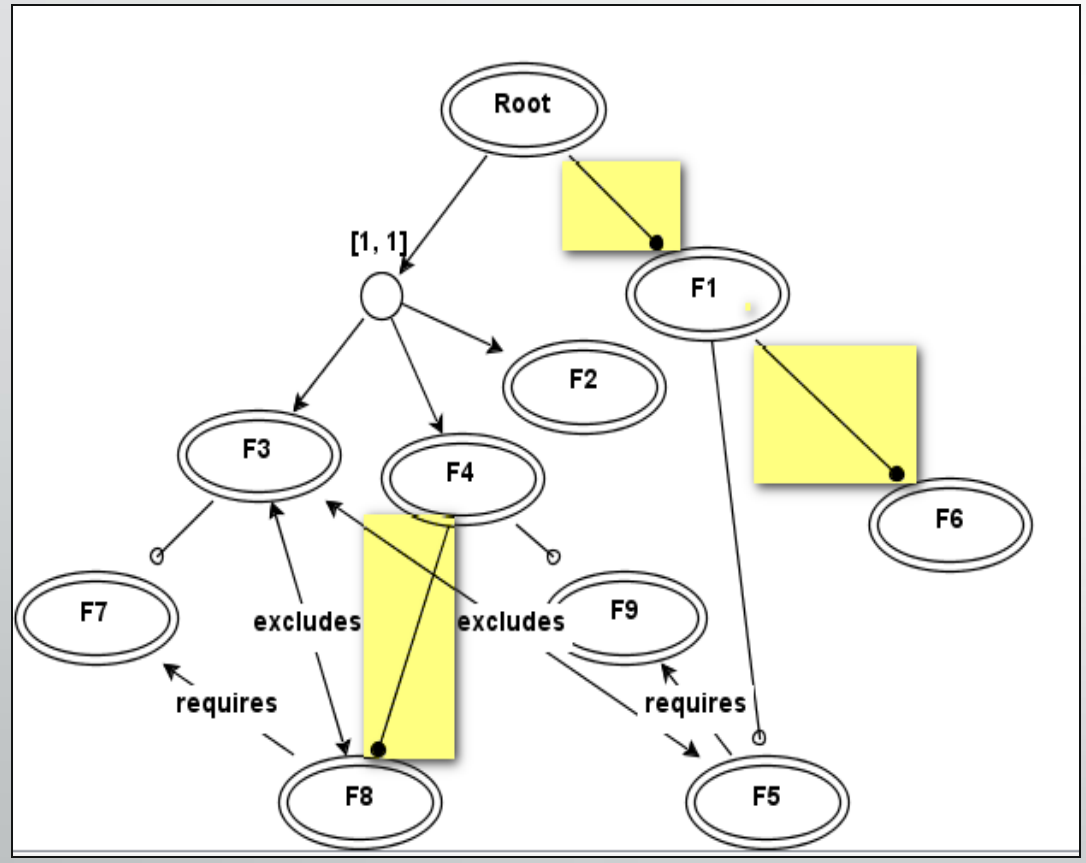


The first step to verify PLMs with a solver is to transform them into the language of the solver

Concept	CP representation
Root	Root = 1
Mandatory	Father < = > Child
Optional	Father >= Child
Requires	Requiring ==> Required Alternatively (1 – Requiring) + Required > 0
Exclusion	Excluding * Excluded = 0 Alternatively (1 – Excluding) v (1 – Excluded) > 0
Group cardinality	M*Father ≤ Child1+…+ChildN ≤ N*Father

(Mazo 2011)

The first step to verify PLMs with GNU Prolog is to transform them into the GNU Prolog language



Create a fact that represents the PLM

pl(L):-

Create the list of variables of the PLM

L=[Root, F1, F2, F3, F4, F5, F6, F7, F8, F9],

Give a domain to the variables

fd_domain (L, 0, 1),

Root

Root #= 1,

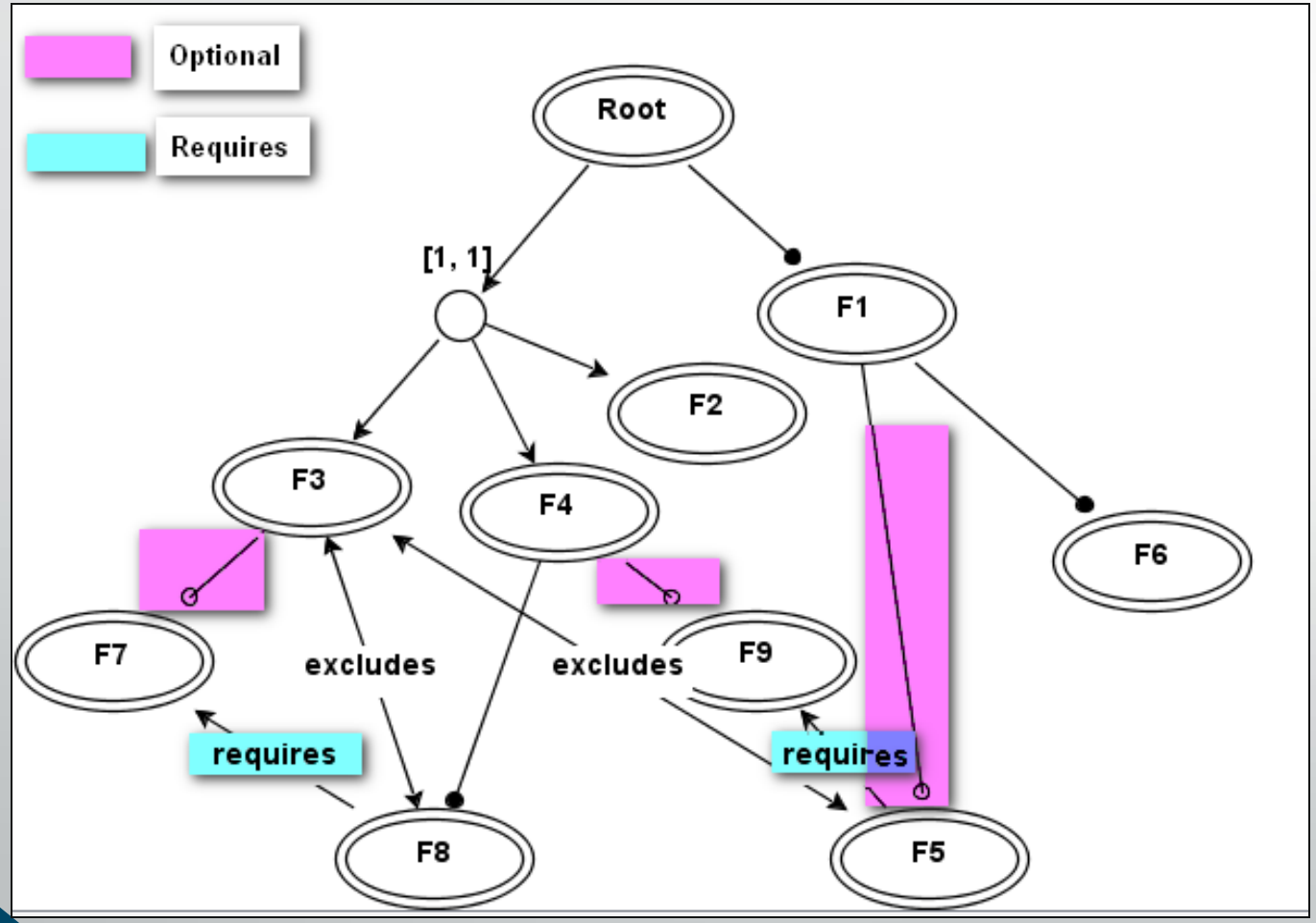
Mandatory

Root #= F1,

F4 #= F8,

F1 #= F6,

The first step to verify PLMs with GNU Prolog is to transform them into the GNU Prolog language



Optional

F1 #>= F5,
F4 #>= F9,
F3 #>= F7,

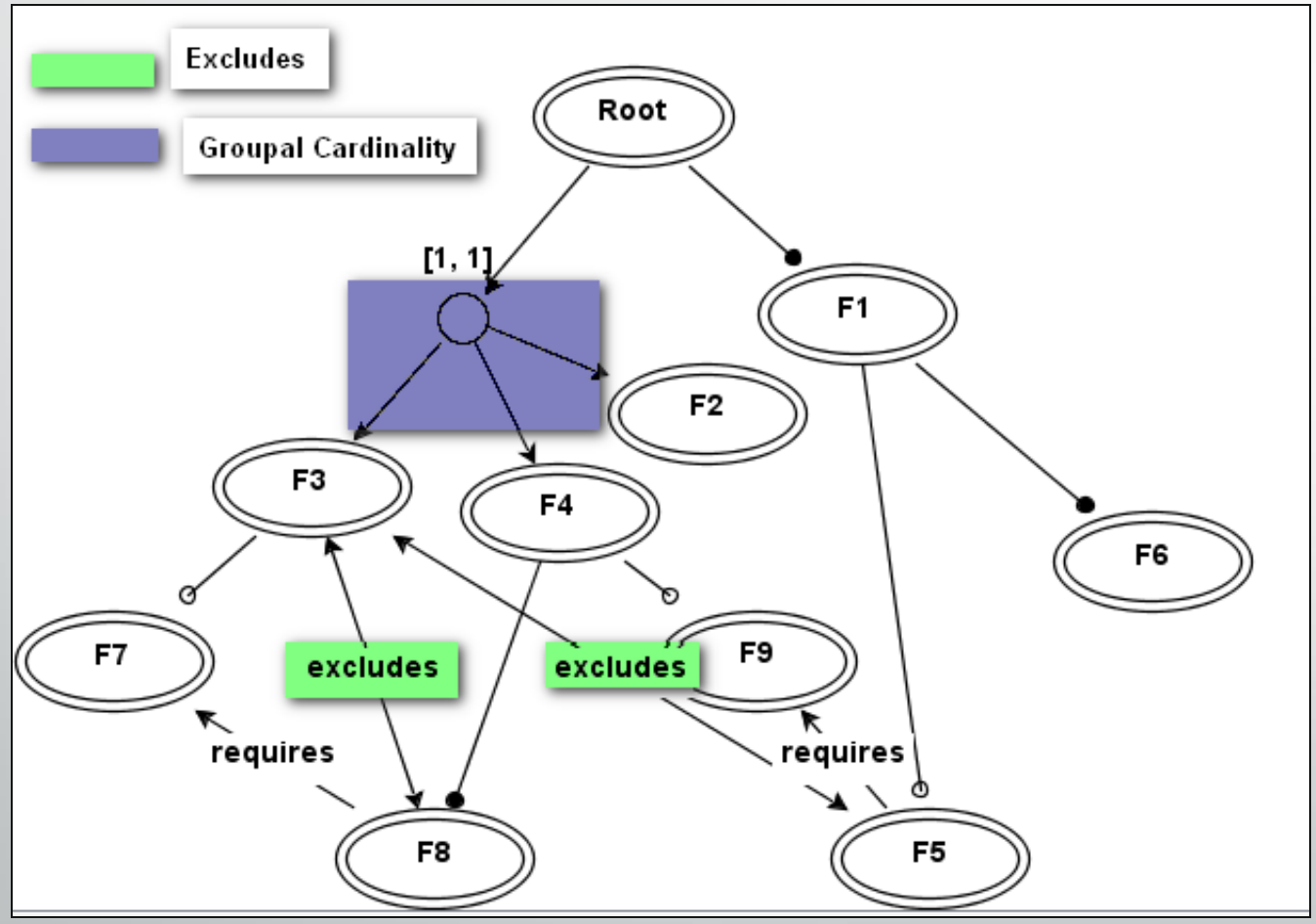
Requires

(1 - F8) + F7 #> 0,
(1 - F5) + F9 #> 0,

Another way to represent the requires

F8 #==> F7,
F5 #==> F9,

The first step to verify PLMs with GNU Prolog is to transform them into the GNU Prolog language



Excludes

(1 - F3) + (1 - F8) > 0,

(1 - F5) + (1 - F3) > 0,

Another way to represent the exclusions

F3 * F8 #= 0,

F3 * F5 #= 0,

Groupal

Root #= F2 + F3 + F4,

To verify a PLM with the help of a solver we need it as a Constraint program and some times we also need a "**verification constraint**"

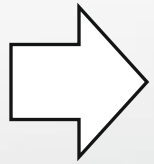
F5 =1,

This is the "verification constraint"

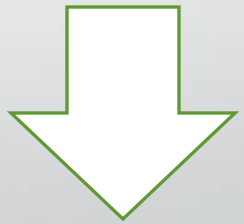
PLM as a Constraint Program

Root=1,
F4 <=> F8,
Root <=> F1,
F1 <=> F6
F1 >= F5,
F4 >= F9,
F3 >= F7,
(1 - F8) + F7 > 0,
(1 - F5) + F9 > 0,
(1 - F3) + (1 - F8) > 0,
(1 - F5) + (1 - F3) > 0,
Root = F2 + F3 + F4,

This part is satisfiable



SOLVER



There is no solution

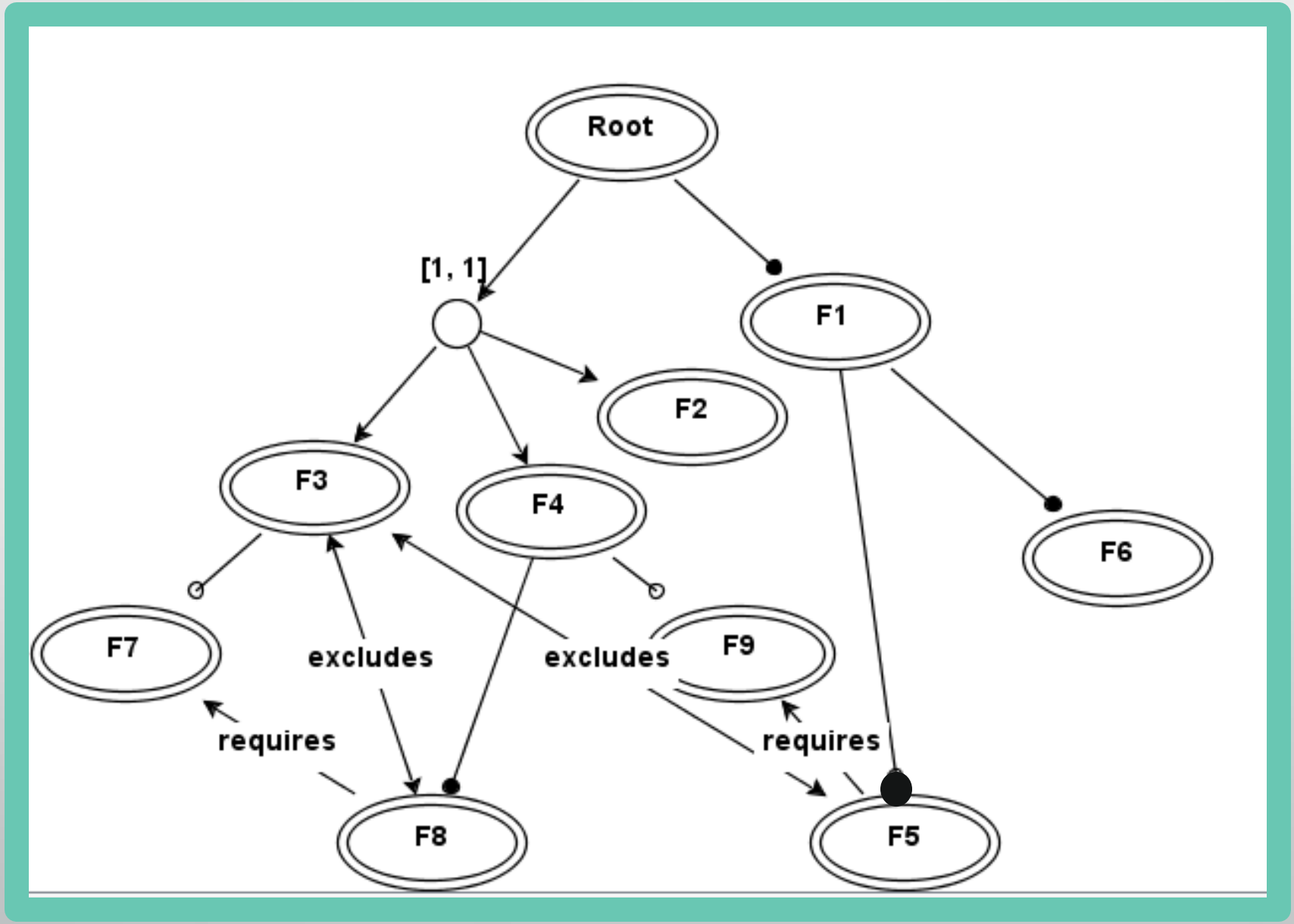


F5 is a dead feature

All the “verification constraint” for the different verification criteria

Defect	Verification constraint
Void model	It is not required since the constraint program is already *Unsatisfiable*.
False PLM	It is not required. This defect is solved if the defects regarding dead and false optional elements are fixed.
Dead elements	A verification constraint must be created to force that the potential dead element to analyze belongs to at least one product of the PL. For instance, to analyze whether F5 is dead of not, we add the verification constraint: F5 #= 1
False optional element	A verification constraint must be created to force that the optional elements take the 0 value (i.e., are are not selected). For instance, to analyze whether the optional feature F5 is really optional, the verif. constraint is: F5 #= 0
Redundancies	The verification constraint must represent the negation of the potential redundant dependency to be analyzed. For instance, to analyze whether the exclusion dependency between F3 and F8 is redundant or not, the verification constraint is: 1 – (F3 * F8 #= 0)

Homework: verify if this evolved model is void or not



Now we have the PLM in GNU Prolog.
To compile it: `consult ('nameOfTheFile.pl').`
To verify that the PLM is not void: `pl(L).`

```
pl(L):-
L=[Root, F1, F2, F3, F4, F5, F6, F7, F8, F9],
fd_domain(L, 0, 1),
Root #= 1,
Root  #= F1,
F1  #= F6,
F4  #= F8,
F1 #= F5,
F4 #>= F9,
F3 #>= F7,
F8 #==> F7,
F5 #==> F9,
F3 * F8 #= 0,
F3 * F5 #= 0,
Root #= F2 + F3 + F4,
fd_labeling(L).
```

Root

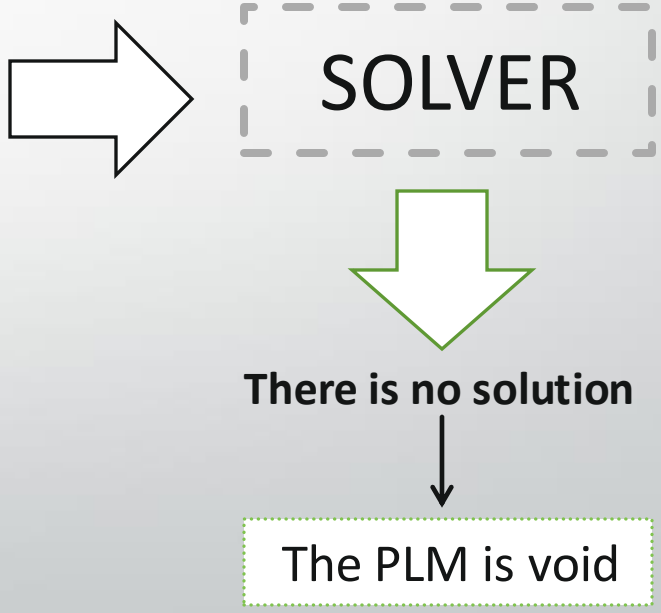
Mandatory

Optional

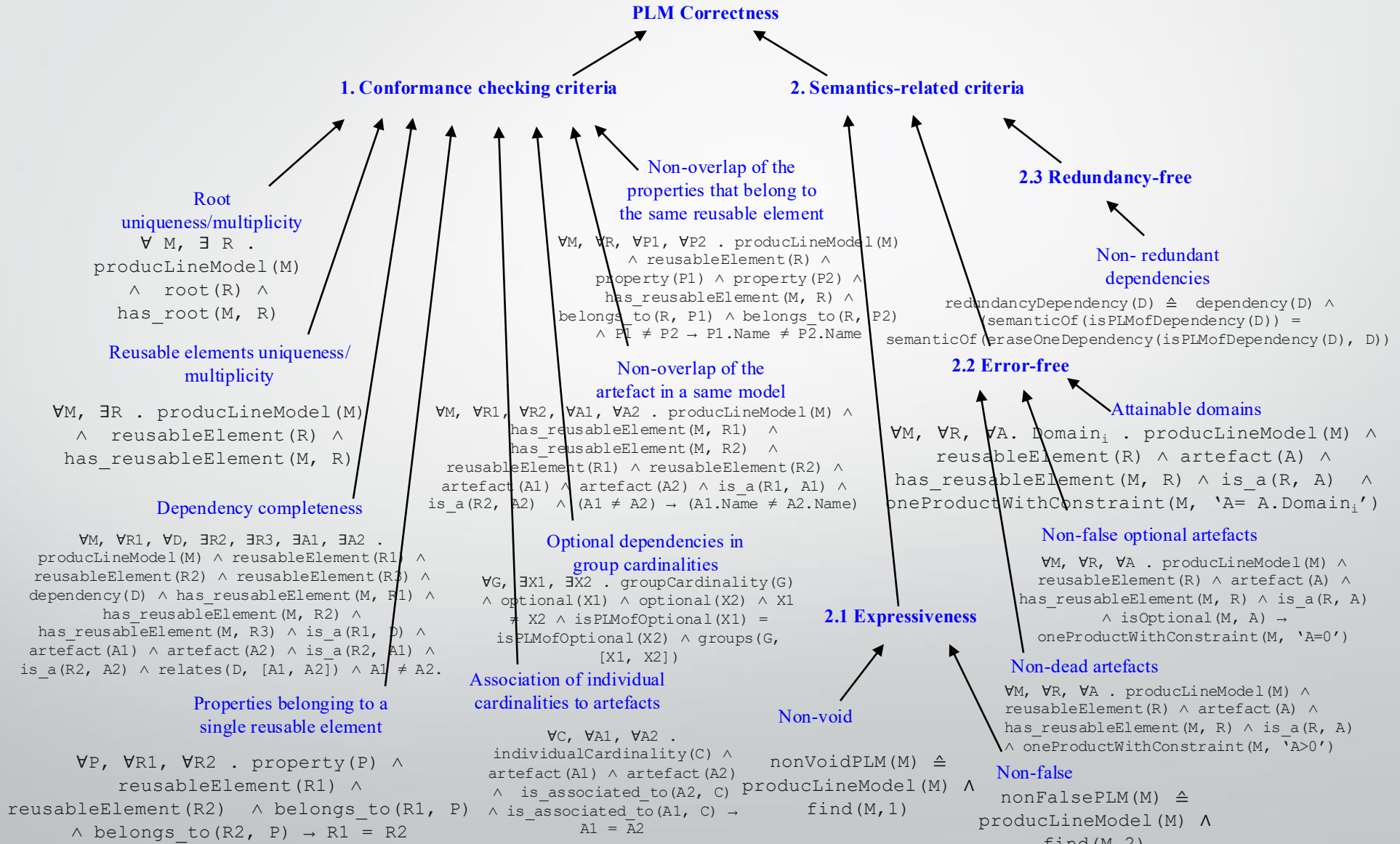
Requires

Excludes

Group cardinality

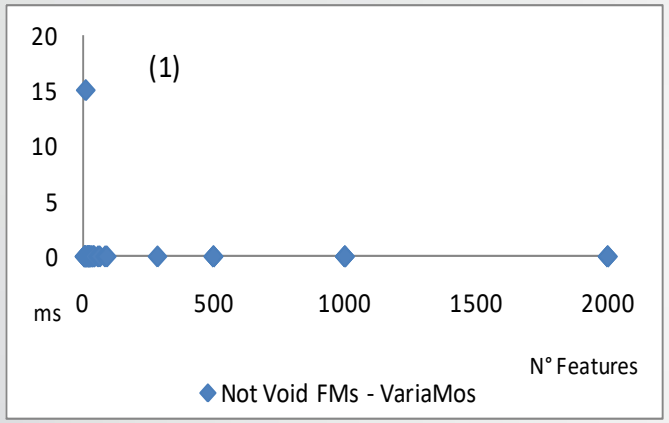


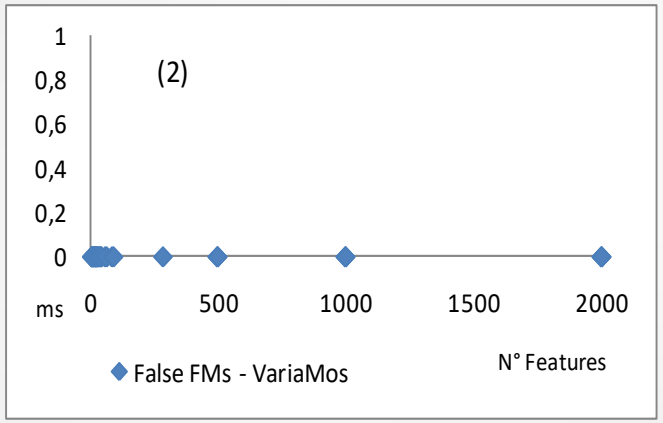
Typology of verification criteria

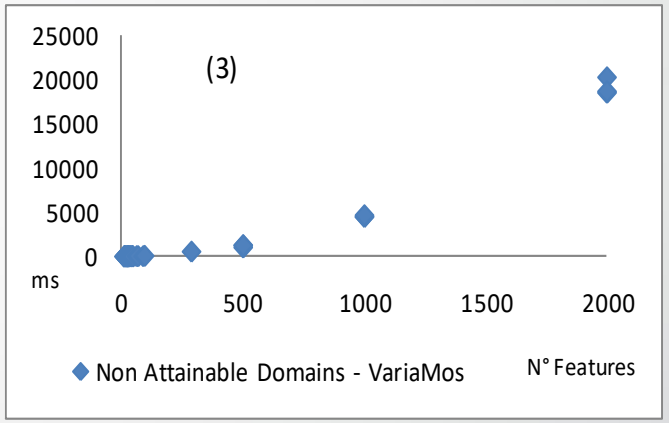


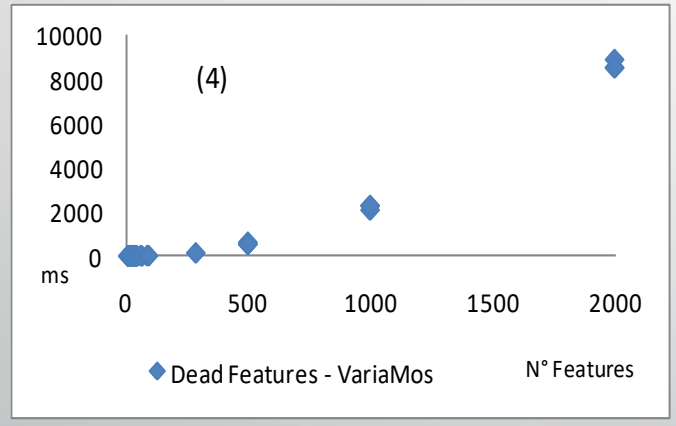
Semantics-related verification.

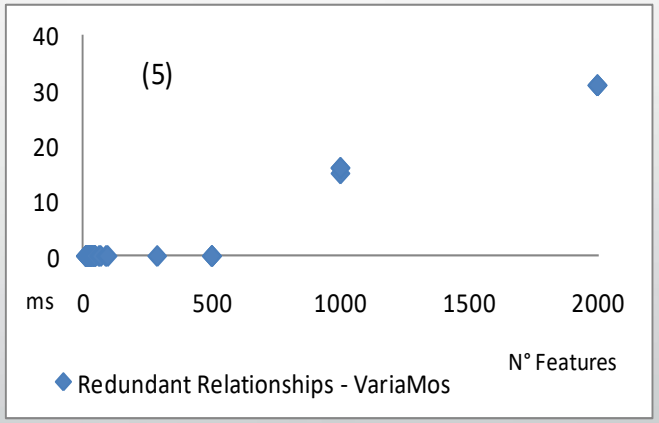
Validation with 43 FMs up to 2000 features

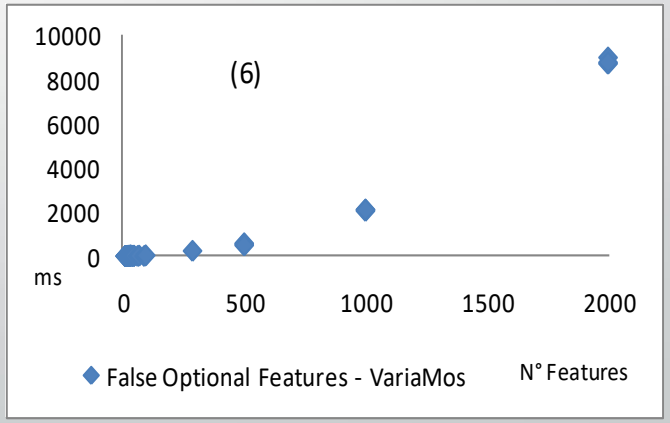








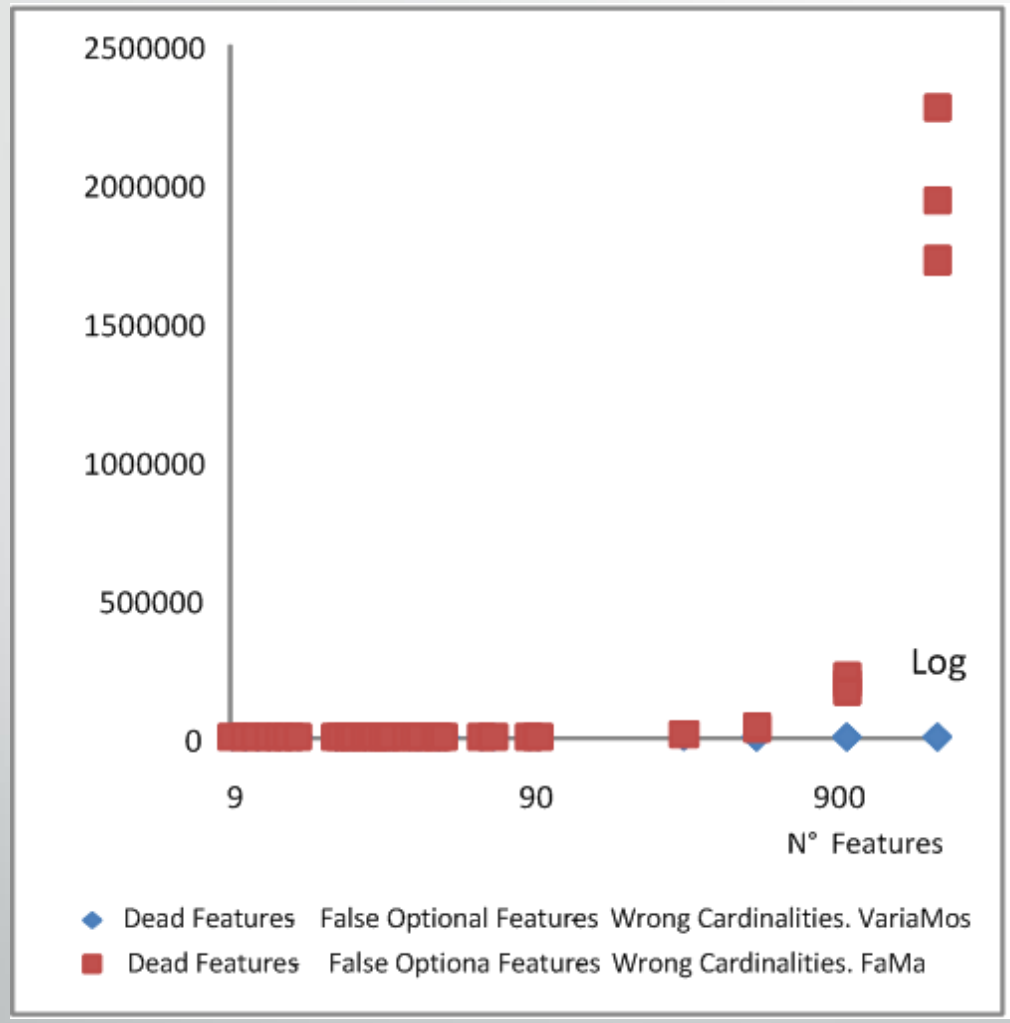


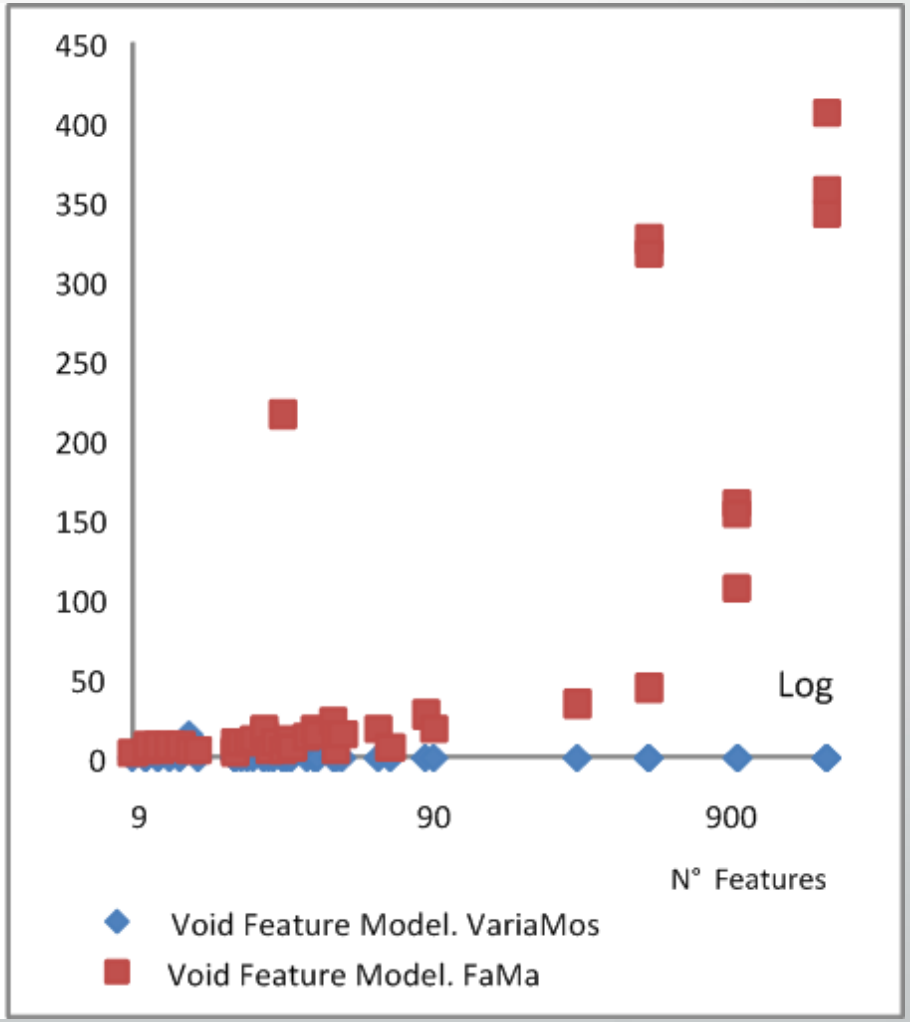


Execution time(ms)

Number of features

Scalability: VariaMos vs. FaMa





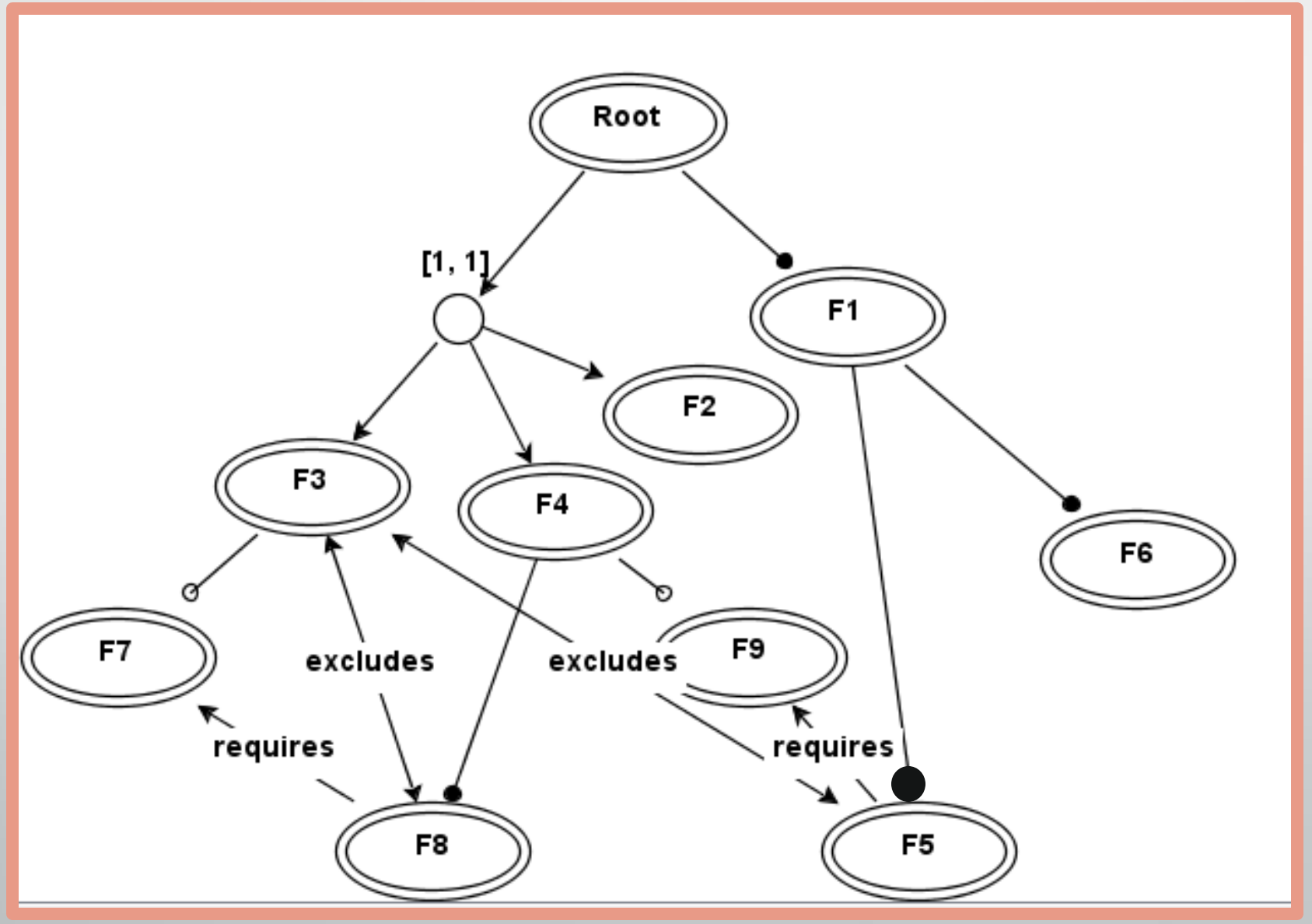
Execution time (ms)

Log 10 (Number of features)

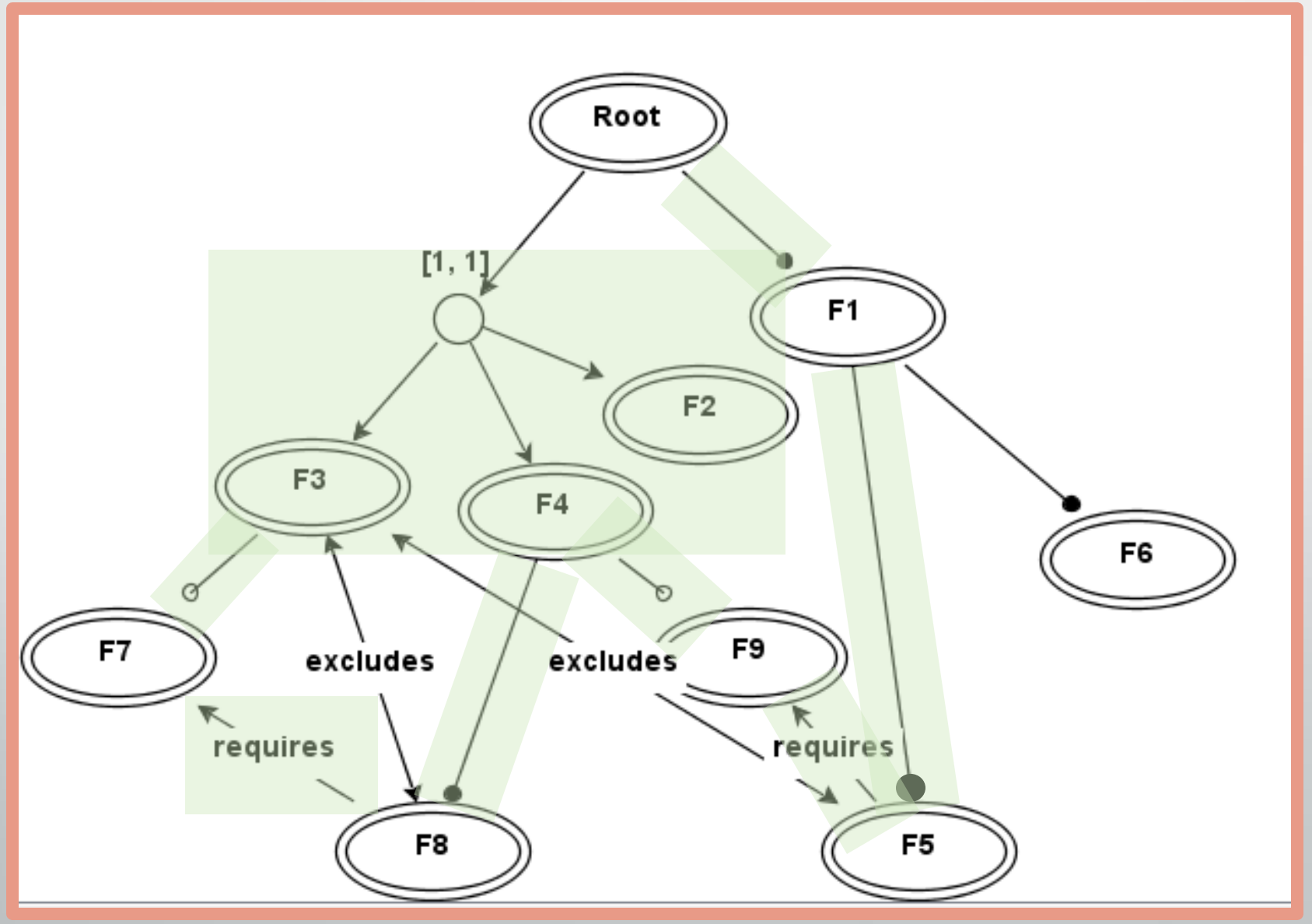
Diagnosis & fixing

Find the source of defects in the PLMs and fix these defects

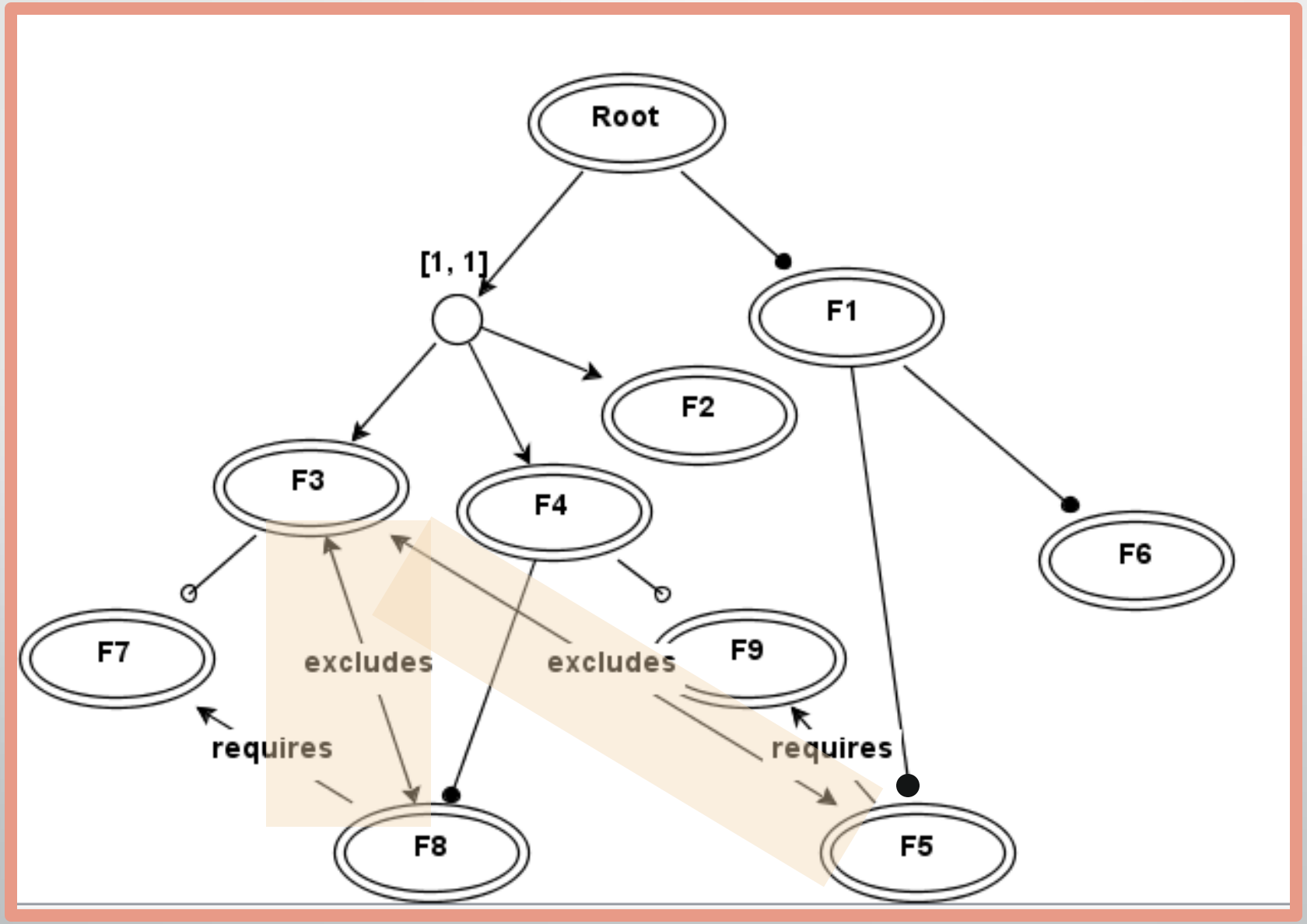
Let's **diagnose**, by manual inspection, **why** the model is void



Let's **diagnose**, by manual inspection, **why** the model is void



Let's **try to fix** the defect which make the model void: **Erasing some constraints (e.g., the exclusions)?**



However, erasing these two constraints is not enough.
To compile it: `consult ('nameOfTheFile.pl').`
To verify that the PLM is not void: `pl(L).`

```
pl(L):-
L=[Root, F1, F2, F3, F4, F5, F6, F7, F8, F9],
fd_domain(L, 0, 1),
Root #= 1,
Root  #= F1,
F1  #= F6,
F4  #= F8,
F1 #= F5,
F4 #>= F9,
F3 #>= F7,
F8 #==> F7,
F5 #==> F9,
F3 * F8 #= 0,
F3 * F5 #= 0,
Root #= F2 + F3 + F4,
fd_labeling(L).
```

Root

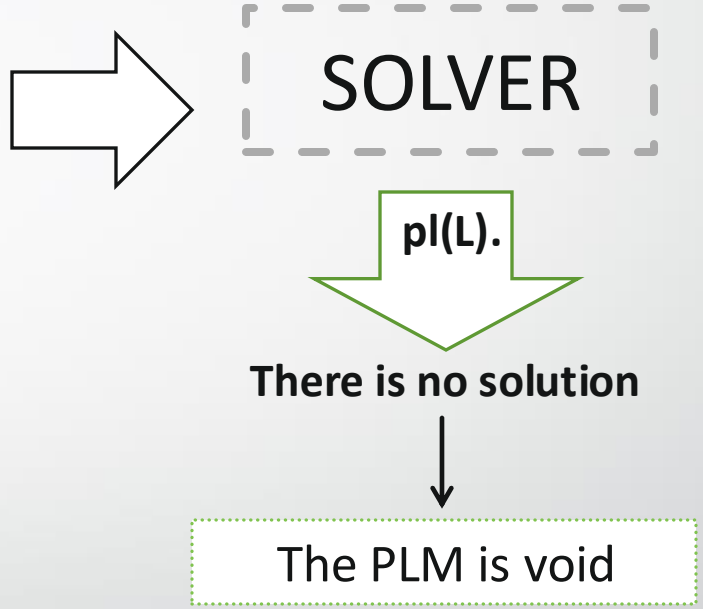
Mandatory

Optional

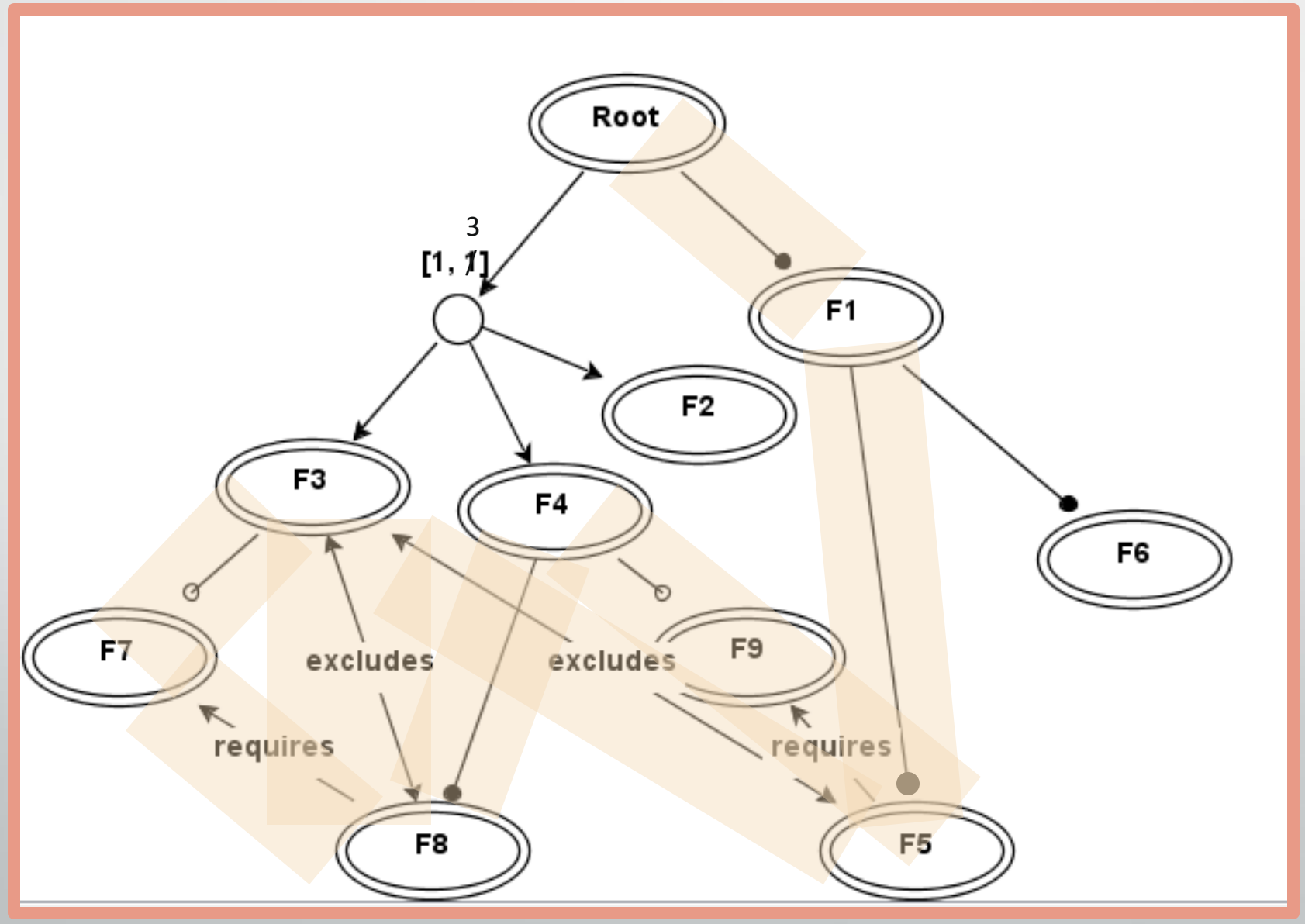
Requires

Excludes

Group cardinality



Let's try to **fix** the defect which make the model void by **relaxing the cardinality [1..1] to [1..3], and then diagnosing it again**



And then: `consult ('nameOfTheFile.pl').`
To verify that the PLM is not void: `pl(L).`

```
pl(L):-
L=[Root, F1, F2, F3, F4, F5, F6, F7, F8, F9],
fd_domain(L, 0, 1),
Root #= 1,
Root  #= F1,
F1  #= F6,
F4  #= F8,
F1 #= F5,
F4 #>= F9,
F3 #>= F7,
F8 #==> F7,
F5 #==> F9,
F3 * F8 #= 0,
F3 * F5 #= 0,
Root #=< F2 + F3 + F4,
F2 + F3 + F4 #=< Root*3,
fd_labeling(L).
```

Root

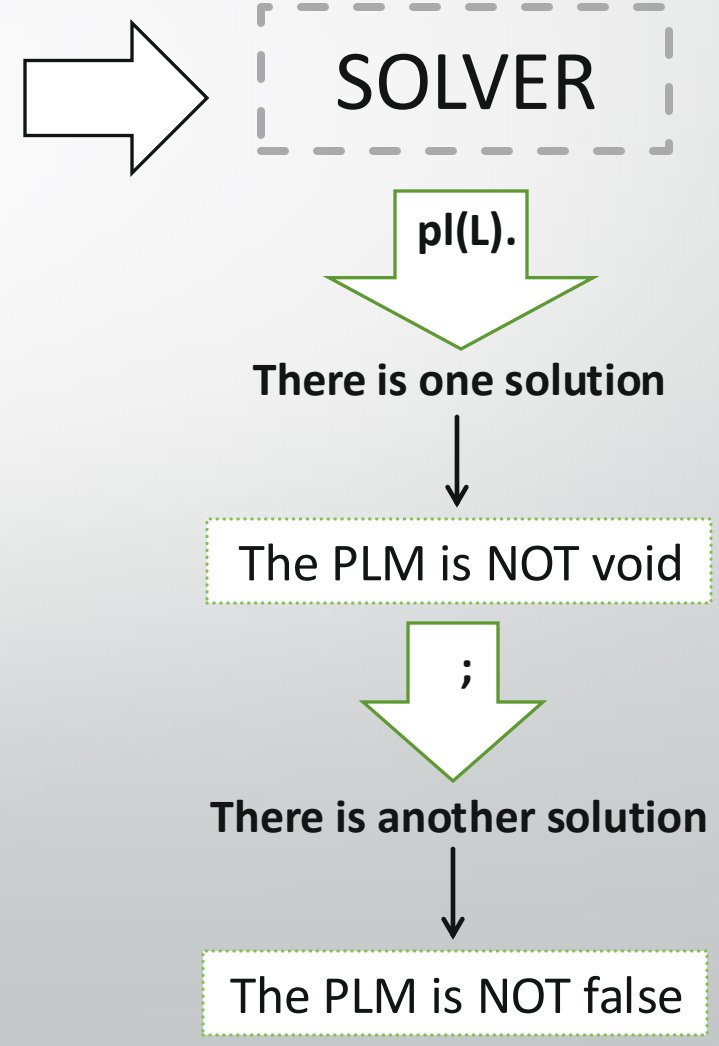
Mandatory

Optional

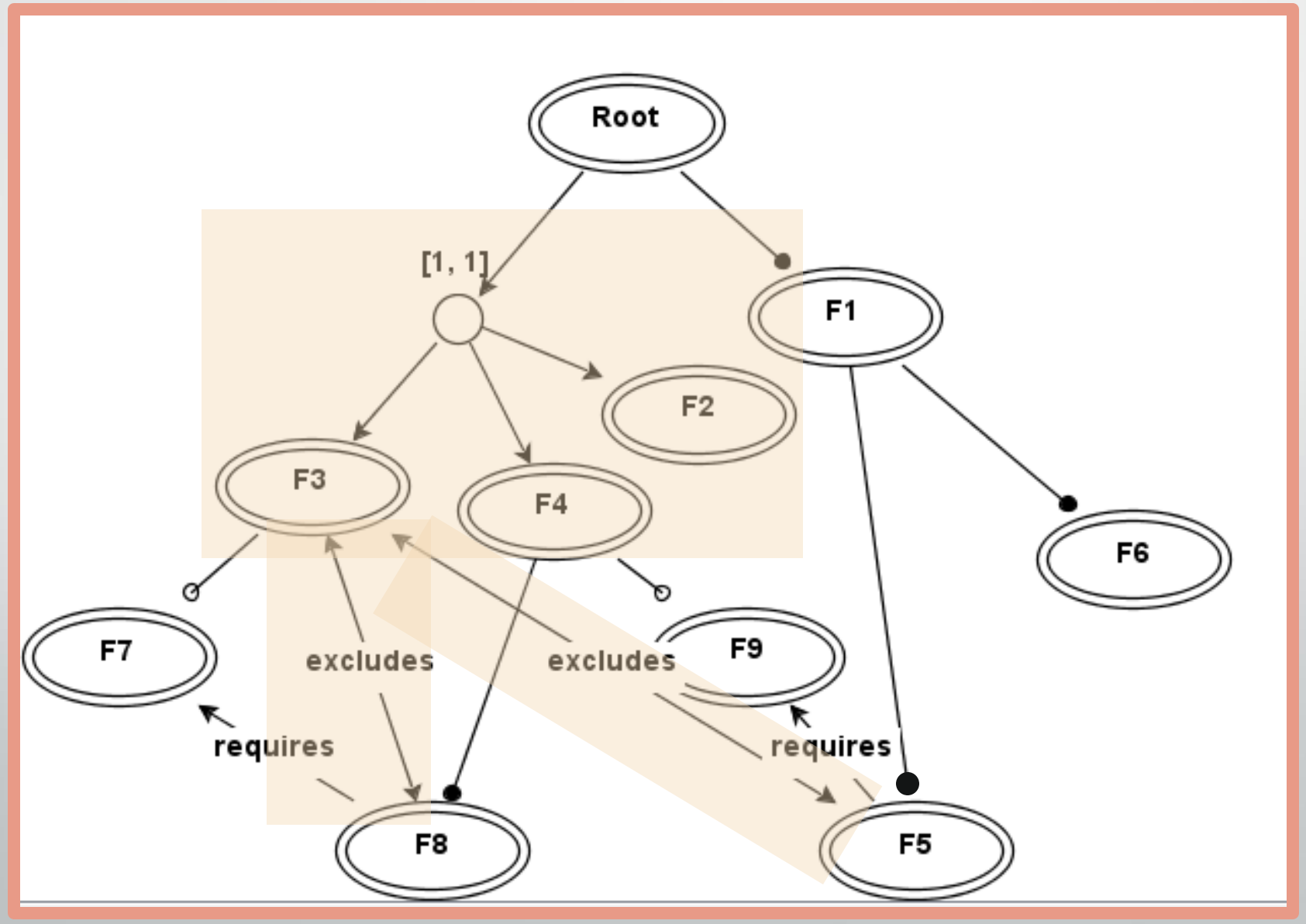
Requires

Excludes

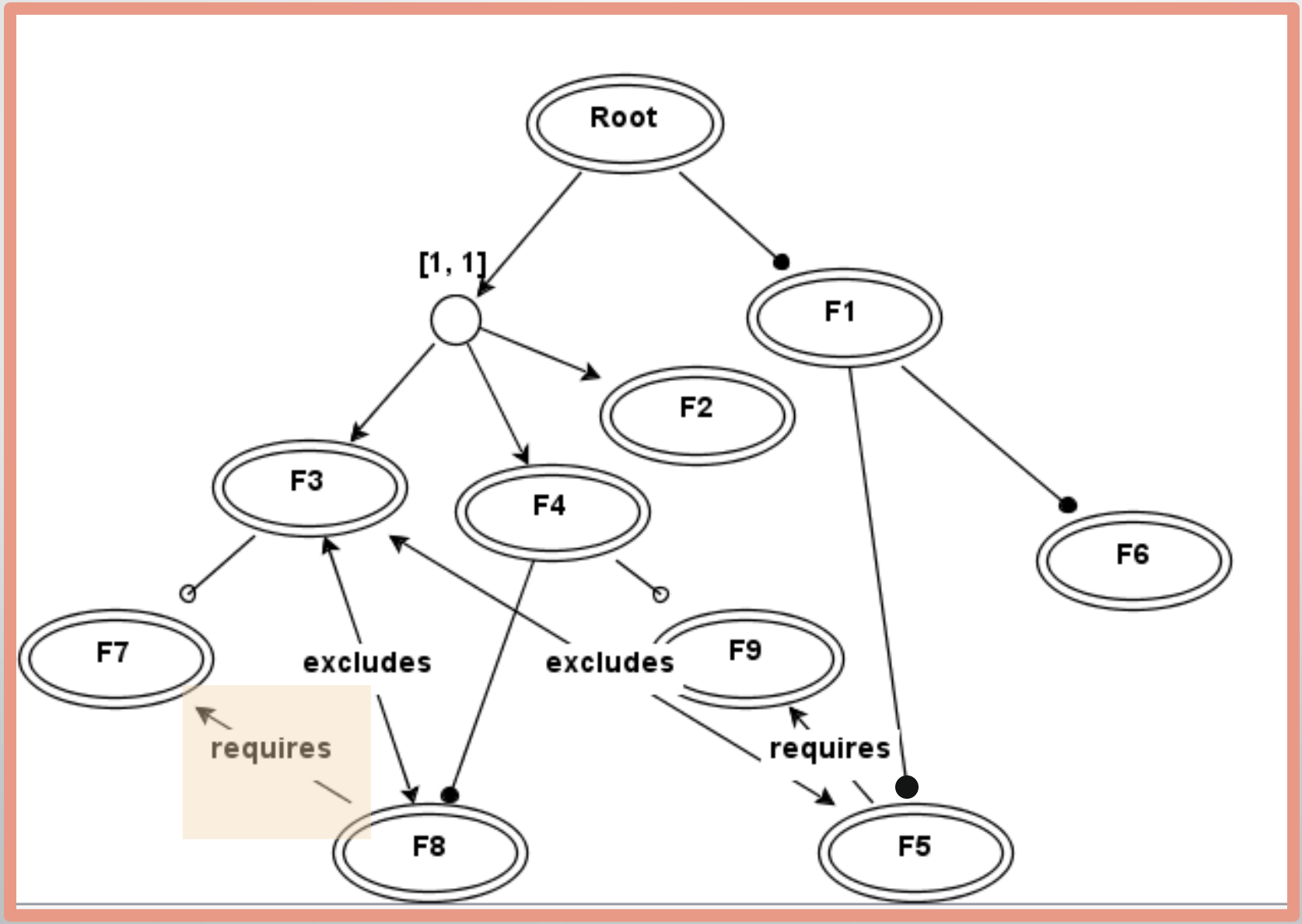
Group cardinality



Let's **show a sub-set of constraints (MUS) that can be used to fix the model (erasing but also relaxing them)**



Exercise: Try to **fix** the defect which make the model void: **erasing a MUS (e.g., one requires)**



To compile it: `consult('nameOfTheFile.pl').`
To verify that the PLM is not void: `pl(L).`

```
pl(L):-
L=[Root, F1, F2, F3, F4, F5, F6, F7, F8, F9],
fd_domain(L, 0, 1),
Root #= 1,
Root  #= F1,
F1  #= F6,
F4  #= F8,
F1 #= F5,
F4 #>= F9,
F3 #>= F7,
F8 #==> F7,
F5 #==> F9,
F3 * F8 #= 0,
F3 * F5 #= 0,
Root #= F2 + F3 + F4,
fd_labeling(L).
```

Root

Mandatory

Optional

Requires

Excludes

Group cardinality



pl(L).

There is one solution

S = [1,1,0,0,1,1,1,0,1,1]

The PLM is NOT void

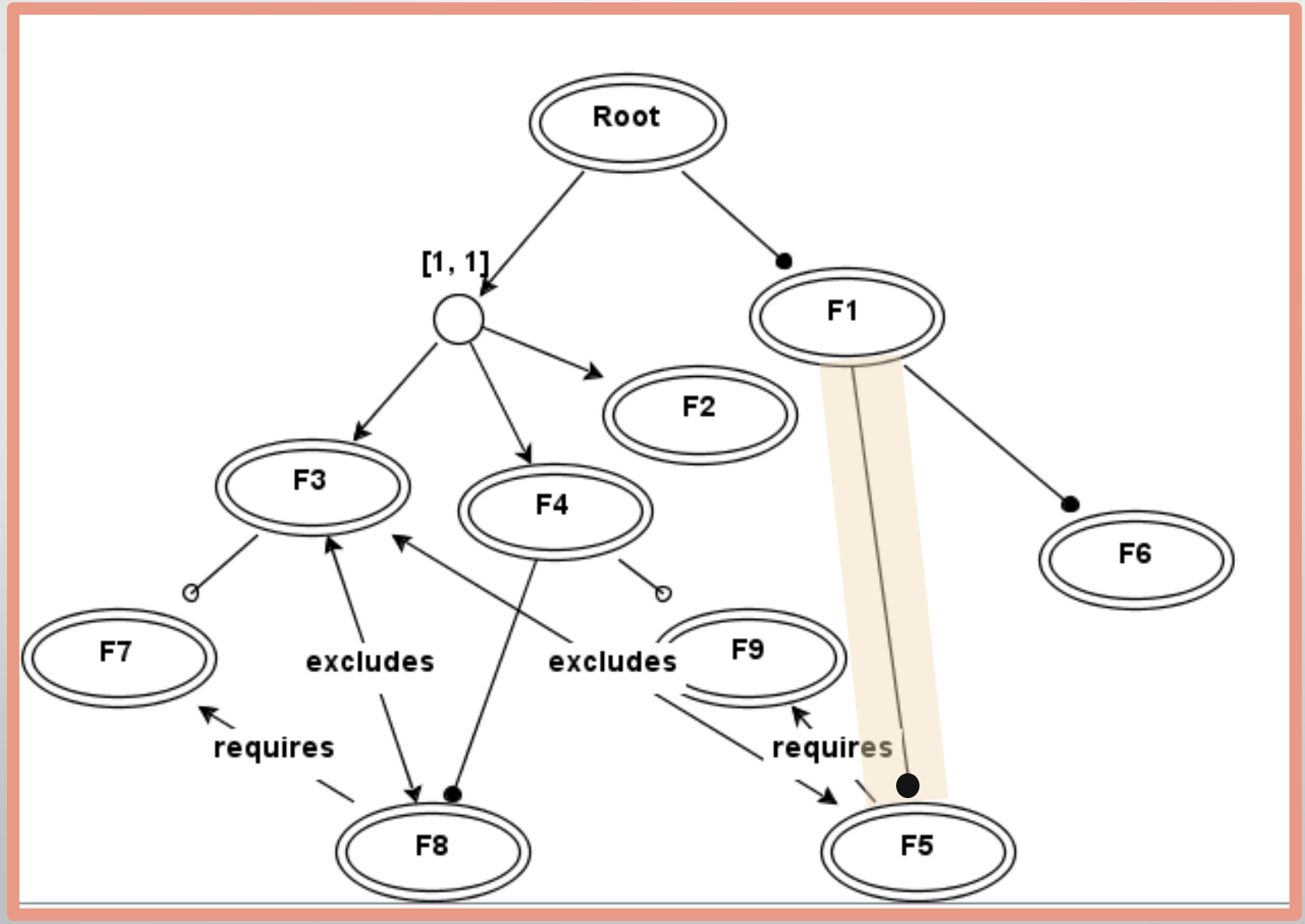
;

There is NO more solutions

The PLM is false

And therefore this solution is not good in PLE

Let's **fix** the defect which make the model void (using another MUS) : **Relaxing a dependency**



To compile it: `consult('nameOfTheFile.pl').`
To verify that the PLM is not void: `pl(L).`

```
pl(L):-
L=[Root, F1, F2, F3, F4, F5, F6, F7, F8, F9],
Fd_domain(L, 0, 1),
Root #= 1,
Root  #= F1,
F1  #= F6,
F4  #= F8,
F1 #>= F5,
F4 #>= F9,
F3 #>= F7,
F8 #==> F7,
F5 #==> F9,
F3 * F8 #= 0,
F3 * F5 #= 0,
Root #= F2 + F3 + F4,
fd_labeling(L).
```

Root

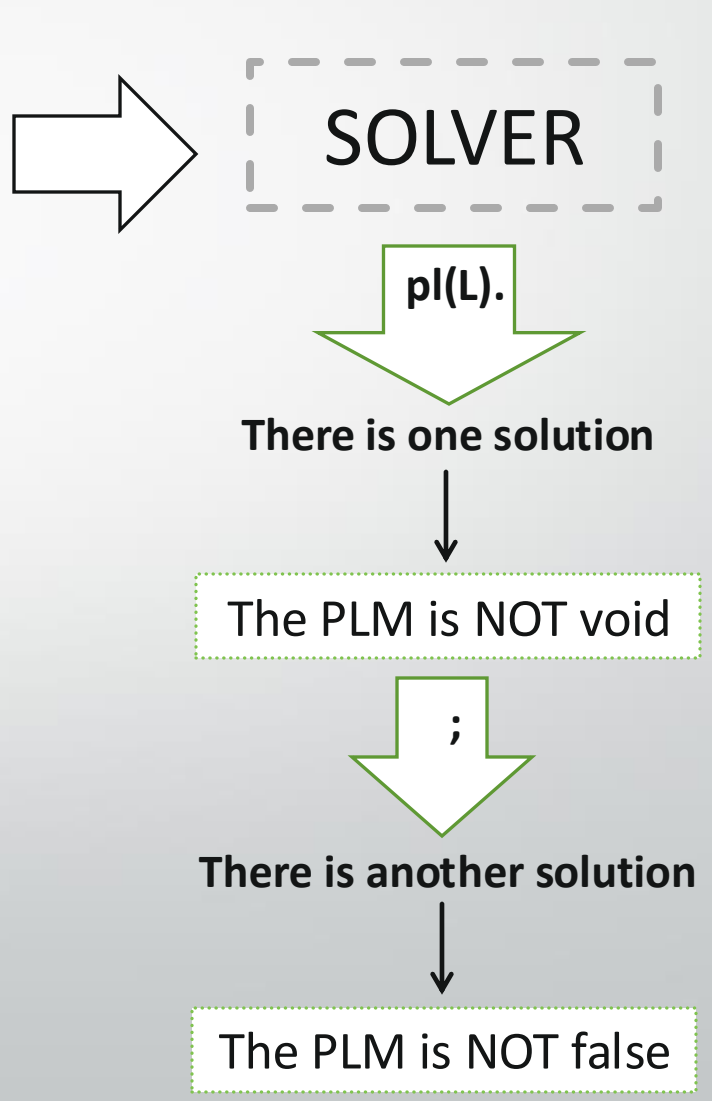
Mandatory

Optional

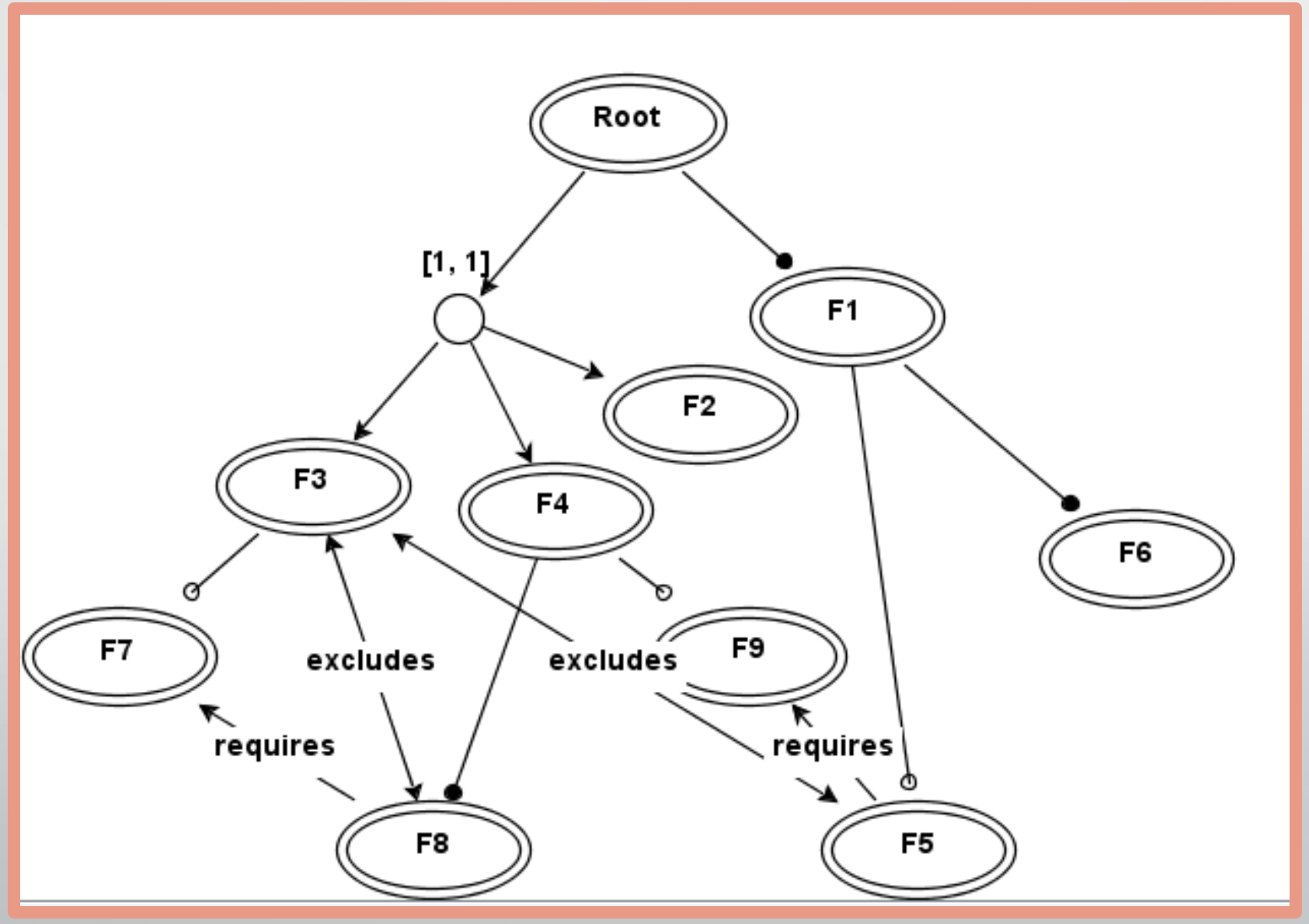
Requires

Excludes

Group cardinality



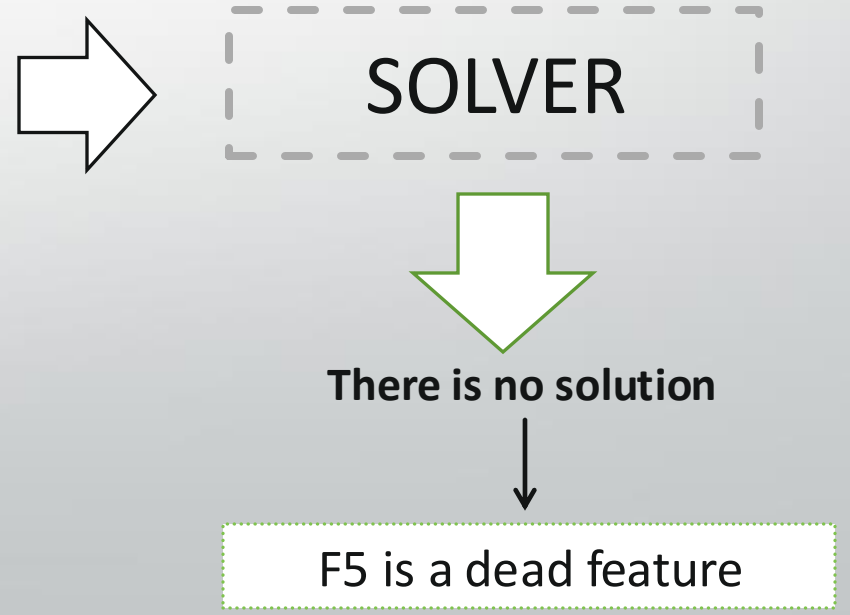
Now, let's **verify if F5 is a dead feature**



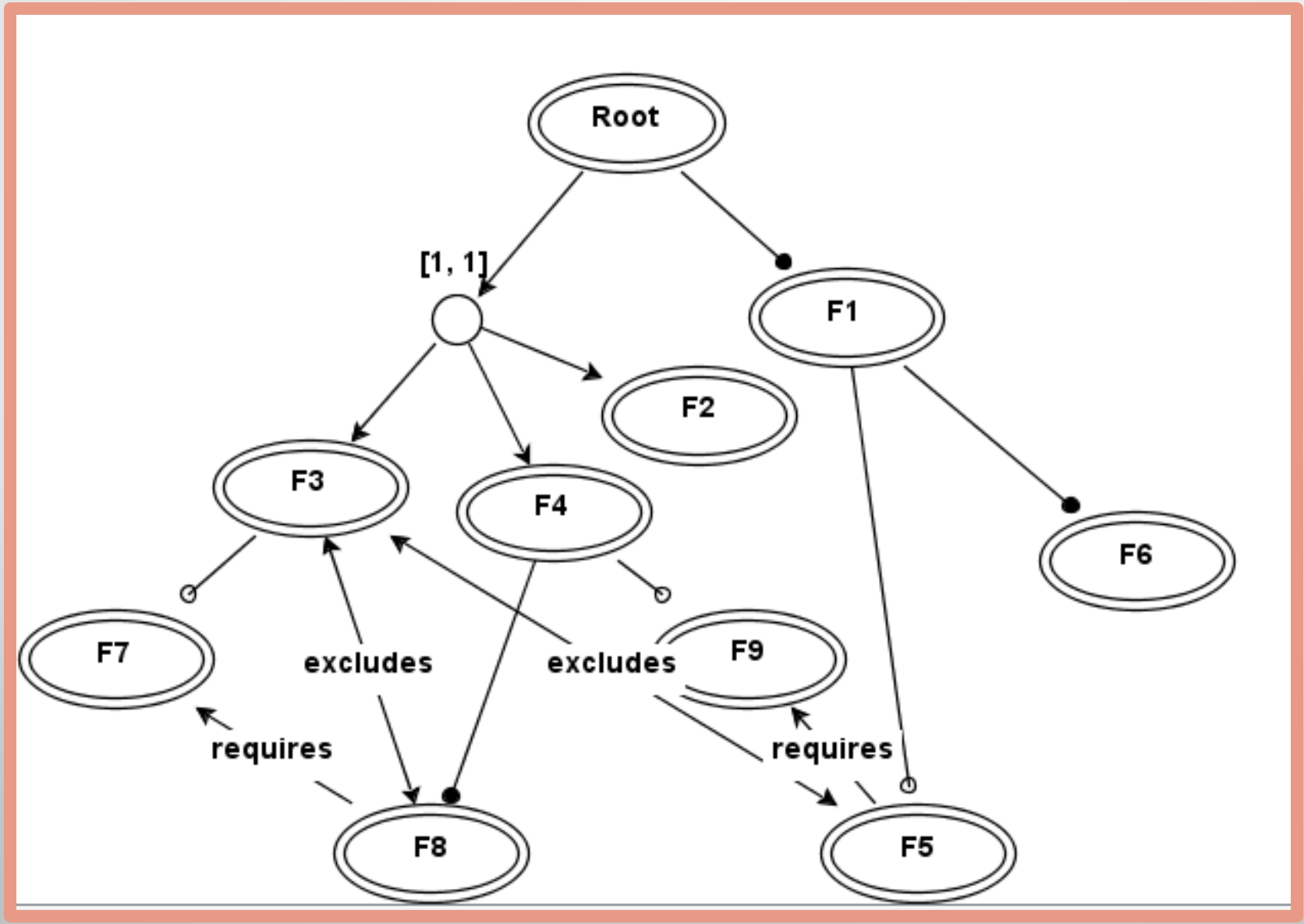
Then, to verify that a certain element (e.g., F5) is not dead, we add the "verification constraint" F5 #= 1 into the CP and we should re-compile it: `consult('nameOfTheFile.pl').`
Or as an external constraint : `F5 #= 1, pl(L).`

```
pl(L):-
L=[Root, F1, F2, F3, F4, F5, F6, F7, F8, F9],
Fd_domain(L, 0, 1),
F5 #= 1,
Root #= 1,
F4  #= F8,
Root  #= F1,
F1  #= F6
F1 #>= F5,
F4 #>= F9,
F3 #>= F7,
F8 #==> F7,
F5 #==> F9,
F3 * F8 #= 0,
F3 * F5 #= 0,
Root = F2 + F3 + F4,
fd_labeling(L).
```

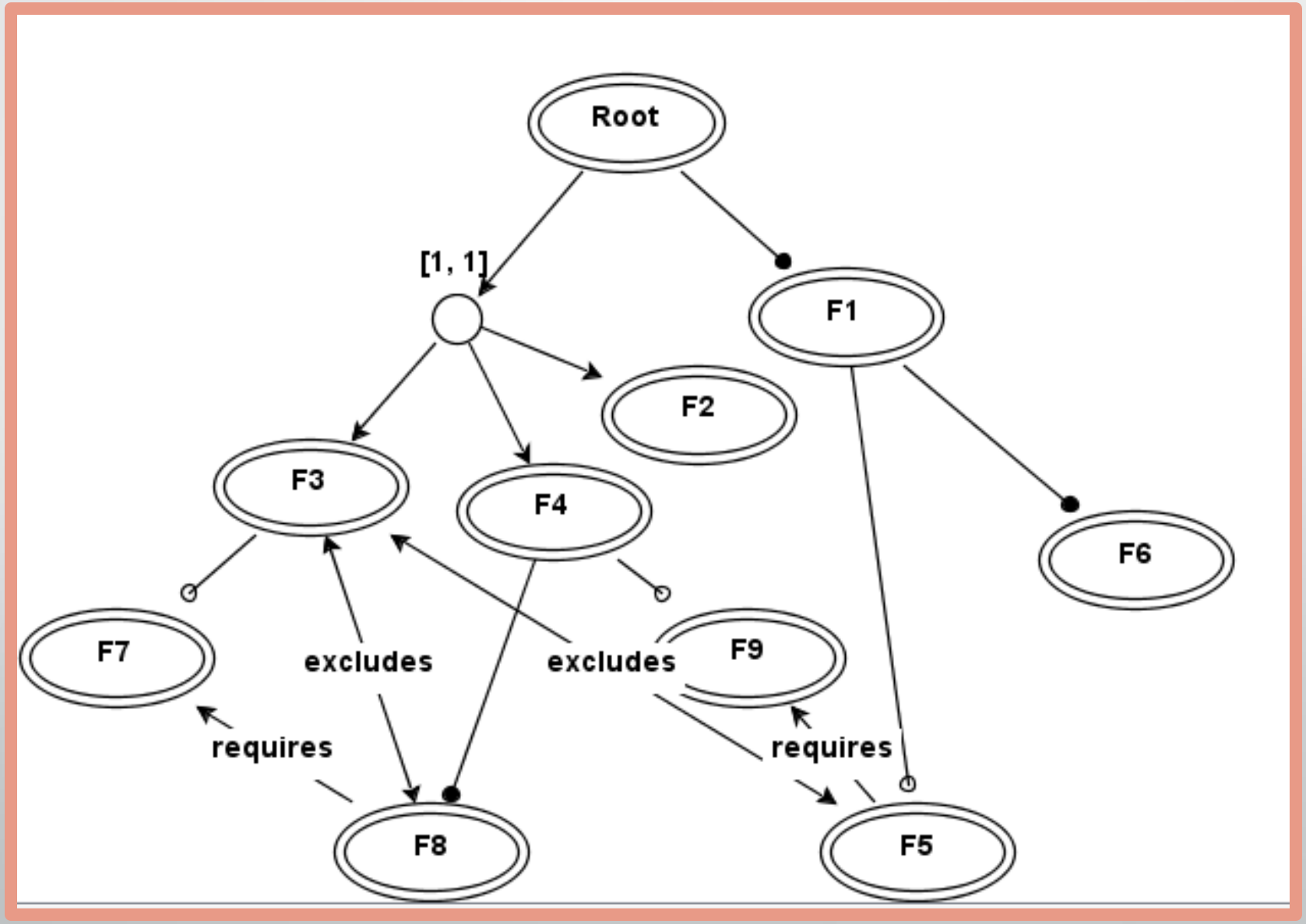
The verification constraint (to verify whether F5 is dead or not) inside the constraint program



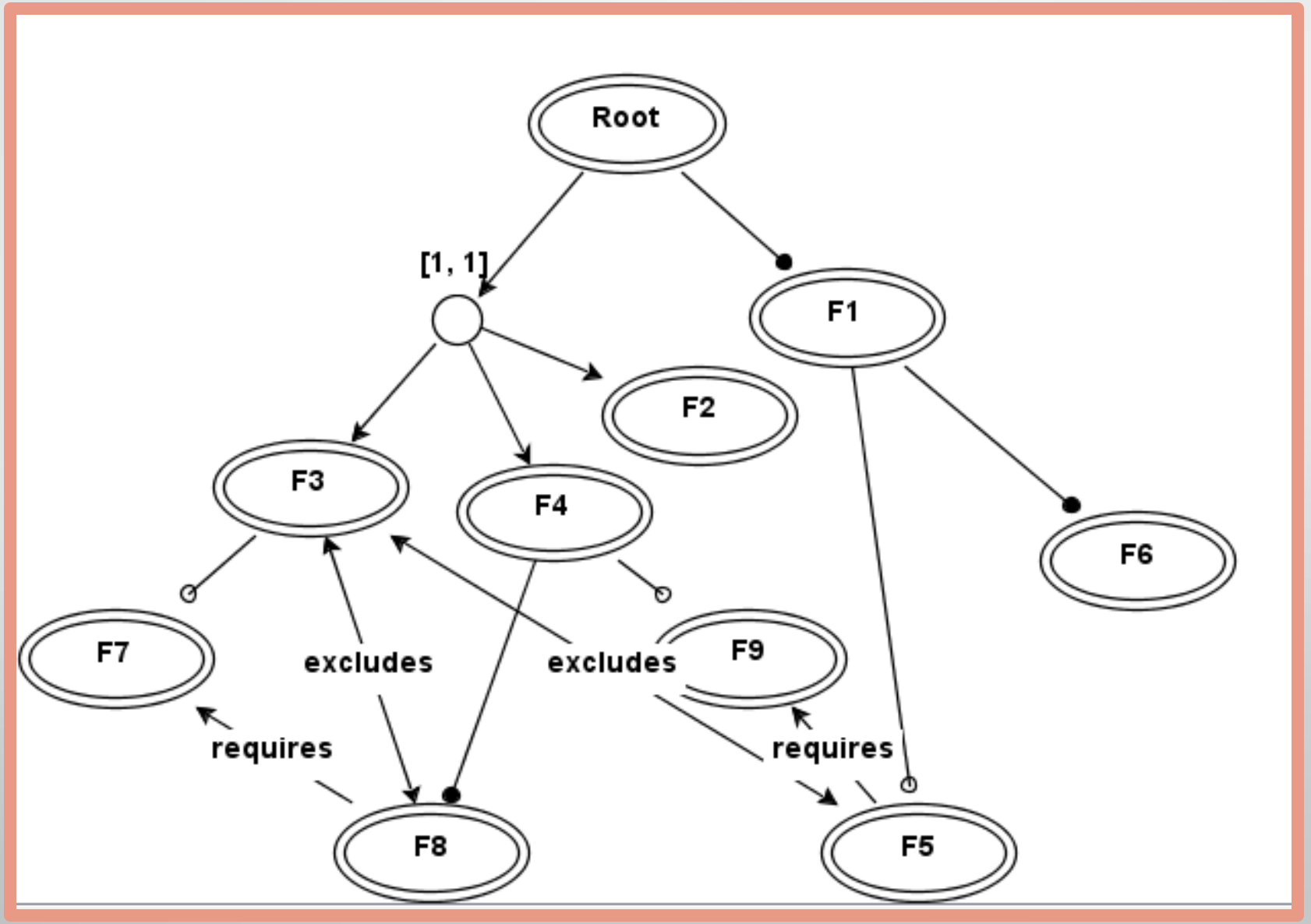
Exercise: **diagnose**, by manual inspection, **why** F5 is a dead feature? **Why** F4, F8 and F9 are also dead?



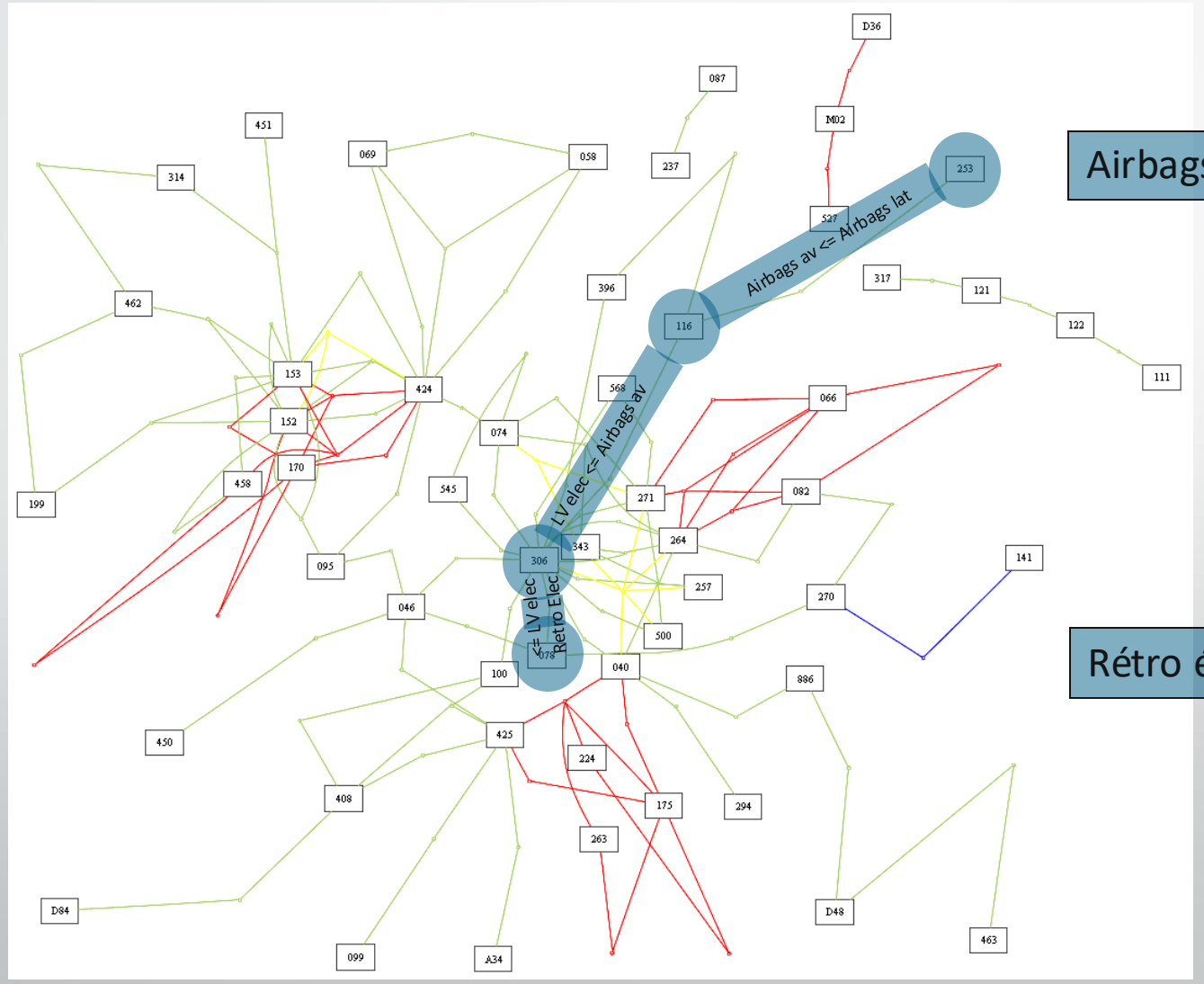
Exercise: propose two solutions to **fix** the "F5 dead feature" error



Exercise: Diagnose and fix the rest of defects in this model



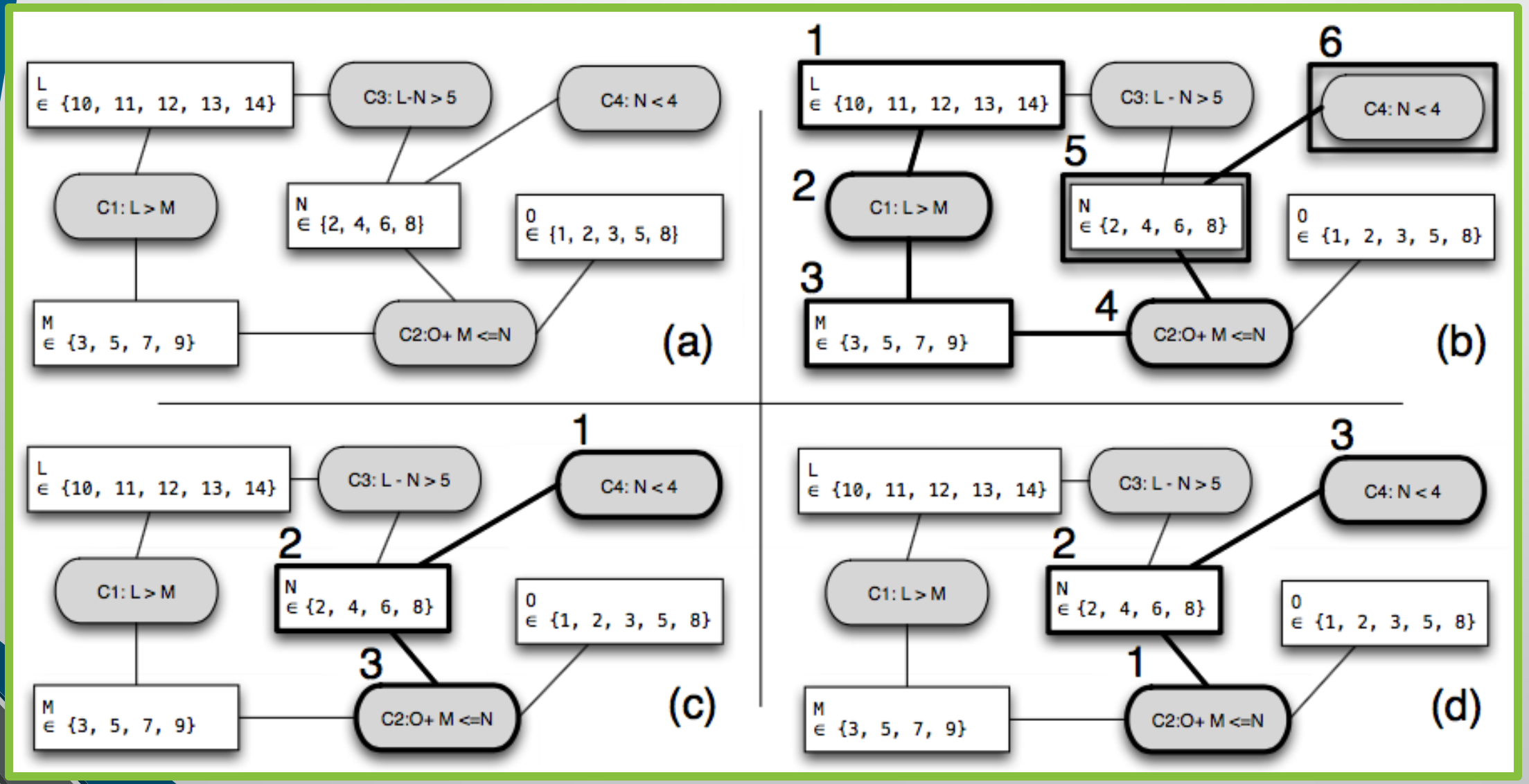
Why are **side airbags** incompatible with **exterior rear-view mirrors** in this Renault product line model excerpt?



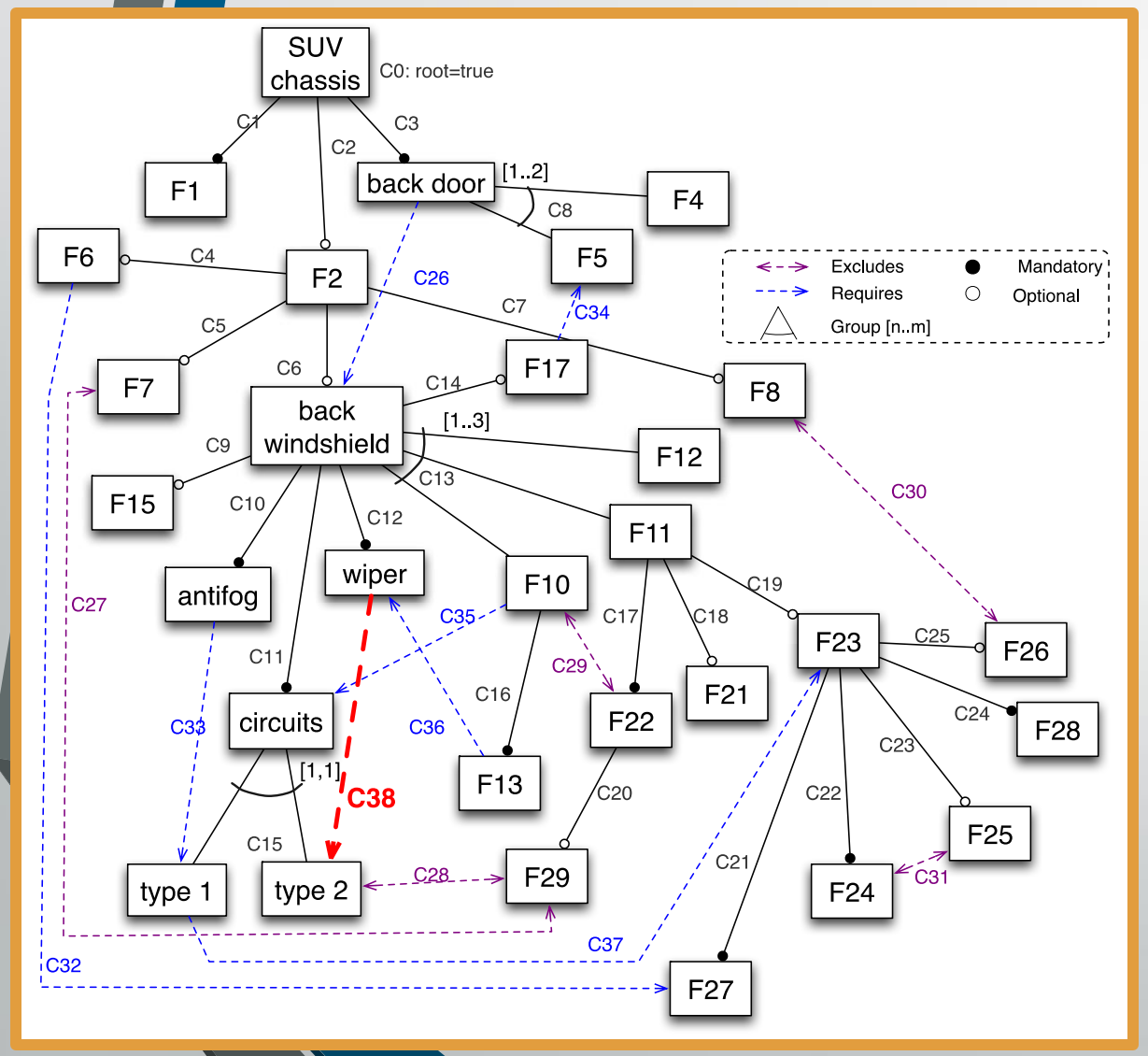
Airbags latéraux

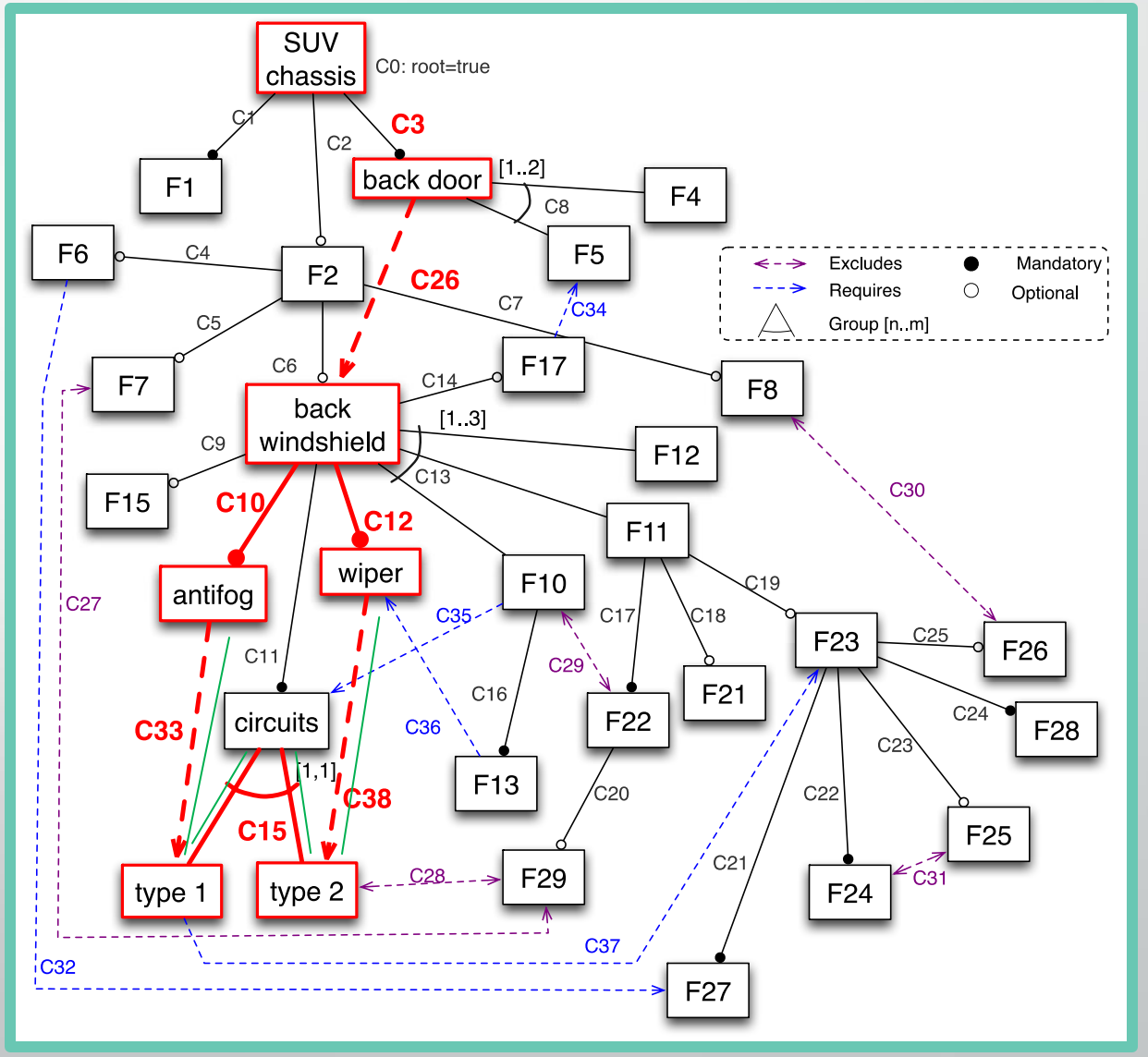
Rétro électriques

MEthod to Diagnose InConsistent PLMs



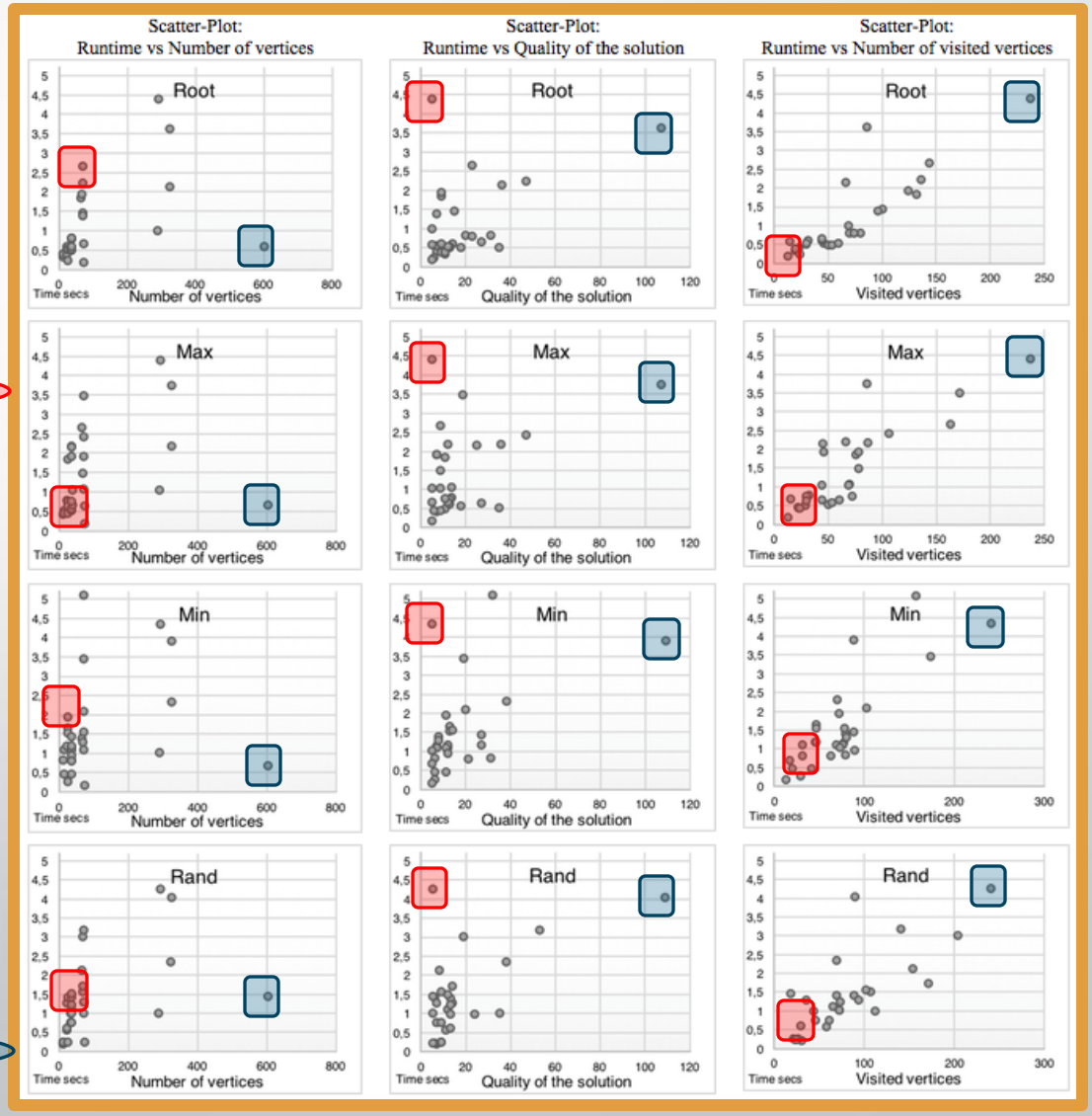
Results of the experimentation of Medic-PLM





Results of the experimentation of Medic-PLM

Model ID	CSP Vars	CSP Cons.	Graph Vertices	Graph Edges	Accuracy	Max Iter.	Time in seconds Root	Time Max	Time Min	Time Rand	Quality of the solution |CL|: size of the conflict list, #C: number of constraints in CL Root |CL|	Root #C	Max |CL|	Max #C	Min |CL|	Min #C	Rand |CL|	Rand #C
FM1	11	13	24	30	S	3	0.23	0.44	0.26	0.24	6	4	6	4	6	4	6	4
FM2	5	6	11	12	S	3	0.41	0.43	0.82	0.20	7	4	7	4	6	4	7	4
FM3	5	7	12	13	S	3	0.33	0.51	0.47	0.25	11	6	11	6	11	6	9	5
FM4	5	7	12	13	S	3	0.39	0.45	1.10	0.25	9	5	9	5	12	7	9	5
FM5	10	12	22	25	S	3	0.60	0.78	1.18	1.28	14	8	14	8	12	7	14	8
FM6	10	13	23	26	S	3	0.39	1.85	1.95	0.57	11	6	11	6	11	6	11	6
FM7	10	13	23	26	S	3	0.51	0.74	1.65	0.61	13	7	13	7	13	7	13	7
FM8	10	14	24	31	S	3	0.53	0.63	1.54	1.41	13	7	13	7	13	7	13	7
FM9	15	19	34	39	S	3	0.54	1.92	1.15	0.76	7	4	7	4	7	4	7	4
FM10	15	19	34	43	S	3	0.49	0.53	0.83	1.01	34	21	34	21	34	18	34	21
FM11	15	20	35	42	S	3	0.49	0.57	0.47	1.42	18	10	18	10	6	4	6	4
FM12	15	21	36	43	S	3	0.52	0.66	1.10	1.11	12	7	12	7	11	7	12	7
FM13	15	20	35	44	S	3	0.81	2.18	0.95	1.51	20	11	12	7	12	7	12	7
FM14	15	21	36	43	S	3	0.60	1.04	0.80	0.76	9	5	9	5	21	12	9	5
FM15	15	20	35	45	S	3	0.79	0.76	1.44	1.23	23	13	11	6	27	15	13	7
FM16	15	20	35	45	S	3	0.81	2.16	1.18	0.99	31	18	25	14	27	15	24	13
FM17	30	34	64	82	S	3	1.84	2.67	1.39	2.13	9	5	9	5	8	5	8	5
FM18	30	37	67	82	S	3	1.94	1.49	1.29	1.56	9	5	9	5	8	5	9	5
FM19	30	39	69	82	S	4	2.66	3.49	3.45	3.01	23	13	19	11	19	11	19	11
FM20	30	39	69	84	S	3	1.45	1.07	1.55	1.72	15	8	14	8	14	8	14	8
FM21	30	41	71	84	S	3	0.66	0.64	2.09	0.99	27	15	27	15	20	11	13	7
FM22	30	40	70	86	S	3	1.39	1.92	1.10	1.29	7	4	7	4	7	4	7	4
FM23	30	40	70	86	S	3	2.23	2.43	5.09	3.19	43	30	47	28	32	18	53	32
Ren1	32	41	73	78	S	3	0.18	0.18	0.18	0.24	5	3	5	3	5	3	5	3
Ren2	139	149	288	327	S	4	1.00	1.04	1.03	1.01	5	3	5	3	5	3	5	3
Ren3	139	153	292	331	S	4	4.38	4.40	4.34	4.26	5	3	5	3	5	3	5	3
Ren4	146	177	323	430	S	2	2.14	2.19	2.32	2.35	36	22	36	22	38	23	38	24
Ren5	146	178	324	431	S	3	3.62	3.74	3.91	4.03	107	62	107	62	109	63	109	63
Ren6	268	334	602	829	S	3	0.58	0.67	0.68	1.45	5	3	5	3	5	3	5	3



Domain analysis (research opportunities)

1. How to automatically discover (and guide the specification of) domain requirements and constraints?
2. How to connect domain requirements and their formal representations?
3. How to automatically transform natural language requirements into formal conceptual models (which languages?)?
4. How to make evolve domain requirements maintaining the consistency of the entire product line?
5. How to specify, represent and reason on domain non-functional requirements?
6. How to deal with automatic diagnosis and correction of product line models (from the problem and solution space perspectives)?